Lecture7 微服务设计

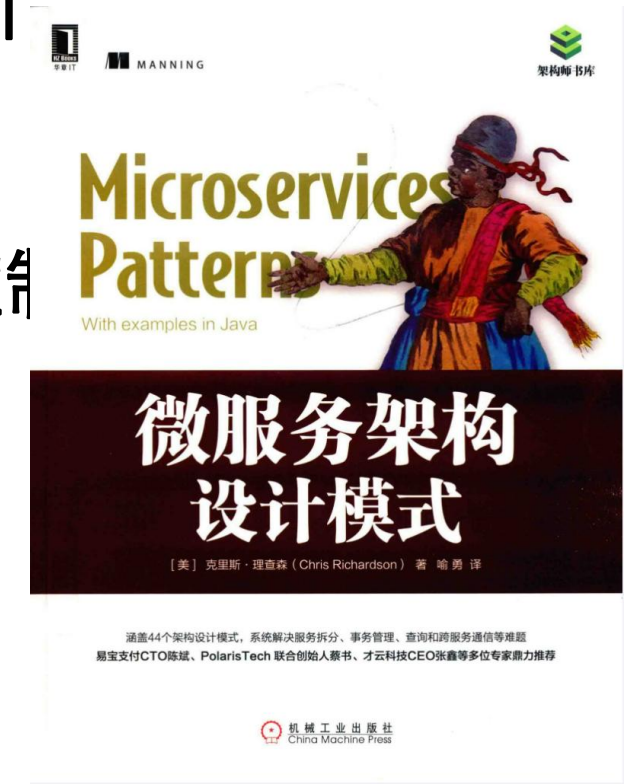


于晗

han__yu@sjtu.edu.cn

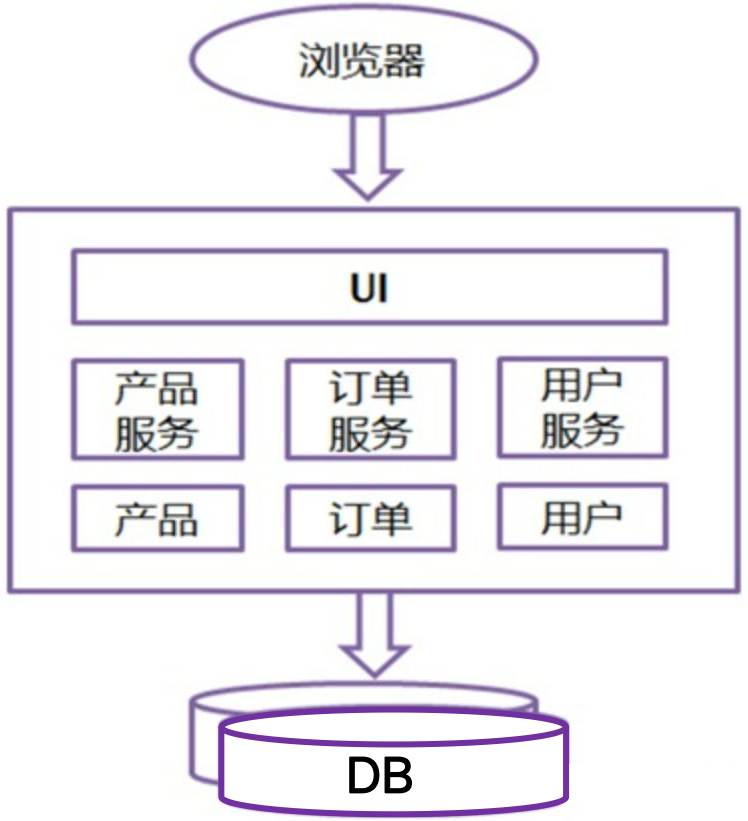
目录

- 1 微服务设计概述
- 2 微服务架构设计
  - 定义系统操作 – 定义业务服务 – 定义服务API
- 3 微服务架构下的进程间通信
  - 服务间通信 – 分布式事务 – 容错设计及流量控制
- 4 小结

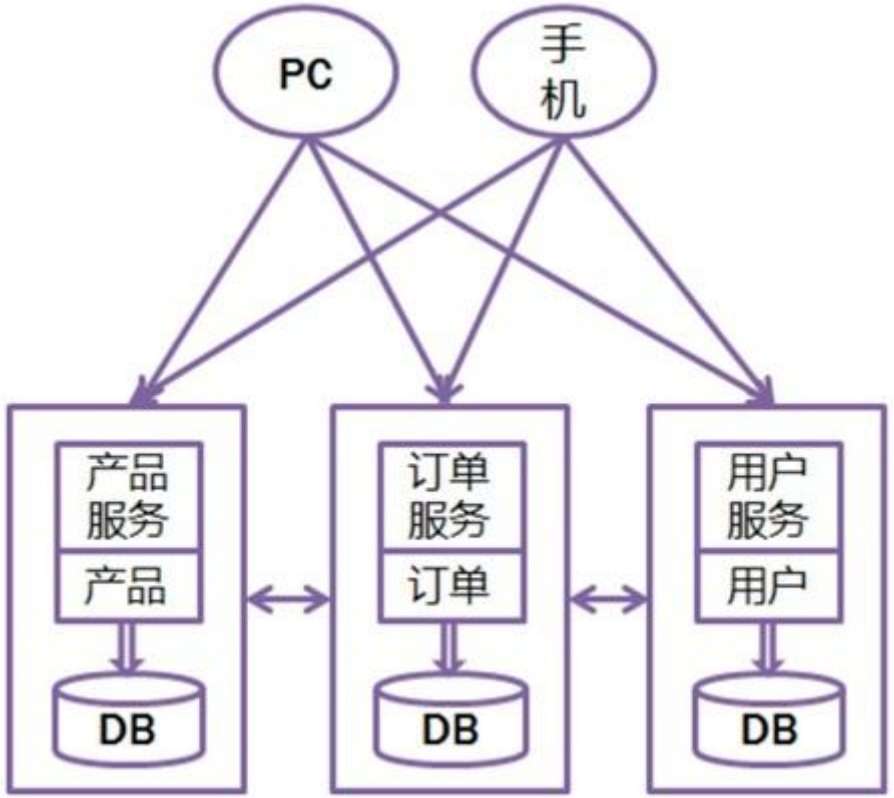


1 微服务设计概述

- 微服务架构的基本思想是：围绕业务领域组件来创建应用，让应用可以独立的开发、管理和加速。



Monolithic（单体式开发）



微服务架构

1 微服务设计概述

- 微服务具有四个方面的优点：
  - 每个微服务组件都是简单灵活的，能够独立部署。不再像以前一样，应用需要一个庞大的应用服务器来支撑。
  - 可以由一个小团队负责更专注专业，相应的也就更高效可靠。
  - 微服务之间是松耦合的，微服务内部是高内聚的，每个微服务很容易按需扩展。
  - 微服务架构与语言工具无关，自由选择合适的语言和工具，高效的完成业务目标即可。

1 微服务设计概述

- 同时，微服务带来的问题也不少：
  - 依赖服务变更很难跟踪
    - 其他团队的服务接口文档过期怎么办，依赖服务不稳定，基于其他依赖服务的新开发功能验证问题。
  - 部分模块重复构建
    - 跨团队、跨系统、跨语言会有很多的重复建设。
  - 微服务放大了分布式架构的系列问题
    - 如分布式事务的处理；
  - 运维复杂度陡增
    - 部署物数量多、监控进程多导致整体运维复杂度提升。

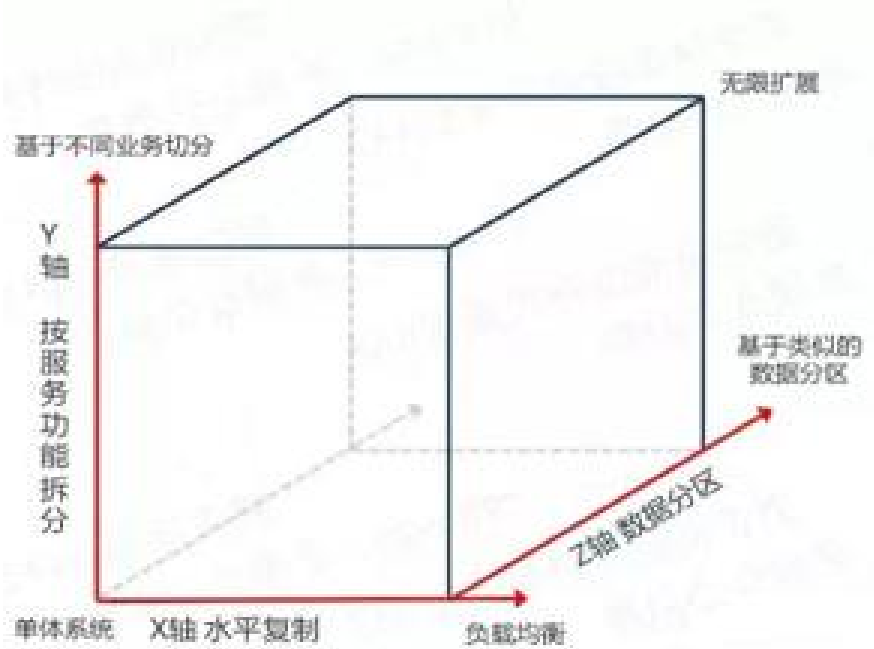
四个原则

微服务设计的四个原则：

- AKF拆分原则
- 无状态服务原则
- 无状态通信原则
- 技术分离原则

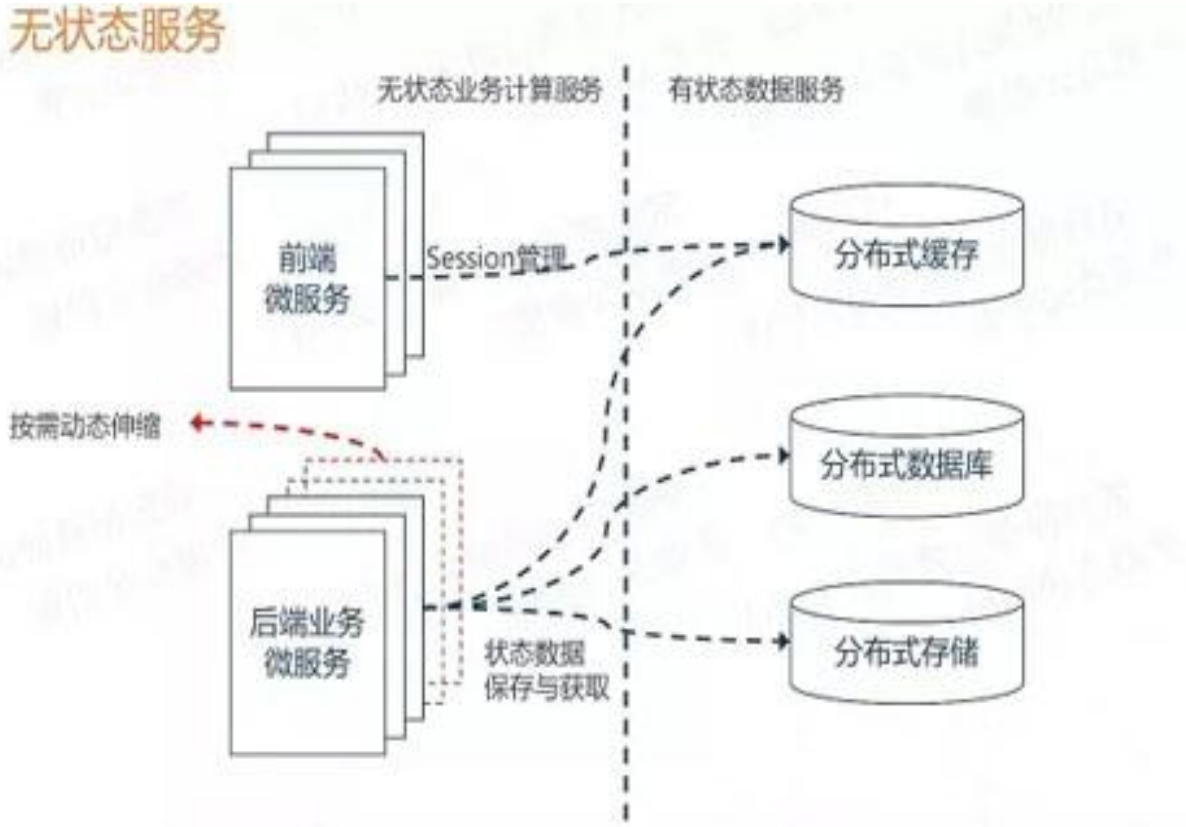
原则1：AKF拆分原则

- AKF扩展立方体(参考《The Art of Scalability》)，是一个叫AKF的公司的技术专家抽象总结的应用扩展的三个维度。
- 理论上按照这三个扩展模式，可以将一个良好结构的单体系统，扩展为微服务架构。
  - X 轴：指的是水平复制，单体系统多运行几个实例，做个集群加负载均衡的模式。
  - Z 轴：是基于类似的数据分区，比如一个互联网打车应用用户量激增，集群模式撑不住了，那就按照用户请求的地区进行数据分区，多建几个集群。
  - Y 轴：就是我们所说的微服务的拆分模式，就是基于不同的业务拆分。



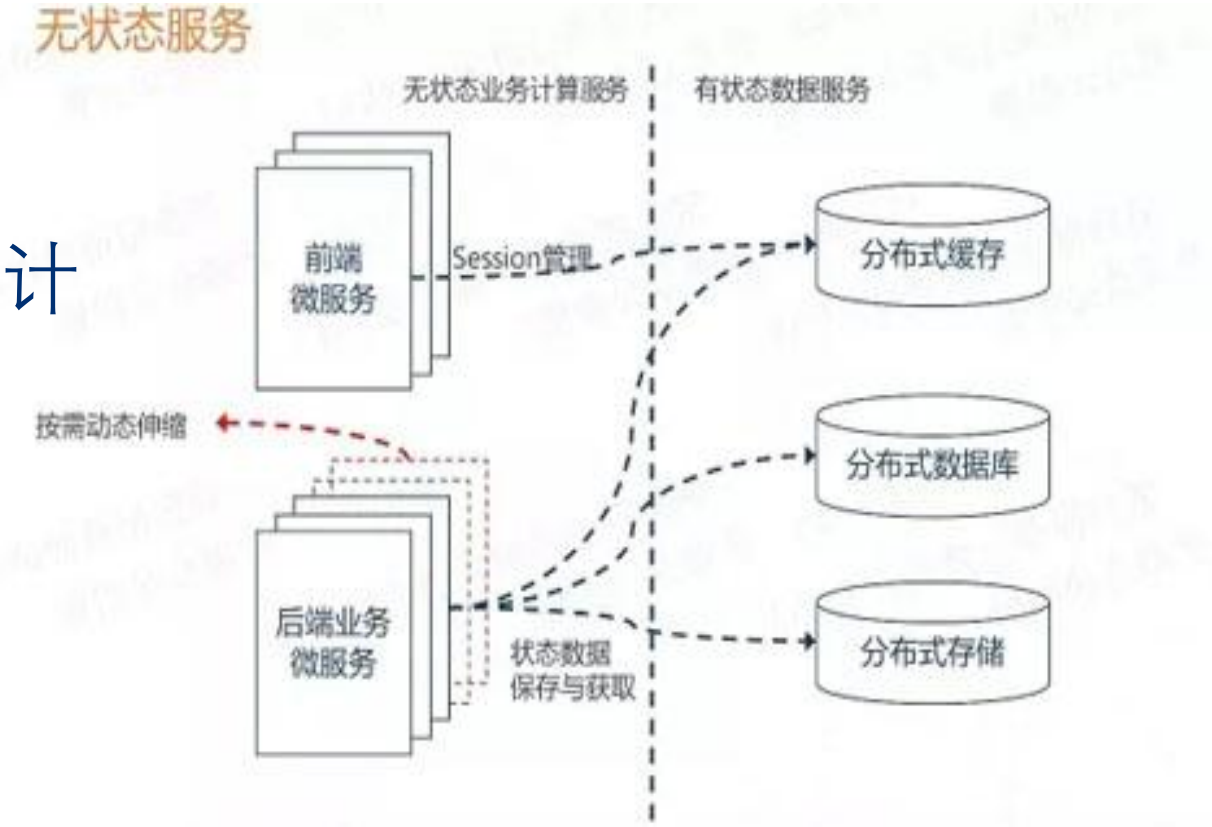
原则2：无状态服务原则

- 状态：如果一个数据需要被多个服务共享，才能完成一笔交易，那么这个数据被称为状态。进而依赖这个“状态”数据的服务被称为有状态服务，反之称为无状态服务。
- 微服务架构里不允许存在状态？



原则2：无状态服务原则

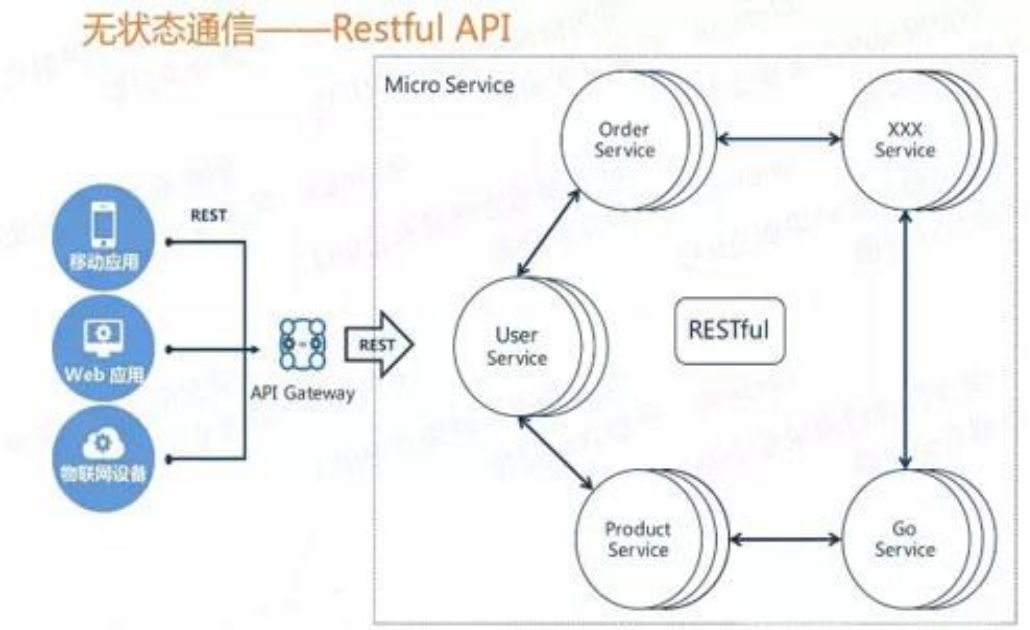
- 状态：如果一个数据需要被多个服务共享，才能完成一笔交易，那么这个数据被称为状态。进而依赖这个“状态”数据的服务被称为有状态服务，反之称为无状态服务。
- 微服务架构里不允许存在状态？✗
- 把有状态的业务服务改变为无状态的计算类服务和“有状态数据服务”。
- 例如缓存到分布式缓存的变化



原则3：无状态通信原则

无状态通信原则：

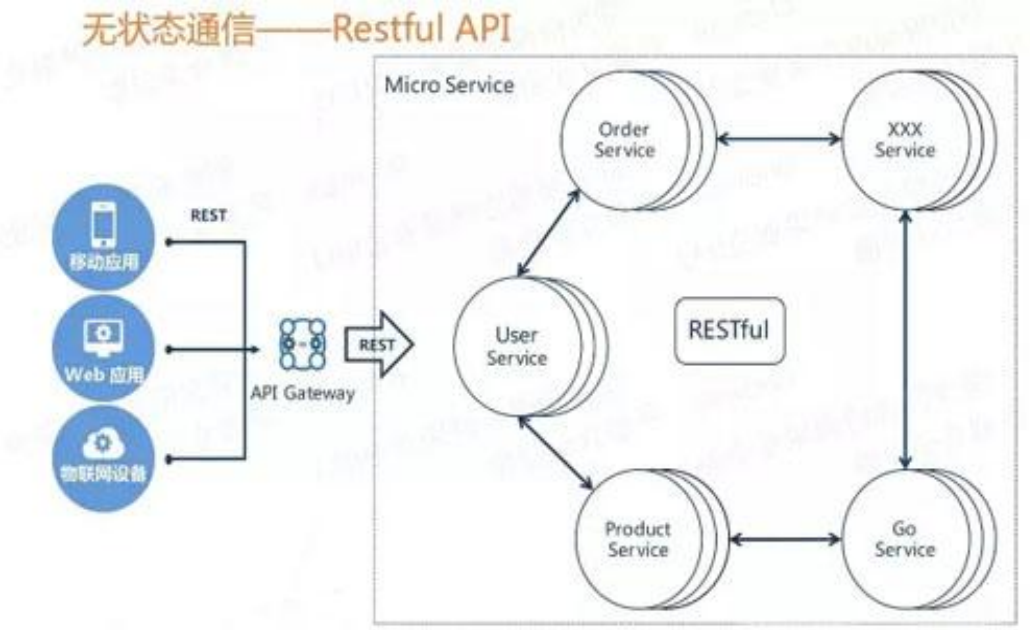
- 请求与请求之间不保存状态
- 对所有客户端的处理一视同仁，提高了服务器的并发能力
- 可以采用包括RESTful、gRPC、基于队列等实现，这些通信模式根据具体的业务场景选择。



原则3：无状态通信原则

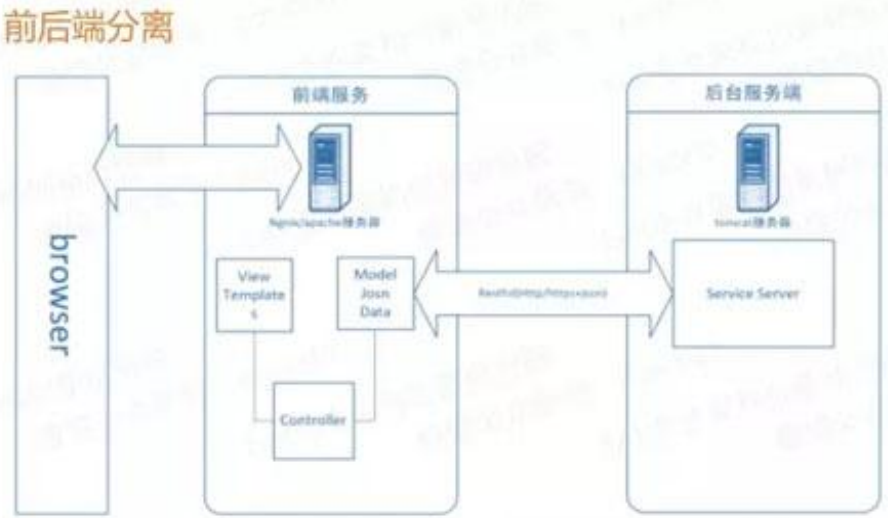
无状态通信原则：

- Restful 通信风格是一个使用较广的方案，其好处在于：
  - 无状态协议HTTP，具备先天优势，扩展能力很强。例如需要安全加密时，有现成的成熟方案HTTPS可用。
  - JSON 报文序列化，轻量简单，人与机器均可读，学习成本低，搜索引擎友好。
  - 语言无关，各大热门语言都提供成熟的Restful API框架，相对其他的一些RPC框架生态更完善。



原则4：分离原则

- 分离原则是微服务性能隔离的关键，软件的各层级可以从多个方面实现，例如前后端代码分离方法。
- 前后端分离，简单来讲就是前端和后端的代码分离也就是技术上做分离。
  - 直接采用物理分离的方式部署，进一步促使进行更彻底的分离。不要继续以前的服务端模板技术，比如JSP ，把Java JS HTML CSS 都堆到一个页面里，稍复杂的页面就无法维护。
  - 前后端技术分离，可以由各自的专家来对各自的领域进行优化，这样前端的用户体验优化效果会更好。
- 分离模式下，前后端交互更加清晰，就剩下接口和模型，后端的接口简洁明了，更容易维护。
- 前端多渠道集成场景更容易实现，后端服务无需变更，采用统一的数据和模型，可以支撑前端的web UI\ 移动App等访问。



目录

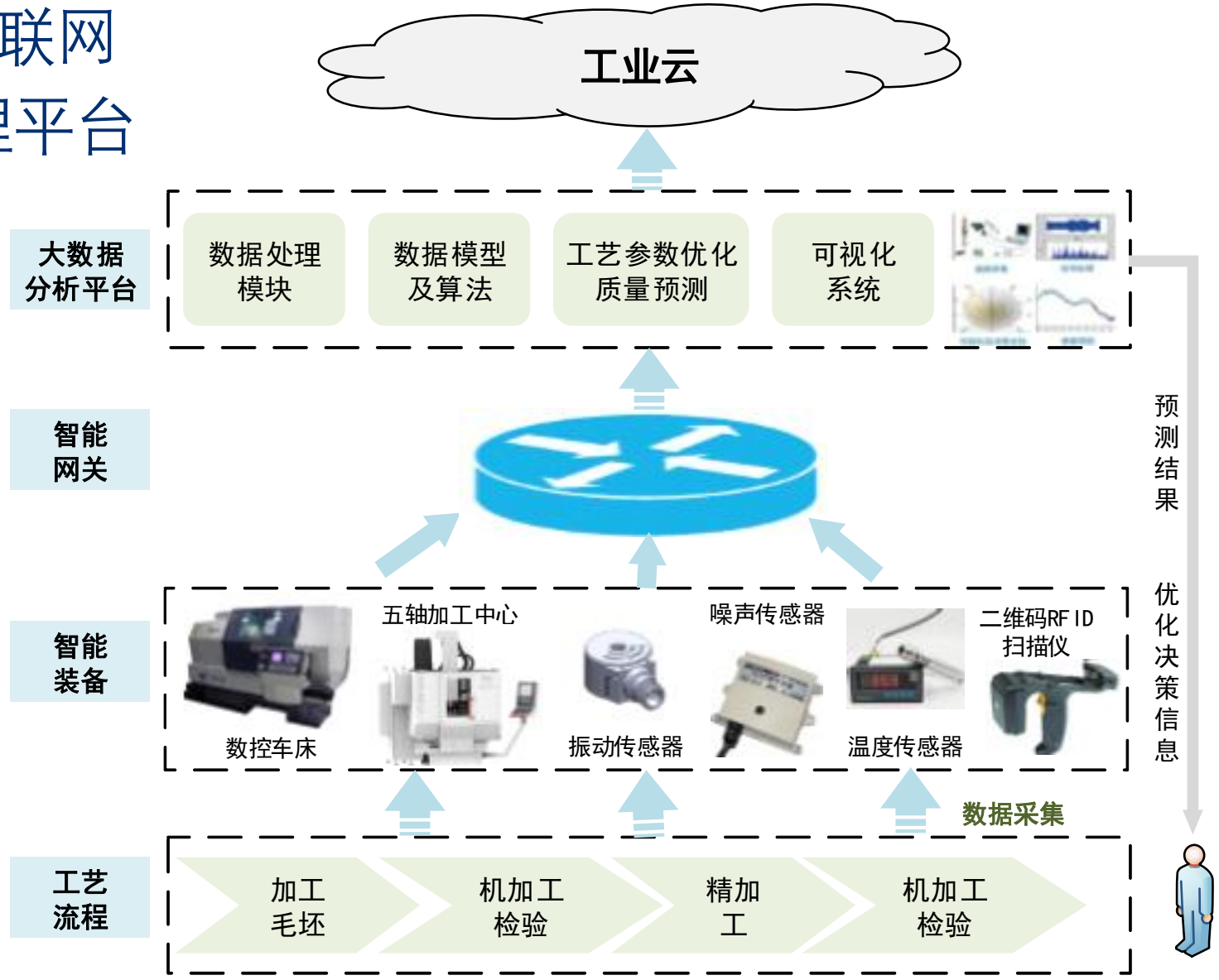
- 1 微服务设计概述
- 2 微服务架构设计
  - 定义系统操作 – 定义业务服务 – 定义服务API
- 3 微服务架构下的进程间通信
  - 服务间通信 – 分布式事务 – 容错设计及流量控制
- 4 小结

2 微服务架构设计

- 设计微服务架构是一项艺术而非技术
- 微服务架构的基本思想是围绕**业务领域组件**来创建应用，设计微服务架构往往采用三步式流程：
  - 定义系统操作：将应用程序的需求提炼为各种关键请求并采用系统操作的概念进行描述；
  - 定义业务服务：围绕业务概念而非技术概念对于服务进行分解和设计；
  - 定义服务接口（API）：将第一步中标识的系统操作分配给服务。

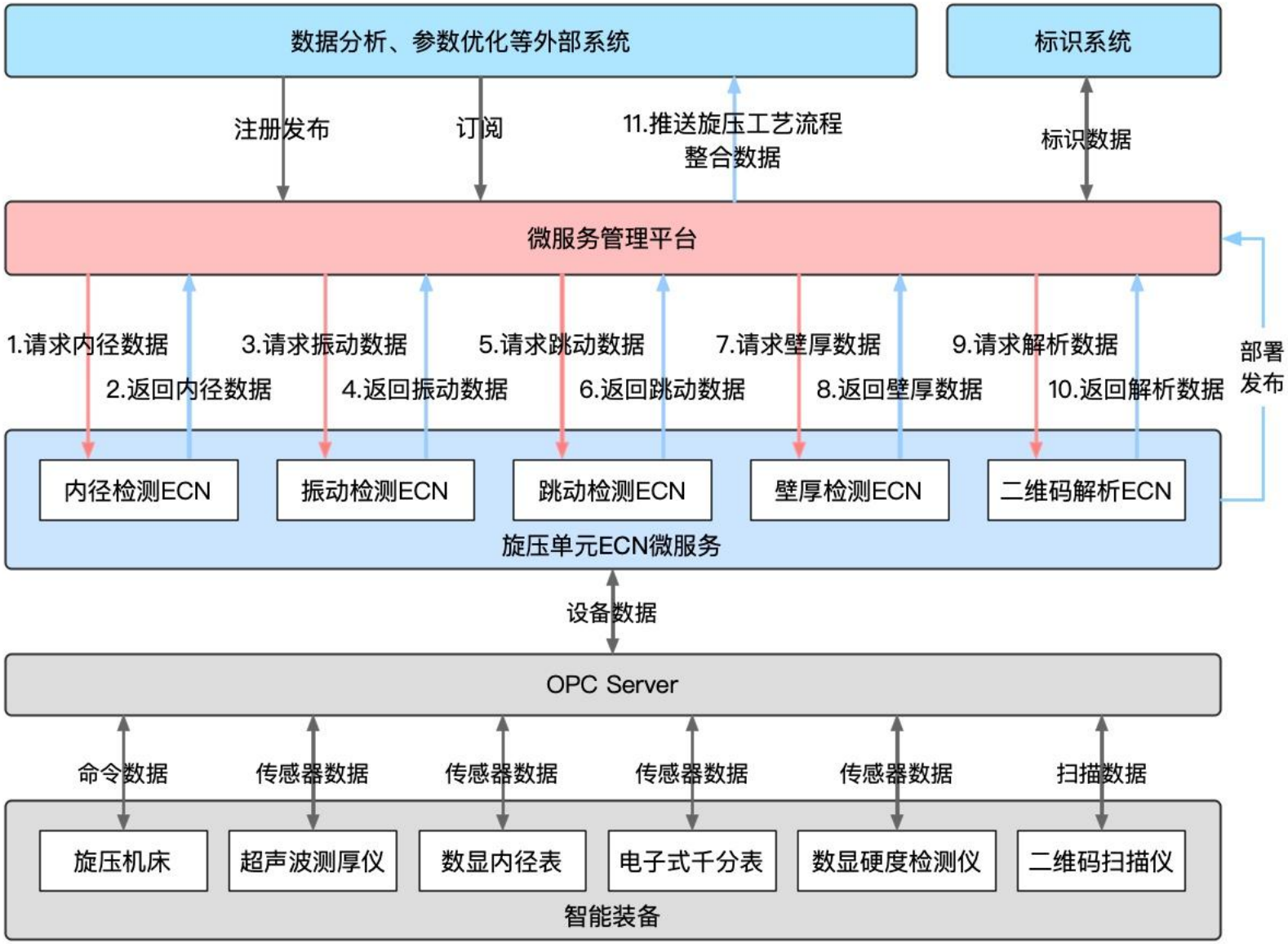
2 微服务架构设计

- 以IST工业互联网边缘计算管理平台为例



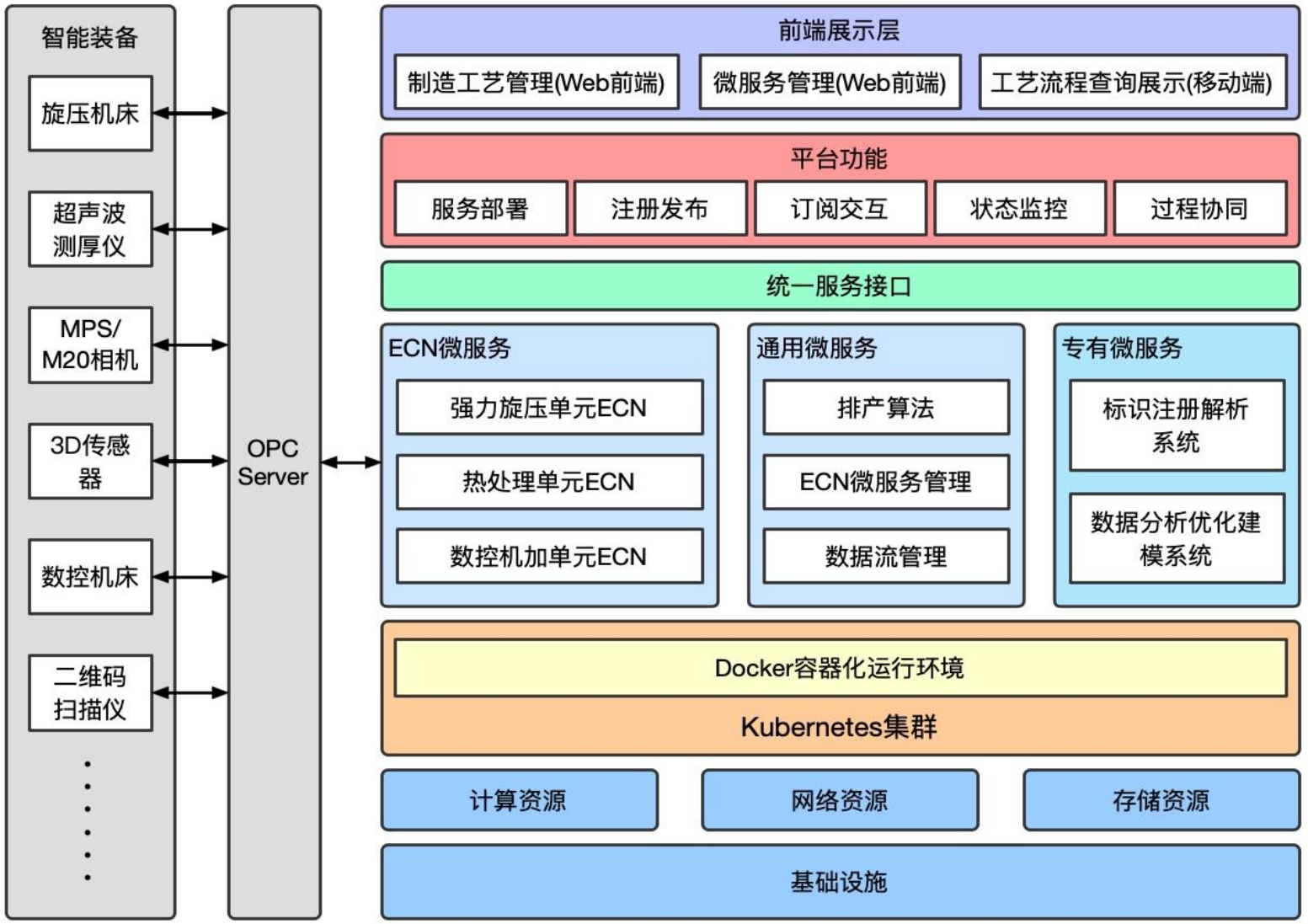
2 微服务架构设计

- 以IST工业互联网边缘计算管理平台为例



2 微服务架构设计

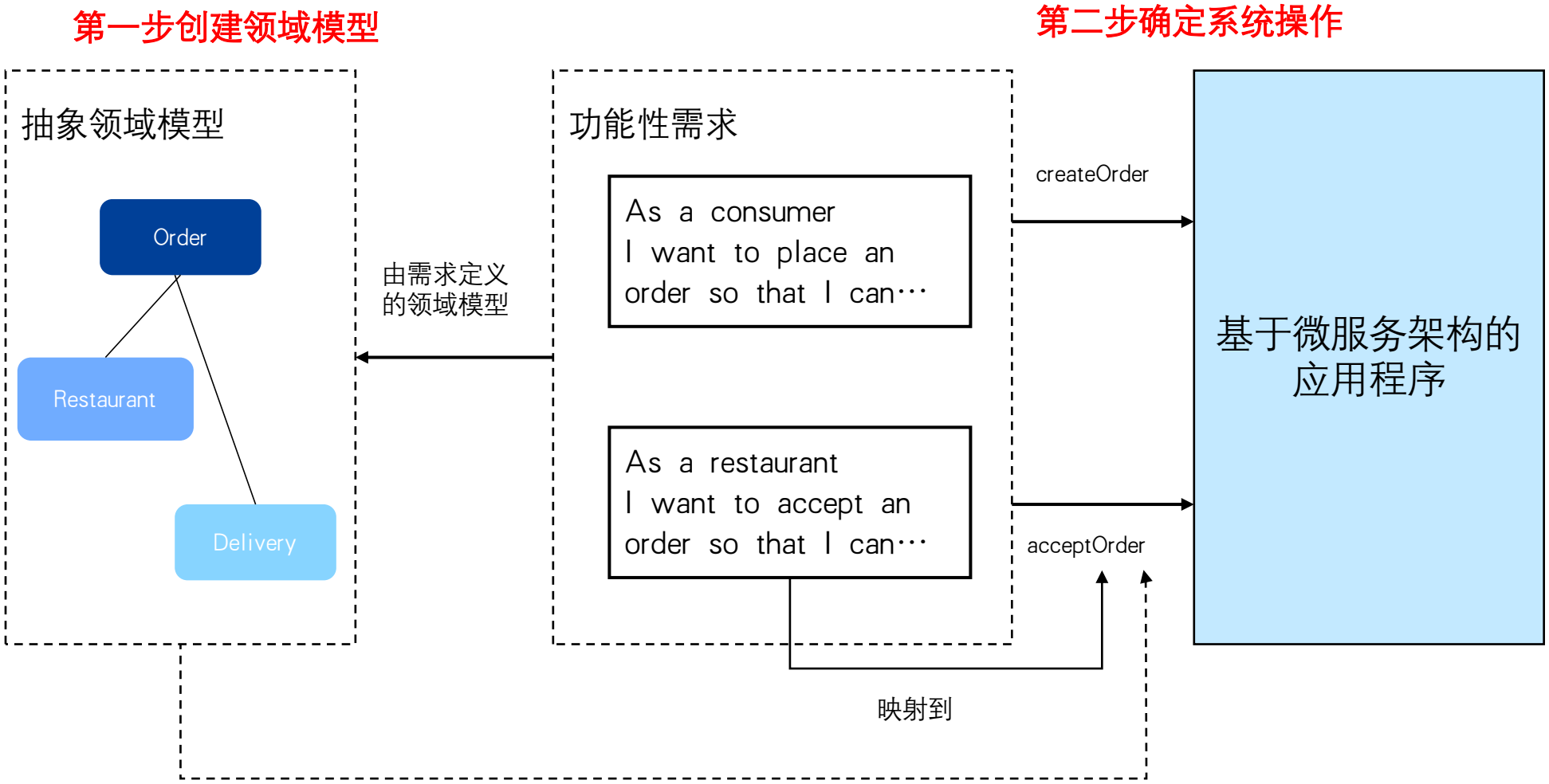
- 以IST工业互联网边缘计算管理平台为例



2.1 定义系统操作

- 设计微服务架构的第一步是定义系统操作，该起点是应用程序的需求，包括用户确定及其相关的用户场景
- 定义系统操作往往采用两个步骤：
  - 第一步创建由关键类组成的抽象领域模型，这些关键类提供用于描述系统操作的词汇表
  - 第二步确定系统操作，并根据领域模型描述每个系统操作的行为
- 抽象的领域模型和系统操作能够回答应用“做什么”这个问题，这有助于推动应用程序的微服务架构设计

2.1 定义系统操作


第一步创建领域模型
第二步确定系统操作
抽象领域模型
Order
Restaurant
Delivery
由需求定义
的领域模型
功能性需求
As a consumer
I want to place an
order so that I can…
As a restaurant
I want to accept an
order so that I can…
createOrder
acceptOrder
基于微服务架构的
应用程序
映射到
系统操作通过领域模型定义

（1）创建抽象领域模型

- 定义系统操作的第一步是为应用程序描绘一个抽象的领域模型，该模型会比最终要实现的简单很多
- 尽管领域模型非常简单，但是抽象的领域模型通过定义了描述系统对象的一些词汇有助于在开始阶段提供帮助
- 创建领域模型会采用一些标准的技术，例如通过与领域专家沟通后，分析用户故事和场景中频繁出现的名词从而提炼为领域模型

创建抽象领域模型

- 例如Place Order用户故事，可以分解为用户场景如下：

Given a consumer

And a restaurant

And a delivery address/time that can be served by that restaurant

When the consumer places an order for the restaurant

Then consumer's credit card is authorized

And an order is created in the PENDING_ACCEPTANCE state

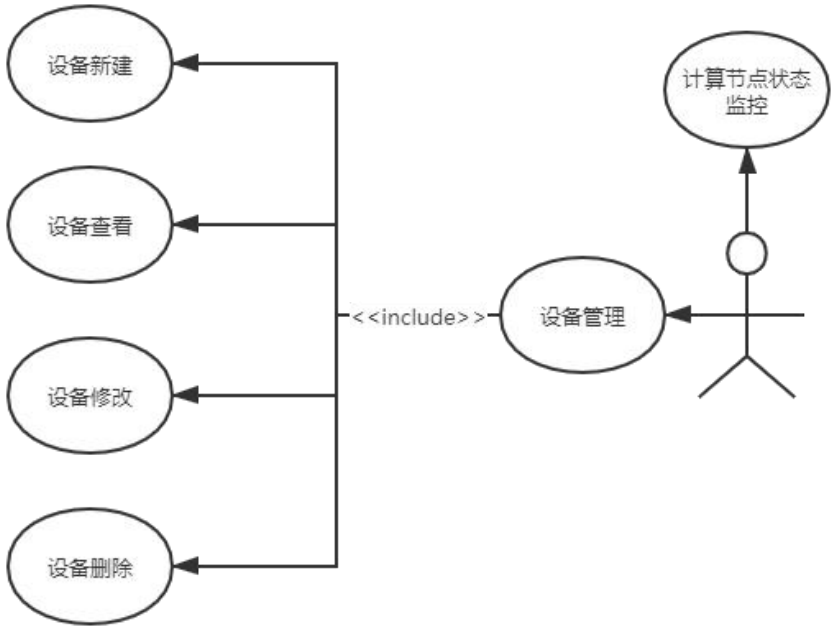
And the order is associated with the consumer

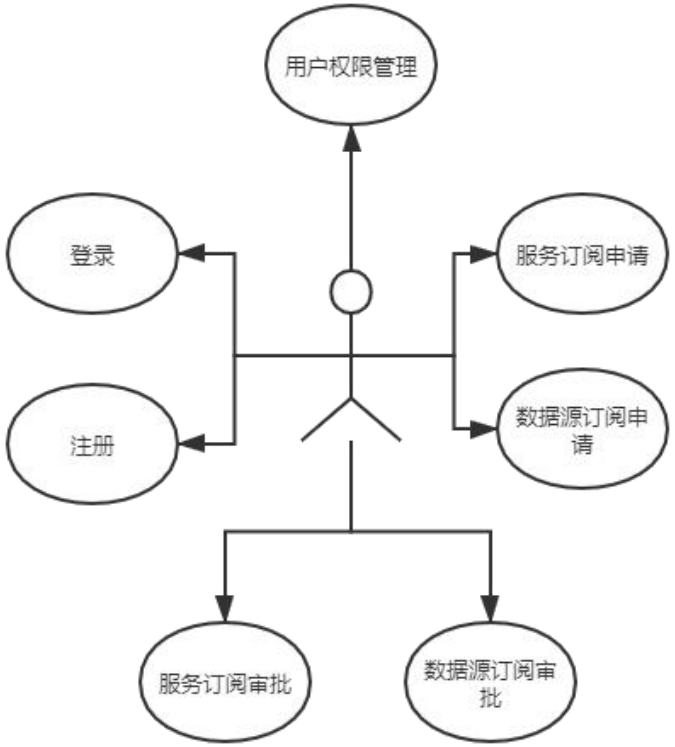
And the order is associated with the restaurant

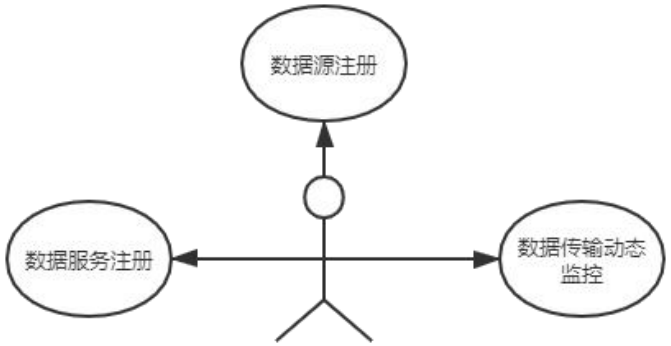
- 在该用户场景中可以抽象出领域模型如：Consumer、Order、Restaurant和CreditCard

创建抽象领域模型

- 以IST工业互联网边缘计算管理平台为例
- 业务微服务：内径、震动、跳动、壁厚、旋压、二维码
- 平台微服务：设备、数据、服务、流程







（2）确定系统操作

- 当定义了抽象的领域模型之后，接下来需要确定系统操作
- 主要有以下两种类型系统操作：
  - 命令型：创建、更新或删除数据的系统操作
  - 查询型：查询和读取数据的系统操作
- 识别系统操作的切入点在于分析用户故事和场景中的动词
- 在完成了系统操作识别后，需要将系统操作整理为系统操作规范：
  - 系统操作的输入参数
  - 系统操作的返回值
  - 系统操作对应的领域模型类
  - 系统操作对应的前置条件
  - 系统操作对应的后置条件

确定系统操作

➢ 以IST工业互联网边缘计算管理平台为例

用例编号	UC2-2	用例名称	数据服务注册
描述	业务部门人员可以根据多个数据源按照一定规则进行组合注册对应的数据服务		
执行者	业务部门人员		
前置条件	业务部门人员已登录，进入数据服务注册页面		
后置条件	系统根据注册信息创建对应的数据服务		
基本流	1. 用户填写数据服务基本信息，包括数据源名称、数据服务对应的队列名称、描述等 2. 用户选择单个或多个用于进行组合的输入数据源 3. 用户设置输入数据源的数据融合规则 4. 用户设置输入数据源的 5. 用户点击提交		
备选流	5a 用户设置的数据融合 5b 数据服务对应队列名		

用例编号	UC2-3	用例名称	数据传输动态监控
描述	业务部门人员可以查看每个数据源或数据服务的数据传输日志记录		
执行者	业务部门人员		
前置条件	业务部门人员已登录，进入数据服务列表页面		
后置条件	无		
基本流	1. 用户从数据服务列表中选取单个数据服务 2. 用户点击数据服务对应的监控按钮 3. 系统展示数据服务最新的数据传输日志记录，包含记录的时间戳以及传输内容		
备选流	2a 用户尚未订阅对应数据服务，系统提示权限不足		

2.2 定义业务服务

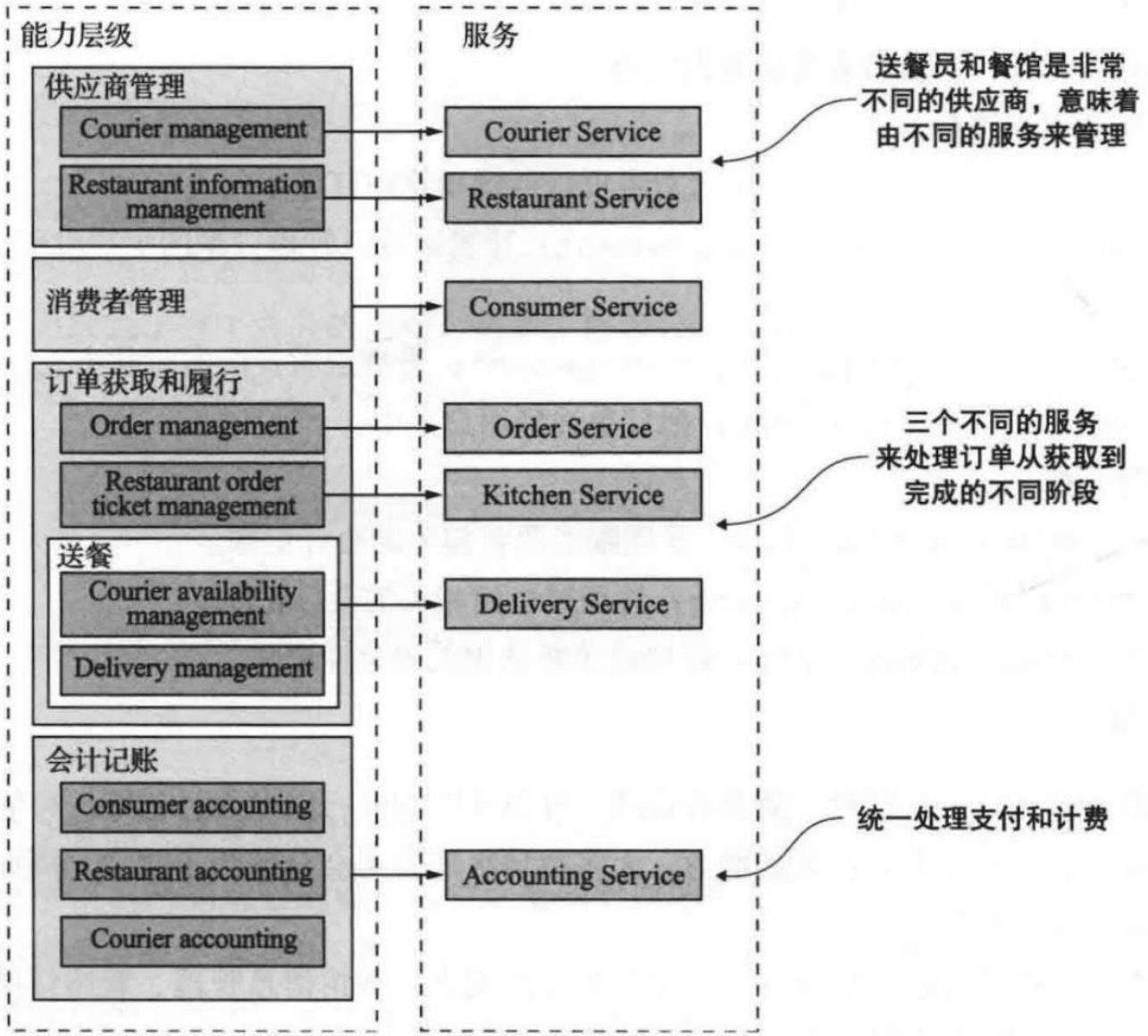
- 在系统操作被定义后，下一步就是完成应用业务服务的识别。
- 拆分应用服务有多种拆分策略可供选择：
  - 根据业务能力进行服务拆分
  - 根据子域进行服务拆分
- 每一种拆分策略都是从不同的切入点来解决问题，但是每一种拆分策略都有着统一的拆分的指导原则。
- 每一种拆分策略的结果都是一个包含若干个服务的以业务而非技术概念为核心的架构。

（1）根据业务能力进行服务拆分

- 业务能力是一个来自于业务架构建模的术语。
- 业务能力是指一些能够为公司产生价值的商业活动。特定业务的业务能力取决于这个业务的类型。例如，保险公司业务能力通常包括承担保险、理赔管理、账务等。
- 一个公司有哪些业务能力是通过对公司的目标、结构和商业流程的分析得来的
- 业务能力通常集中在特定的业务对象。例如，理赔业务对象是理赔管理功能的重点。
- 业务能力通常可以分解为子能力，从而形成能力层次结构。例如，理赔管理能力具有多个子能力，包括：理赔信息管理、理赔审核和理赔付款管理。

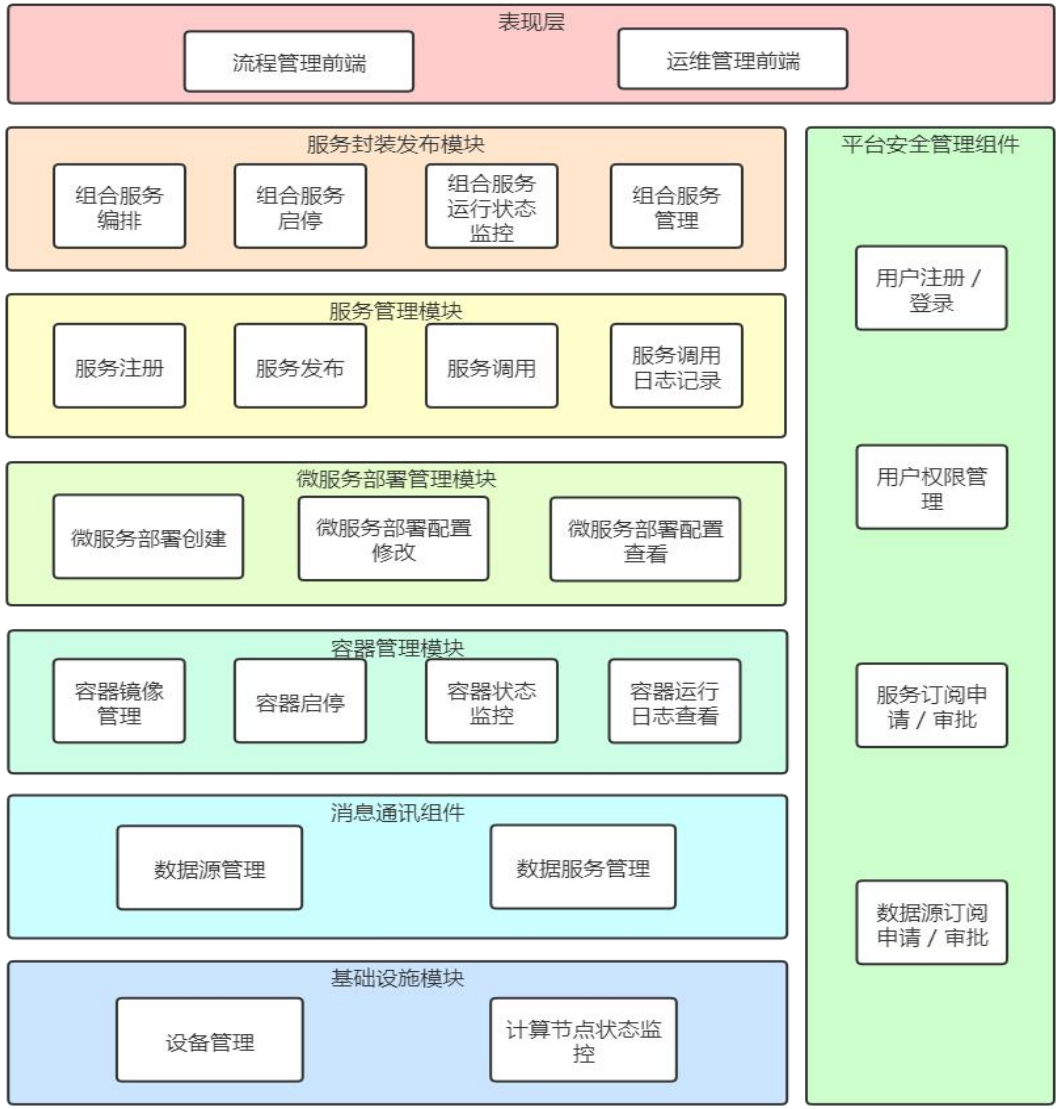
根据业务能力进行服务拆分

- 一旦确定了业务能力，就可以为每个能力或相关能力组定义为服务，决定将哪个级别的业务能力层次结构映射到服务是一个非常主观的判断；
- 围绕业务能力进行服务拆分的一个关键好处是，因为它们是稳定的，所以最终的架构也将相对稳定。
- 架构的各个组件可能会随着业务的具体实现方式的变化而发展，但是架构仍保持不变。



根据业务能力进行服务拆分

- 以IST工业互联网边缘计算管理平台为例

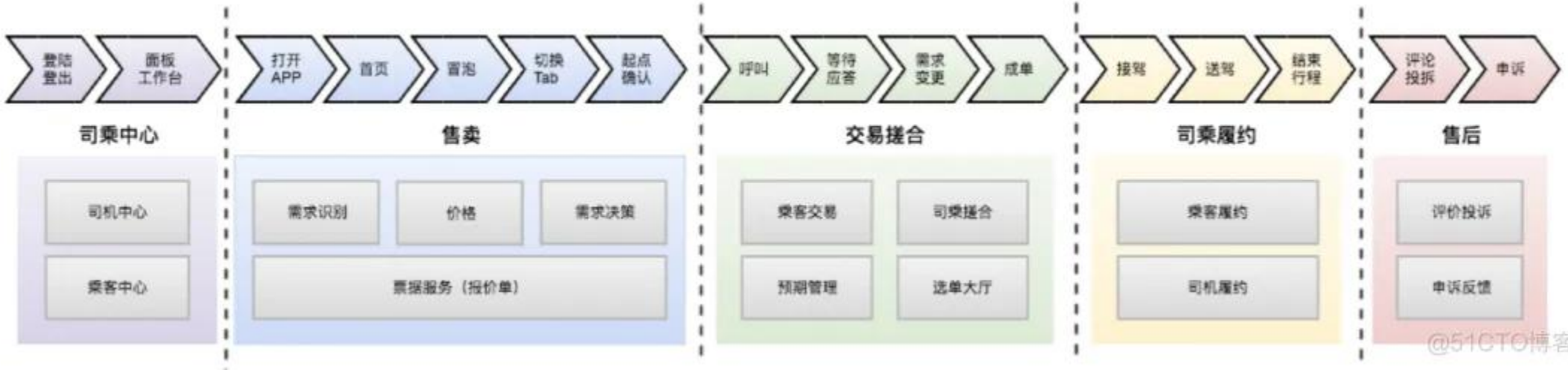


（2）根据子域进行服务拆分

- 领域驱动设计（Domain-Driven Design，DDD）的概念产生于Eric Evans写的《Domain Driven Design》一书，DDD是对面向对象设计的改进，是开发复杂业务逻辑的一种方法，其主要由两大模式组成：战略性模式和战术性模式。
- 战略性模式：DDD使用子域划分和限界上下文作为战略性模式，战略性模式有助于指导如何把应用程序分解为不同微服务。
- 战术性模式：DDD使用实体、值对象、工厂、聚合、领域事件等基本元素作为战术性模式，战术性模式有助于指导单个微服务如何设计。

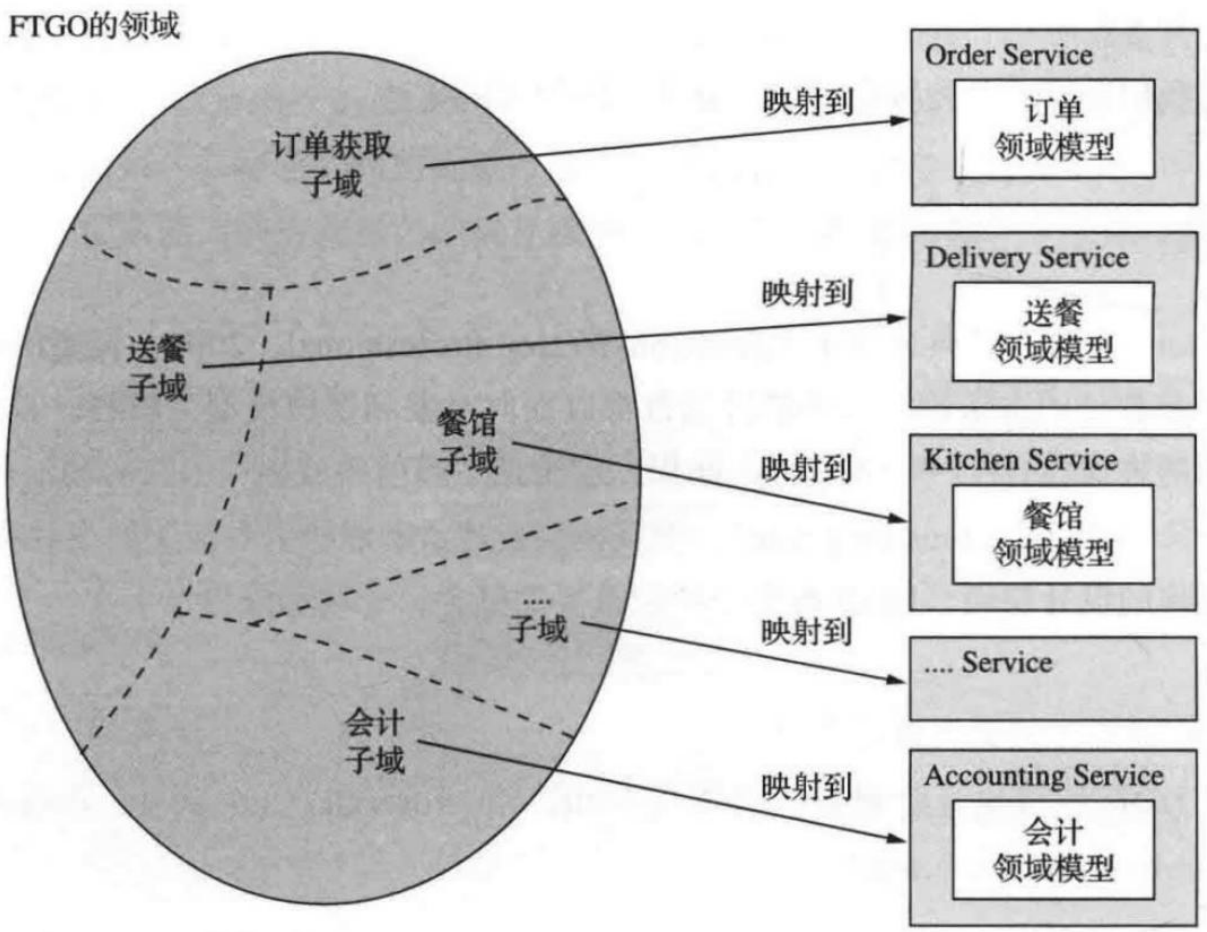
根据子域进行服务拆分

- Why：传统的企业架构建模方式往往会为整个企业建立一个单独的模型，这类传统建模的挑战在于，让组织内的所有团队都对全局单一的建模和术语定义达成一致非常困难
- How：领域驱动设计通过定义多个领域模型来避免了上述的问题，每个领域模型都有其明确的范围



根据子域进行服务拆分

- 领域驱动设计为每一个子域定义单独的领域模型。子域是领域的一部分，领域是DDD中用来描述应用程序问题域的一个术语。
- 识别子域的方式跟识别业务能力基本一致：分析业务并识别业务的不同专业领域，分析产出的子域定义结果也会跟业务能力基本一致

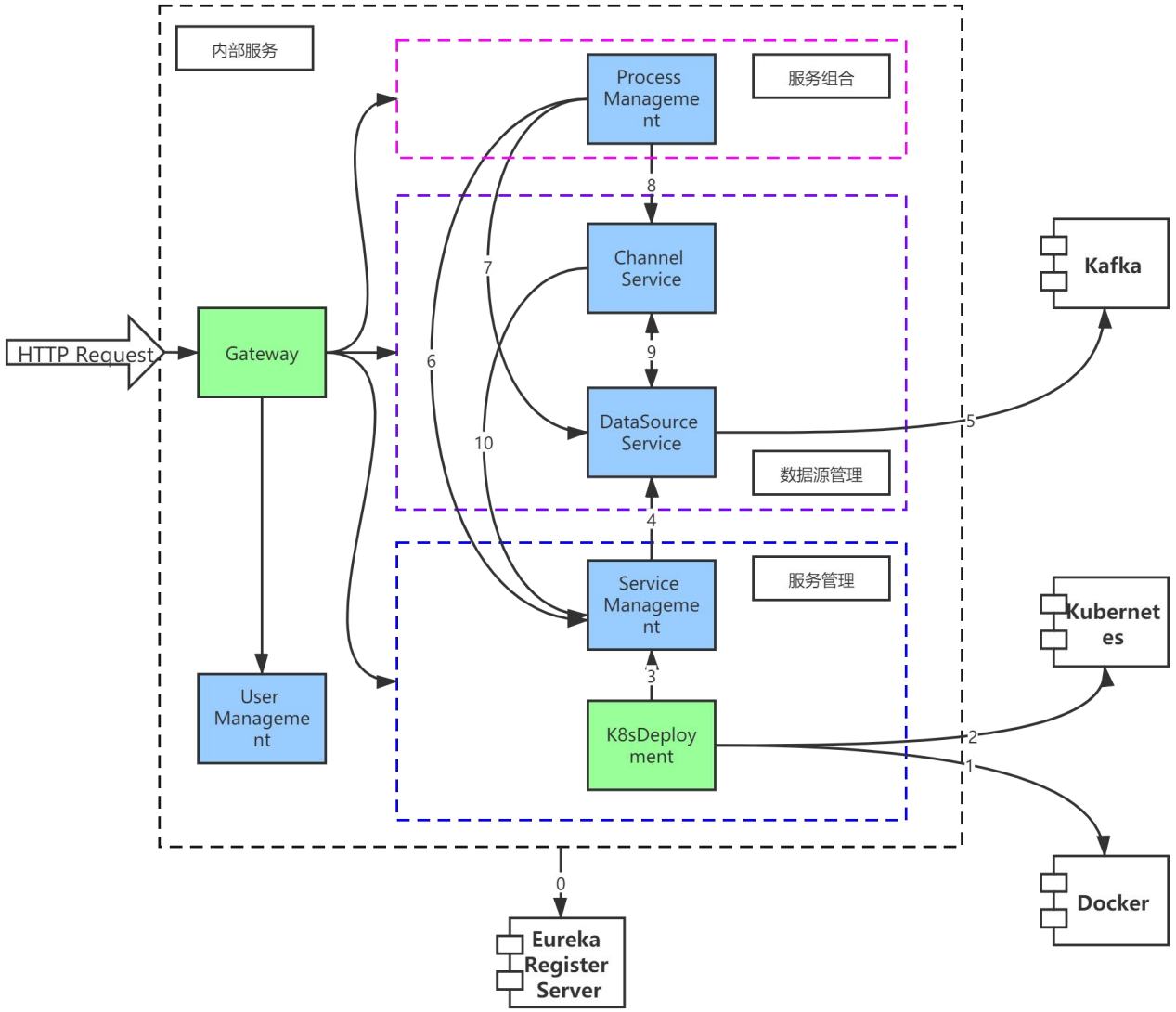


根据子域进行服务拆分

- 领域驱动设计把领域模型的边界称为限界上下文，限界上下文包括实现这个模型的代码集合
- 在根据子域进行服务拆分时，往往通过DDD的方式定义子域，然后把子域映射为限界上下文，并把每一个限界上下文对应一个服务
- 领域驱动设计可以非常好地指导微服务架构设计。一方面DDD的子域和限界上下文的概念可以很好地与微服务架构中的服务进行匹配，另一方面微服务架构中的自治化团队负责开发的概念，也跟DDD中每个领域模型都由一个独立团队负责开发的概念吻合

根据子域进行服务拆分

- 以IST工业互联网边缘计算管理平台为例



（3）指导原则：单一职责原则

- 单一职责原则（SRP）定义如下：
  - 改变一个类应该只有一个理由
    - Robert C. Martin
- 类所承担的每一个职责都是对它进行修改的潜在原因，如果一个类承载了多个职责，并且互相之间的修改是独立的，那么这个类就会非常不稳定
- 在设计微服务架构时应该遵循SRP原则，设计小的、内聚的、仅仅含有单一职责的服务，这会缩小服务的大小并提升其稳定性

（4）指导原则：闭包原则

- 闭包原则（CCP）定义如下：
- 在包中包含的所有类应该是对同类的变化的一个集合，如果对包做出修改，需要调整的类应该都在这个包中

  - Robert C. Martin

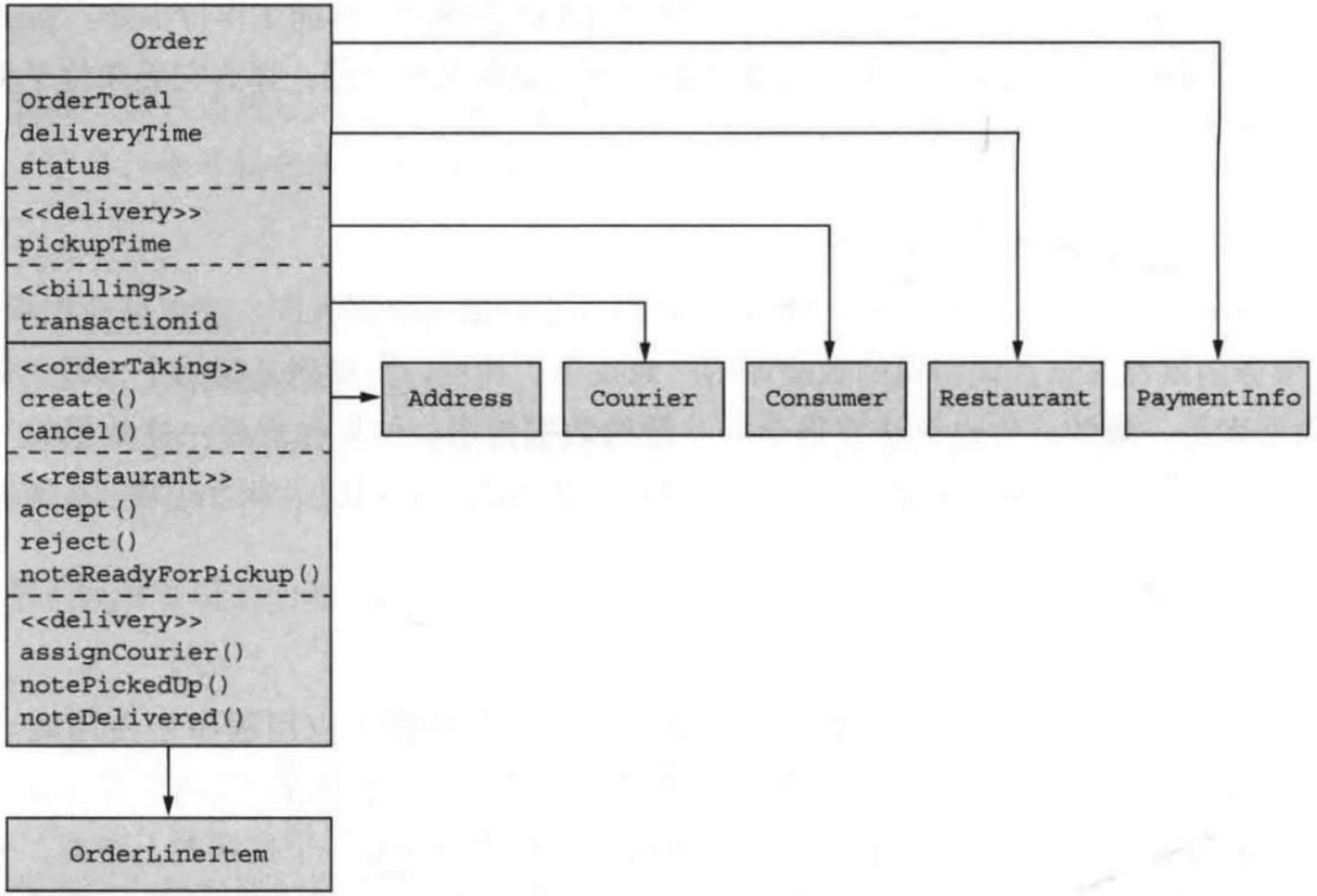
- 如果两个类的修改必须耦合先后发生，那就应该把它们放在同一个包中，从而当业务规则发生变化时，开发者只需要对一个交付包做出修改，而不是大规模地修改整个应用。采用闭包原则可以改善应用程序的可维护性
- 在微服务架构下采用CCP原则，就能把根据同样原因进行变化的服务放在一个组件，从而可以控制服务的数量，并且当需求发生变化时，使得变更和部署都更加容易

拆分的难点

- 网络延迟
  - 问题：网络延迟是分布式系统中一直存在的问题，对服务的特定分解会导致两个服务之间的大量往返调用
  - 解决方案：将多个相关的多个服务组合成一个服务
- 同步进程间通信导致可用性降低
  - 问题：使用REST同步调用其他服务会降低服务本身的可用性，如果任何一个被调用的其他服务处在不可用状态，都会导致调用服务不可用
  - 解决方案：使用基于消息的异步通信机制取代同步进程间通信
- 在服务之间维持数据一致性
  - 问题：在系统操作需要更新多个服务中的数据时，服务之间的数据一致性会遭到破坏
  - 解决方案：采用分布式事务管理机制

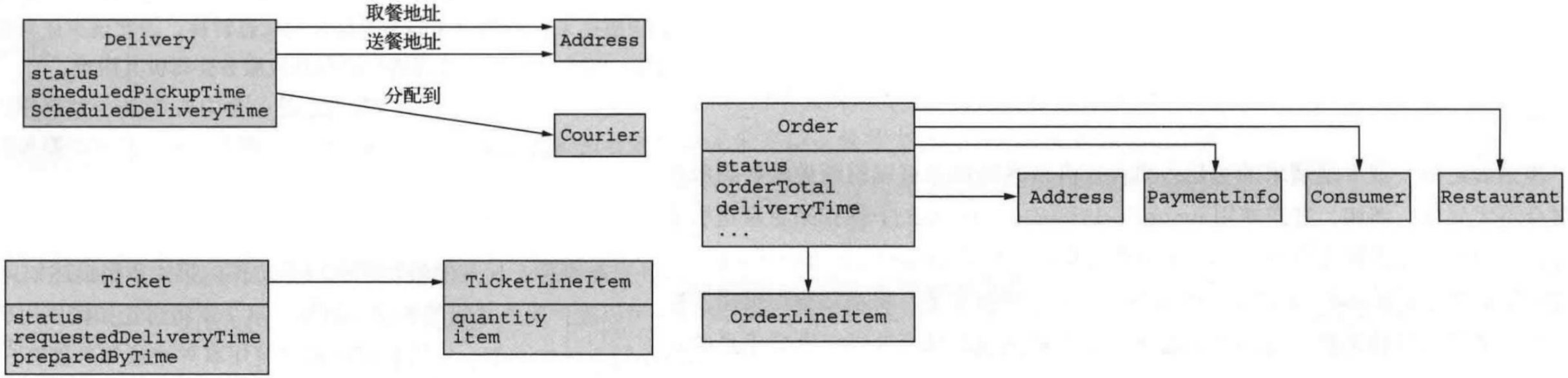
拆分的难点

- 全局类阻碍了拆分
  - 问题：全局类是在整个应用程序中多个服务都会涉及到的全局类，全局类通常为应用程序的许多不同方面实现业务逻辑。例如，在电商场景下的Order类，系统大多数部分都会涉及订单



拆分的难点

- 全局类阻碍了拆分
  - 解决方案：应用DDD并将每个服务视为具有自己领域模型的单独子域，从而每个与订单有关的服务都有自己的领域模型及其对应的Order类的版本，并且仅仅关注订单在当前限界上下文中有意义的属性。



2.3 定义服务API

- 服务API有两大类：
  - 由外部客户端调用的对应于系统操作的服务API
  - 由其他服务调用的用于支持服务间协作的服务API
- 把系统操作分配给服务：需要确定哪个服务是请求的初始入口点，

服　务	操　作
Consumer Service	createConsumer()
Order Service	createOrder()
Restaurant Service	findAvailableRestaurants()
Kitchen Service	▪ acceptOrder() ▪ noteOrderReadyForPickup()
Delivery Service	▪ noteUpdatedLocation() ▪ noteDeliveryPickedUp() ▪ noteDeliveryDelivered()

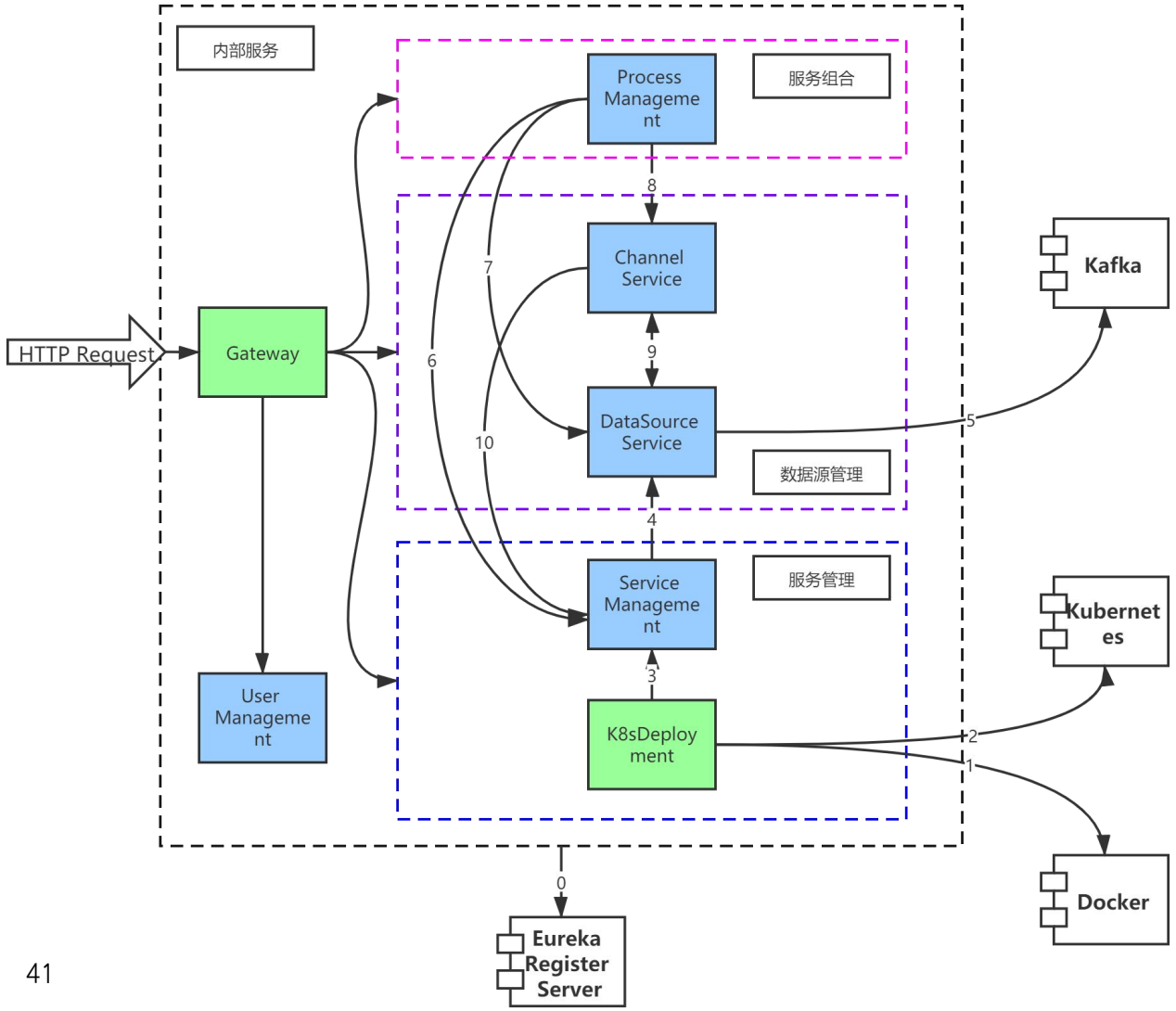
2.3 定义服务API

- 另一方面需要考虑哪些系统操作需要跨越那些服务，从而划分为支持服务间协作的服务API。

服　务	操　作	协 作 者
Consumer Service	verifyConsumerDetails()	—
Order Service	createOrder()	▪Consumer Service verifyConsumerDetails() ▪Restaurant Service verifyOrderDetails() ▪Kitchen Service createTicket() ▪Accounting Service authorizeCard()
Restaurant Service	▪findAvailableRestaurants() ▪verifyOrderDetails()	—
Kitchen Service	▪createTicket() ▪acceptOrder() ▪noteOrderReadyForPickup()	▪Delivery Service scheduleDelivery()
Delivery Service	▪scheduleDelivery() ▪noteUpdatedLocation() ▪noteDeliveryPickedUp() ▪noteDeliveryDelivered()	—
Accounting Service	▪authorizeCard()	—

定义服务API

以IST工业互联网边缘计算管理平台为例



后端服务的访问

所有请求的访问通过 Gateway 来路由到下面的各个微服务。目前的 Gateway URL 如下：

```
http://202.120.40.87:14642/
```

各微服务访问入口

各个微服务访问入口为 Gateway URL + 微服务访问入口（K8sDeployment 除外）

微服务	访问入口
通道服务	/channel-service
数据源服务	/datasource-service
容器管理	/k8sdeployment
流程管理	/process-management
服务管理	/service-management
用户管理	/user-management
日志服务	/log-service

接口文档访问入口

各个微服务对应的接口文档为 Gateway URL + 微服务访问入口 + /swagger-ui.html (对于 k8sdeployment, 为了避免 gateway 强

目录

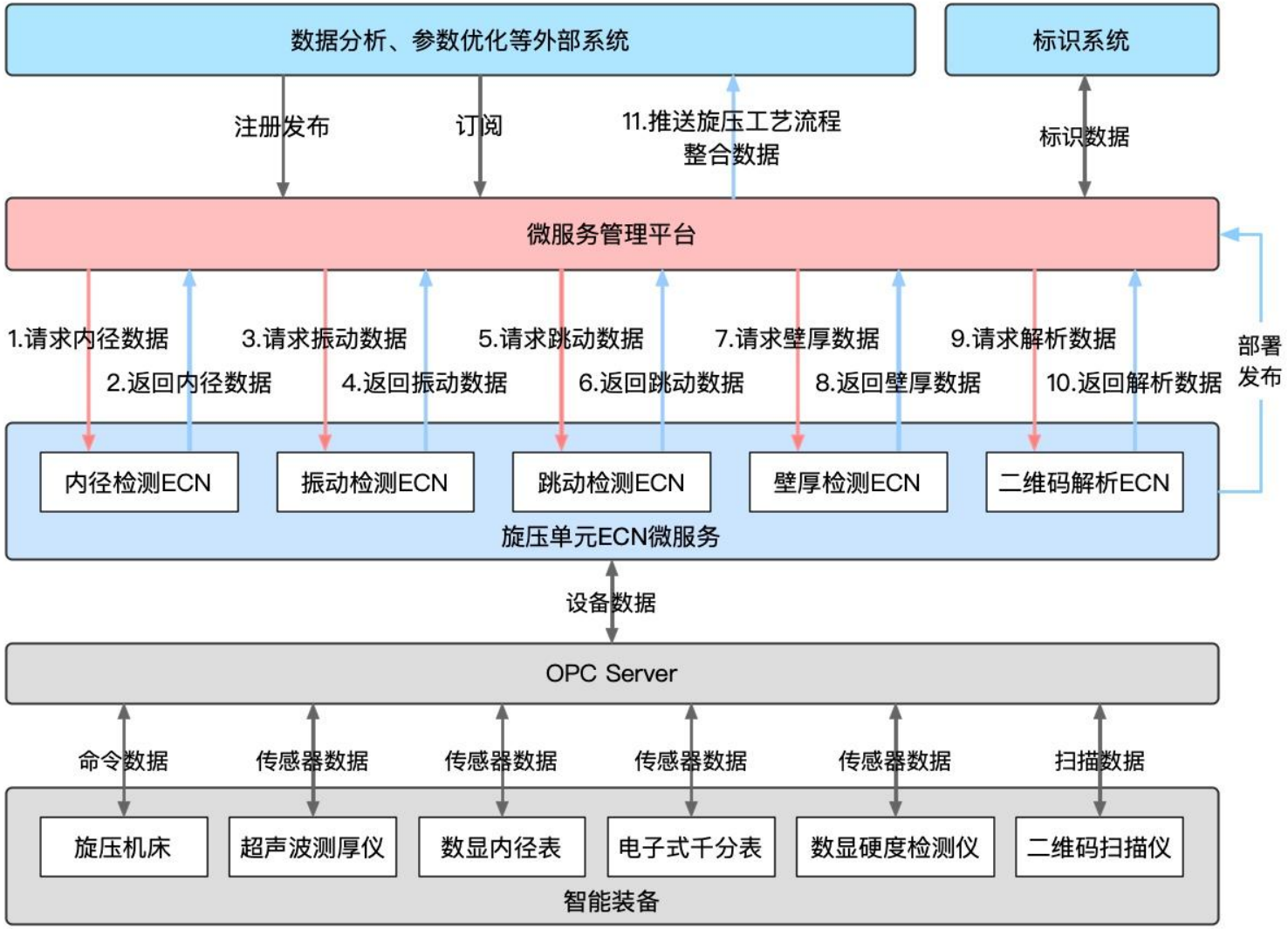
- 1 微服务设计概述
- 2 微服务架构设计
  - 定义系统操作 – 定义业务服务 – 定义服务API
- 3 微服务架构下的进程间通信
  - 服务间通信 – 分布式事务 – 容错设计及流量控制
- 4 小结

3 微服务架构下的进程间通信

- 使用微服务架构构建的应用程序是分布式系统，因此进程间通信（IPC）是微服务架构的重要组成部分，其主要分为了以下三块内容：
  - 微服务间通信：使用哪一类微服务间通信机制？
  - 分布式事务：对于跨多个微服务节点的事务该如何处理？
  - 流量治理：在服务不可用或者服务流量压力过大时，如何保证微服务应用的可靠性？

微服务架构下的进程间通信

- 以IST工业互联网边缘计算管理平台为例



3.1 服务间通信模式

- 微服务间通信模式有许多划分方式：
  - 服务传递的信息也既可以使用具备可读性的文本格式（JSON，XML等），又可以使用更高效的、基于二进制的格式（Avro，Protocol Buffers等）；
  - 既可以使用基于同步请求/响应的通信机制，也可以使用基于消息的异步通信机制。
- 从更高层的微服务间交互方式的角度来看，交互方式分为两个维度表示：一对一和一对多，同步和异步。

	一对一	一对多
同步模式	请求/响应	无
异步模式	异步请求/响应 单向通知	发布/订阅

（1）消息格式

- 微服务间通信的本质是交换消息。消息格式的选择会对进程间通信的效率、API的可用性以及可演化性产生影响。
- 消息格式需要选择跨语言的消息格式，不应该使用类似于Java序列化这样跟编程语言强相关的消息格式。
- 消息的格式可以分为两大类：文本和二进制

基于文本的消息格式

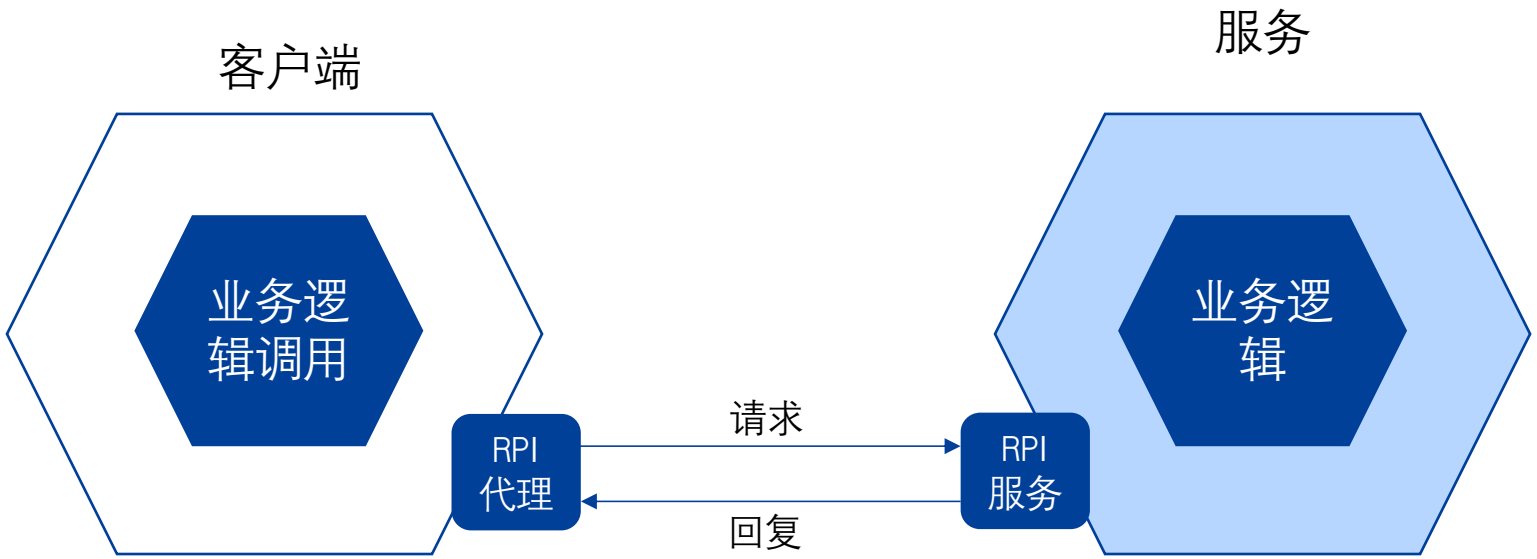
- 基于文本的消息格式包括：JSON和XML。
- 基于文本的消息格式的好处在于：
  - 可读性很高
  - 信息自描述
  - 对消息结构的修改可以做到很好的后向兼容性
- 基于文本的消息格式的弊端在于：
  - 消息往往过度冗长，消息的传递必须包含除了值以外的属性名称
  - 解析文本会引入额外的开销尤其是在消息较大的时候
- 在对效率和性能并不敏感的场景下，可以优先考虑基于文本的消息格式。

二进制消息格式

- 常用的二进制消息格式包括：Protocol Buffers和Avro都提供了一个强类型定义的接口描述文件（IDL）用于定义消息的格式，编译器会自动根据这些格式生成序列化和反序列化的代码。
- 二进制消息格式的好处在于：
  - 通过接口描述文件定义了键以及相对位置，从而在消息的传递只需要携带对应的值减少了信息冗余
  - 通过接口描述文件定义了消息结构，减轻了解析消息结构的开销
- 二进制消息格式的弊端在于：
  - 消息的可读性极差
  - 对消息结构的修改无法做到很好的后向兼容性
- 在对效率和性能敏感的场景下，可以优先考虑二进制消息格式。

（2）基于远程过程调用的同步通信

- 当使用基于远程过程调用的同步通信机制时，客户端中的业务逻辑调用代理接口，这个接口由远程过程调用代理适配器类实现。
- 远程过程调用代理向服务发送请求。
- 该请求由远程过程调用服务器适配器类处理，该类通过接口调用服务的业务逻辑。
- 最后服务将回复发送回远程过程调用代理，该代理将结果返回给客户端的业务逻辑。



基于REST的远程过程调用

- REST是一种使用HTTP协议的进程间通信机制。
- REST的一个关键概念是资源，它通常表示单个业务对象，例如客户或产品，或业务对象的集合。REST使用HTTP动词来操作资源，使用URL引用资源：
  - GET请求返回资源的表示形式，该资源通常采用XML文档或者JSON对象的形式；POST请求创建新资源；PUT请求更新已有资源；DELETE请求删除已有资源
  - URL为/orders 代表订单资源集合；URL为/orders/{id} 代表对应id的一个订单资源
- 将HTTP动词与URL结合起来即为REST风格，例如：
  - GET /orders 代表获取所有订单资源
  - GET /orders/{id} 代表获取对应id的一个订单资源
  - POST /orders 代表创建一个订单资源
  - DELETE /orders/{id} 代表删除对应id的一个订单资源

REST成熟模型

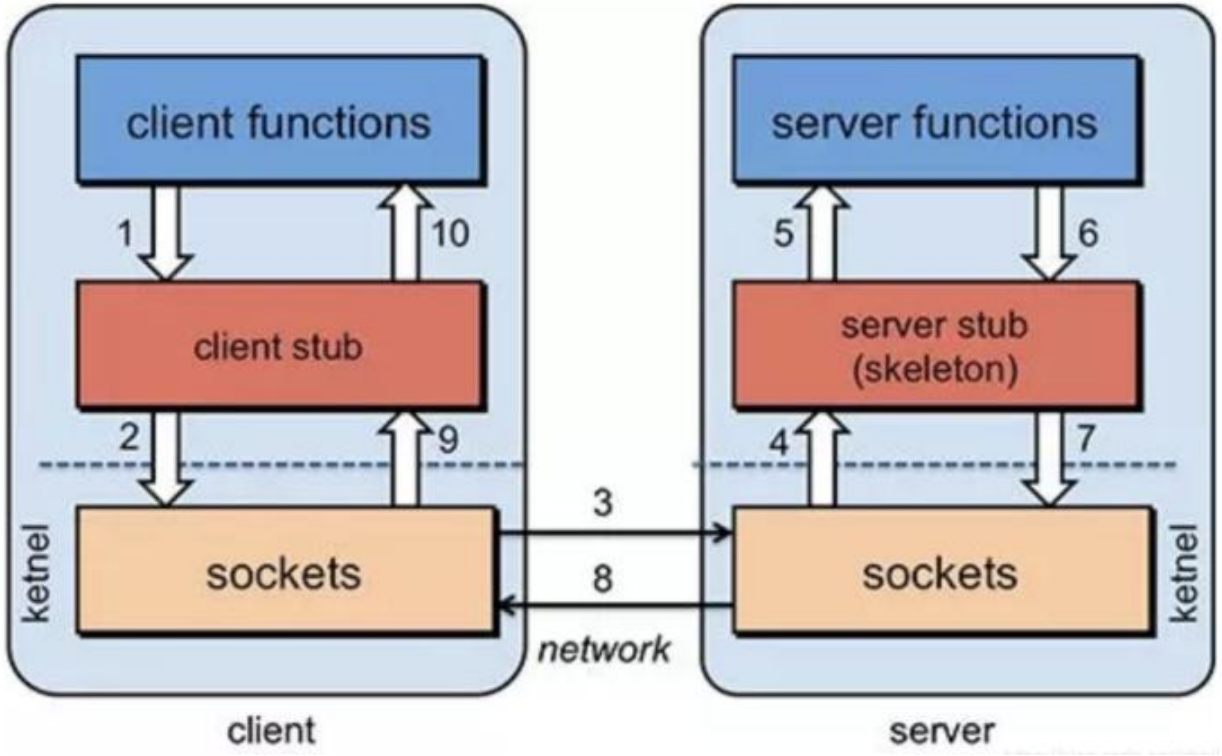
- Lenoard Richardson为REST定义了一个成熟度模型，具体包含以下四个层次：
  - Level 0：客户端只是向服务端点发起HTTP POST请求，进行服务调用。每个请求都指明了需要执行的操作、操作针对的目标和必要的参数。
  - Level 1：引入了资源的概念，要执行对资源的操作，客户端需要发出指定要执行的操作和包含任何参数的POST请求。
  - Level 2：使用HTTP动词来执行操作，客户端仅仅需要发出请求查询参数或者主体即可，这让服务能够借助Web基础设施服务，例如通过CDN来缓存GET请求。
  - Level 3：基于HATEOAS（Hypertext As The Engine Of Application State）原则进行设计，基本思想是在由GET请求返回的资源信息中包含链接，这些链接包含在当前资源状态下允许执行的该资源对应的操作，从而使得客户端无须猜测在资源的当前状态下执行何种操作，从而避免了在客户端代码中写入硬链接的URL。

REST的好处与弊端

- REST的好处如下：
  - REST理解非常简单且直观
  - 可以使用简单的方式（Postman浏览器插件或者curl命令行等）来测试HTTP API
  - 直接支持请求/响应方式的通信
  - 对于防火墙友好
  - 不需要中间代理，简化了系统架构
- REST的弊端如下：
  - 客户端必须知道服务实例的位置（URL）
  - 对于复杂的业务动作难以映射到HTTP动词
  - 在单个请求中难以获取多个资源
  - 请求与响应采用文本的消息格式导致性能可能出现问题

基于gRPC的远程过程调用

- gRPC的出现是为了解决REST中HTTP仅提供有限数量的动词而难以设计支持复杂行为的REST API的问题。它是一个用于编写跨语言客户端和服务端的框架，使用了基于二进制消息的协议，往往采用基于Protocol Buffer的接口描述文件（IDL）定义gRPC API。



gRPC API

```
service OrderService {
  rpc createOrder(CreateOrderRequest) returns (CreateOrderReply) {}
  rpc cancelOrder(CancelOrderRequest) returns (CancelOrderReply) {}
  rpc reviseOrder(ReviseOrderRequest) returns (ReviseOrderReply) {}
  ...
}

message CreateOrderRequest {
  int64 restaurantId = 1;
  int64 consumerId = 2;
  repeated LineItem lineItems = 3;
  ...
}

message LineItem {
  string menuItemId = 1;
  int32 quantity = 2;
}
```

基于gRPC的远程过程调用

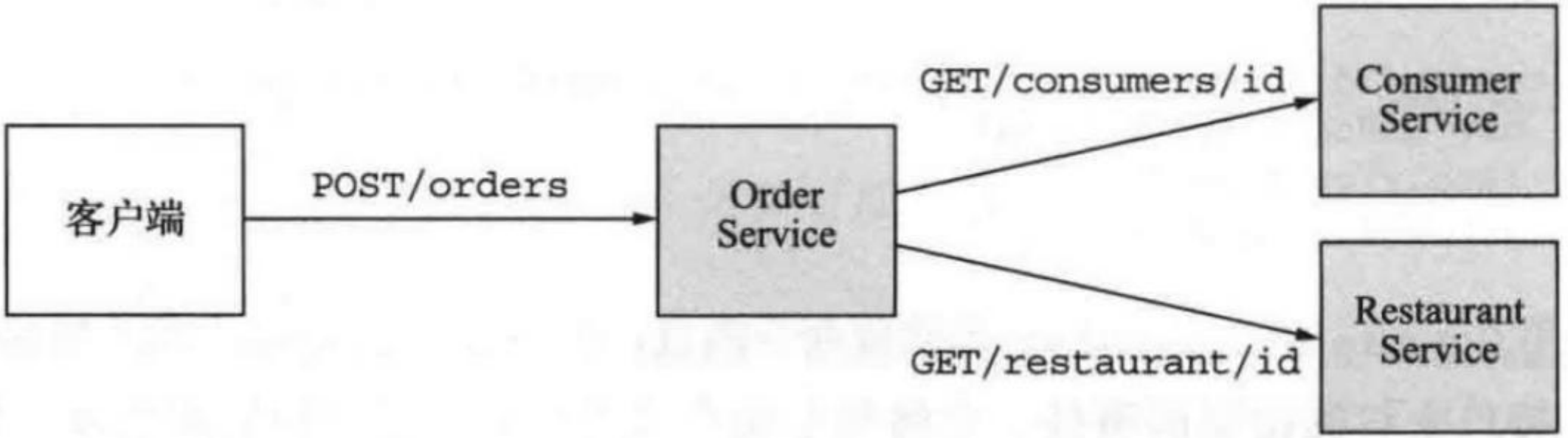
- gRPC API往往由一个或多个的服务和请求/响应消息定义组成，服务定义类似于Java接口，是强类型方法的集合。
- gRPC具有如下好处：
  - 设计具有复杂行为的API非常简单
  - 具有高效、紧凑的进程间通信机制，降低了传输带宽
  - 实现了客户端和用各种语言编写的服务端之间的互操作性
- gRPC具有如下弊端：
  - 旧式防火墙可能不支持HTTP/2
  - 基于JavaScript的Web客户端使用gRPC需要做许多额外的工作
  - 无法做到很好的后向兼容性

gRPC vs REST

指标	gRPC	RESTful
负载	Protobuf（不可读的二进制数据）	JSON（可读性文本数据）
协议	TCP	HTTP
性能	快	较慢
耦合性	强耦合	松散耦合
客户端	强类型客户端、一般自动生成，可支持多语言客户端	一般http client可访问，也可支持多语言
开发者友好程度	客户端比较方便，但是二进制消息不可读	文本消息开发者可读、浏览器可直接访问查看结果
对外开放	需要转换成REST/文本协议	直接对外开放

同步通信的问题

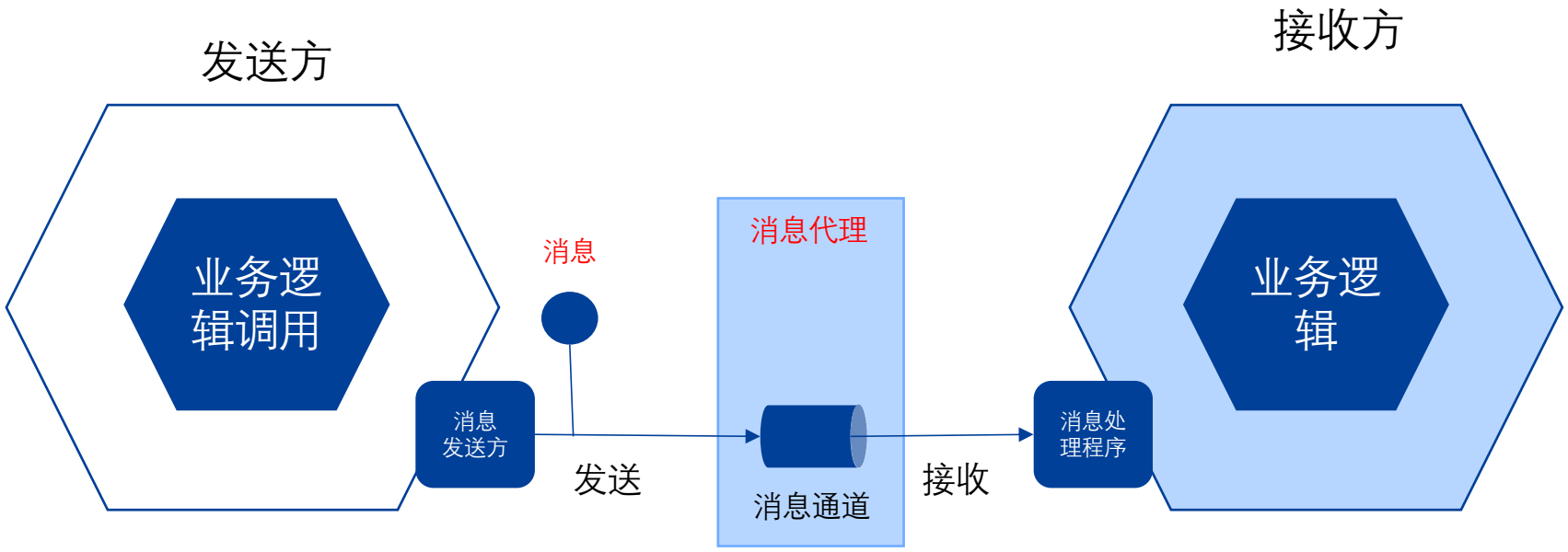
- Order Service使用REST调用其他服务很简单，但要求所有服务同时可用，这会降低API的可用性。
- 单个服务的可用性是99.5%，这个系统操作的可用性就是$99.5\%^3$



如果想要最大化系统可用性，就要降低同步操作量

（3）基于消息的异步通信

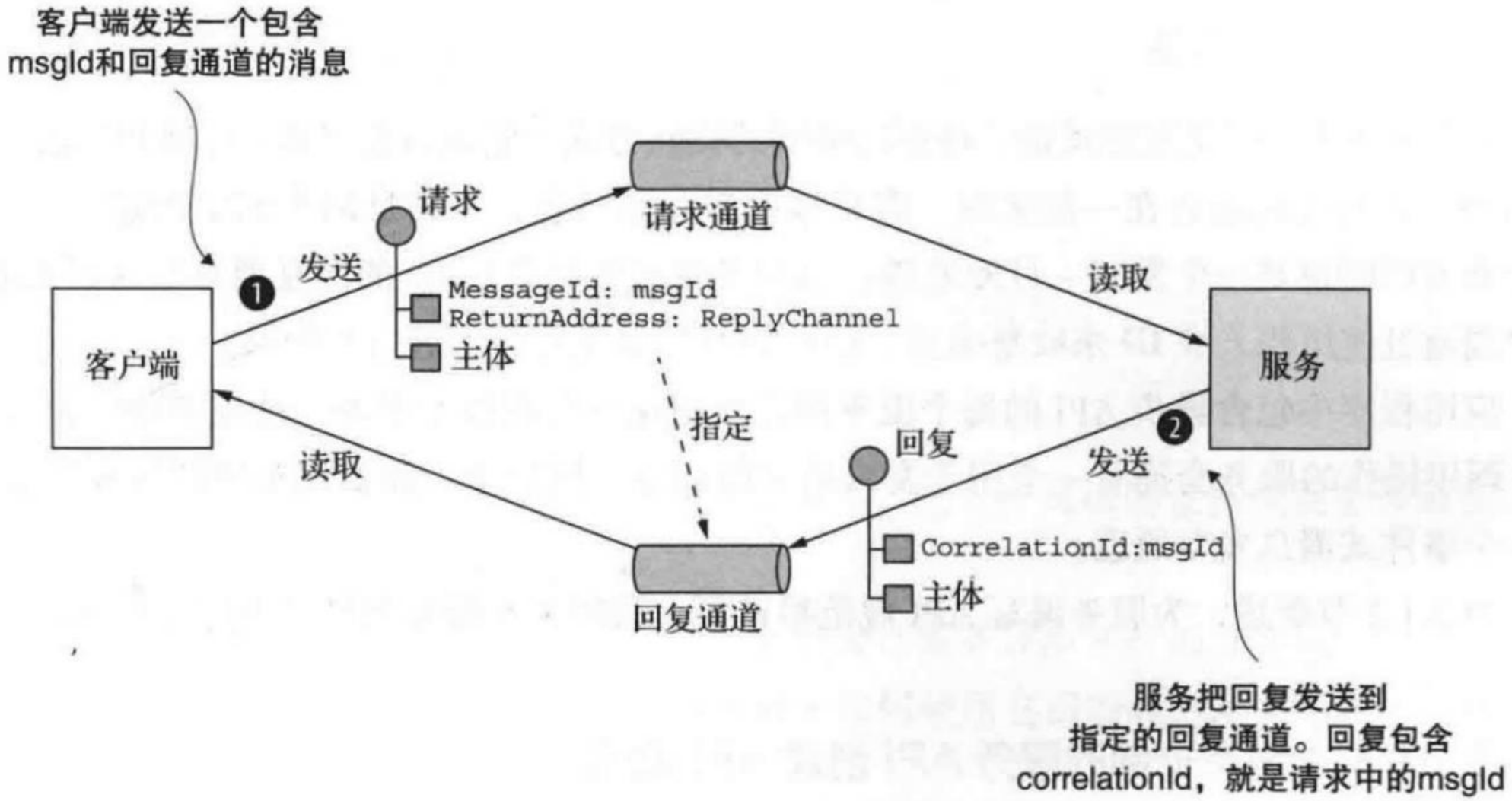
- 当使用基于消息的异步机制时，消息通过消息通道进行交换。
- 发送方中的业务逻辑调用发送端接口，该接口由消息发送适配器类实现，它将消息发送到消息通道后立即返回，然后消息通道向接收方异步地转发消息，接收方中的消息处理程序适配器从消息通道异步地接收到消息之后调用接收方业务逻辑实现的接收端接口完成消息的交换。
- 任意数量的发送方都可以向通道发送消息，任意数量的接收方都可以从通道接收消息



基于消息的异步通信

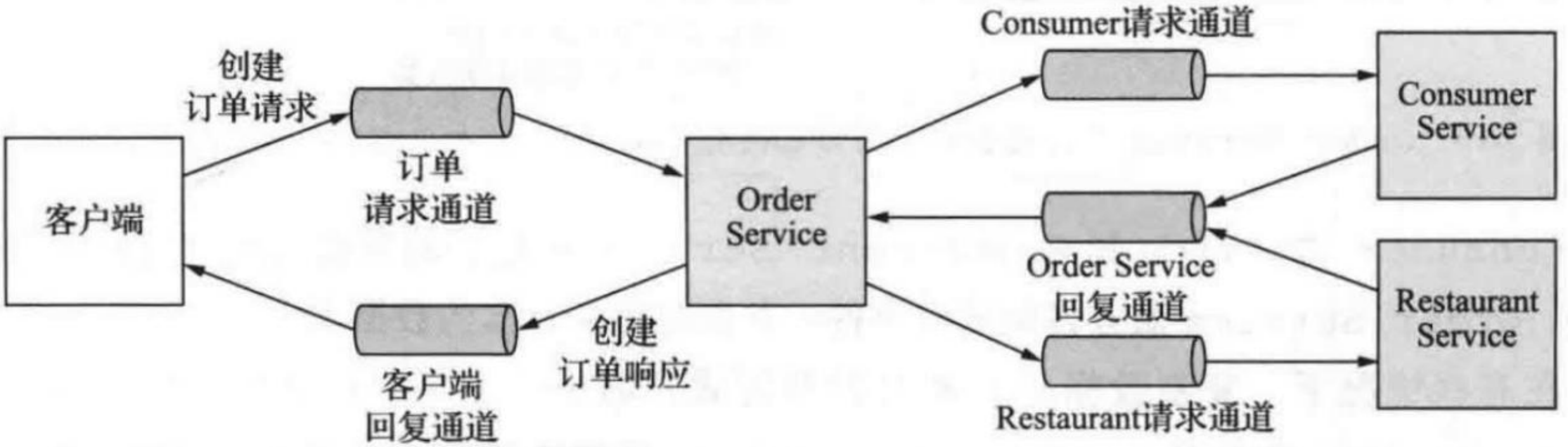
- 消息由消息头部和消息主体组成。
- 消息头部是名称与值对的集合，描述正在发送的数据的元数据，例如：为了防止反复投递生成的唯一消息ID，发件人信息等。
- 消息主体是以文本或二进制格式发送的数据，有以下几种不同类型的消息：
  - 文档：仅包含数据的通用消息，接收者决定如何解释它。
  - 命令：一条等同于RPC请求的消息，它指定要调用的操作及其参数。
  - 事件（状态）：表示发送方这一端发生了重要的事件，事件通常是领域事件，表示领域对象的状态更改。
- 在微服务架构中大量使用命令式消息和事件式消息。

基于消息的异步通信


客户端发送一个包含
msgId和回复通道的消息
请求
请求通道
发送
1
MessageId: msgId
ReturnAddress: ReplyChannel
主体
客户端
读取
服务
指定
回复
2
发送
读取
CorrelationId:msgId
主体
回复通道
服务把回复发送到
指定的回复通道。回复包含
correlationId，就是请求中的msgId

基于消息的异步通信

- Order Service采用异步交换消息完成订单创建。



基于消息的异步通信

- 消息代理是所有消息的中介节点。
- 发送方将消息写入消息代理，消息代理将消息发送给接收方。
- 使用消息代理的一个重要好处是发送方不需要知道接收方的网络位置从而降低微服务间的耦合性，另外一个好处是消息代理缓冲消息，直到接收方能够处理它们。
- 不同的消息代理都有各自的概念来实现消息通道，常用的消息代理对比图如下：

消息代理	点对点通道	发布–订阅通道
JMS	队列	主题
Apache Kafka	主题	主题
RabbitMQ	交换+队列	组播式交换+每客户端队列
AWS Kinesis	流	流

MQTT vs Kafka

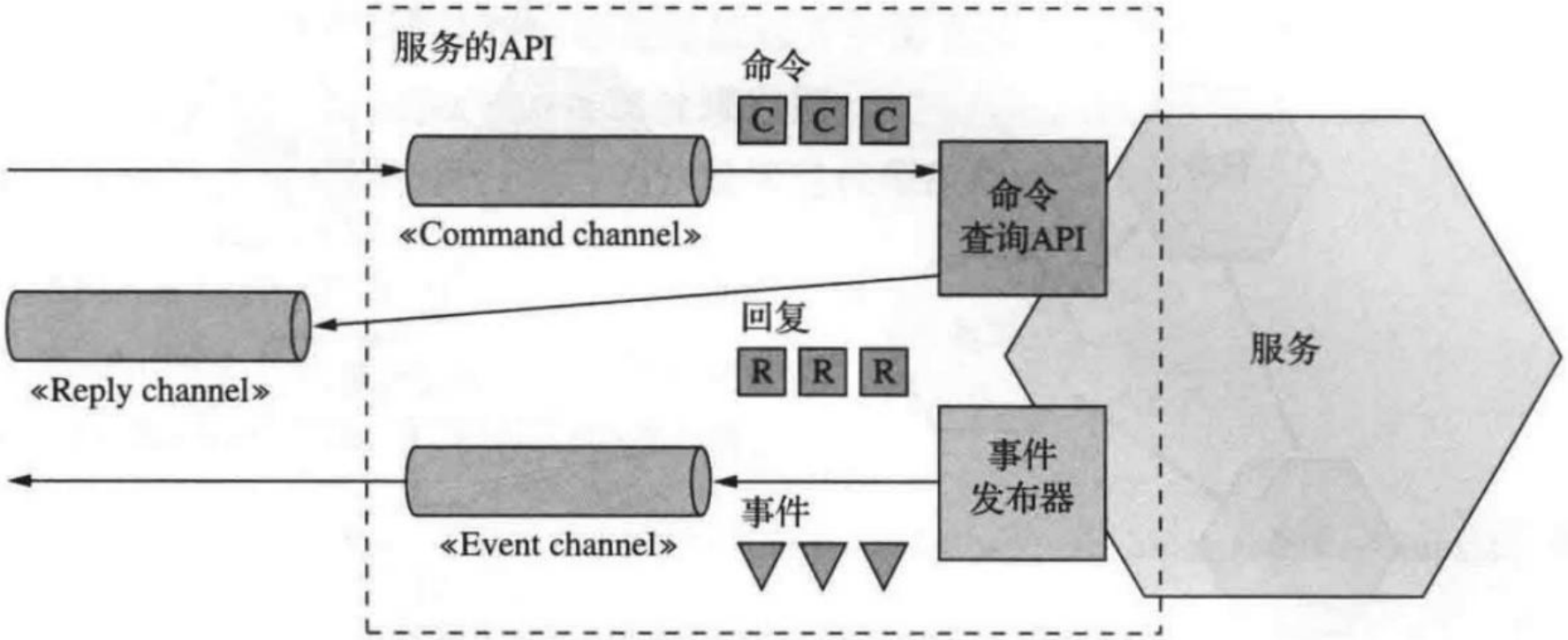
指标	ActiveMQ	Apache Kafka
开发语言	Java	Scala & Java
客户端支持语言	Java、C、C++、Python、PHP、Perl、.net等	官方支持Java，开源社区有多语言版本
协议支持	OpenWire、STOMP、REST、XMPP、AMQP	自有协议，社区封装了HTTP协议支持
消息推拉模式	Pull/Push均有支持	Pull
单机吞吐量	万级	十万级
是否有序	可以支持有序	多Client保证有序
管理界面	提供较为简单的管理页面	官方只提供命令行版，Yahoo开源自己的Kafka Web管理界面Kafka-Manager

基于消息的异步通信

- 消息机制的最具有价值的特性在于它足够灵活，它可以支持之前描述的所有交互方式。
  - 异步请求/响应：客户端和服务端通过交换一对消息来实现异步请求/响应，当服务端从请求通道接收到客户端发来的消息并处理结束后，会将回复发送到客户端指定的回复通道，客户端使用相关性ID将回复消息和请求进行匹配。
  - 单向通知：消息机制内置了对于单向通知的支持，客户端将消息（通常是命令式消息）发送到服务所拥有的点对点通道，服务订阅该通道并处理该消息，但是服务不会发回回复。
  - 发布/订阅：消息机制内置了对于发布/订阅的支持，客户端将消息发布到由多个接收方订阅的发布/订阅通道即可实现发布/订阅的交互模式。

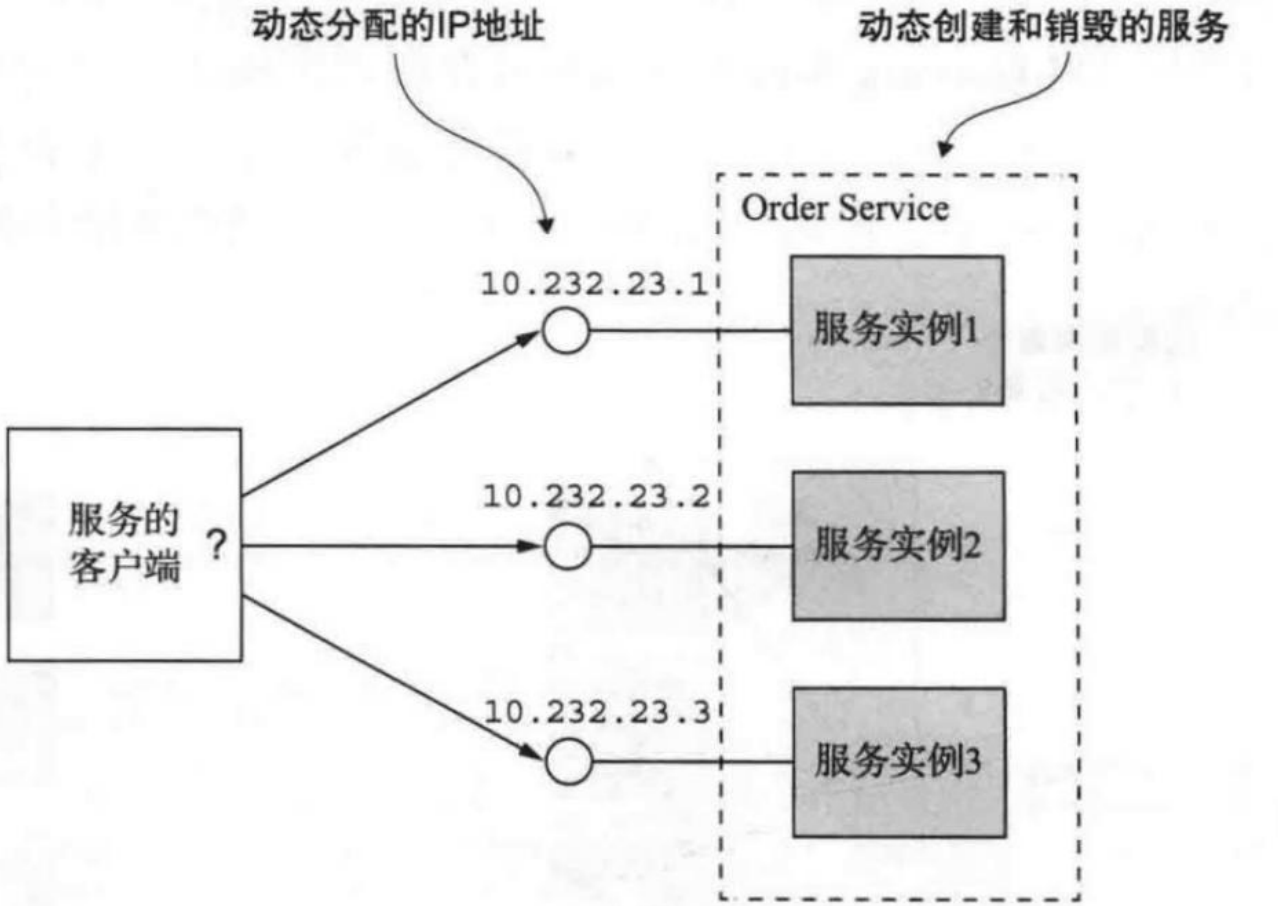
基于消息的异步通信

- 消息机制创建API的规范



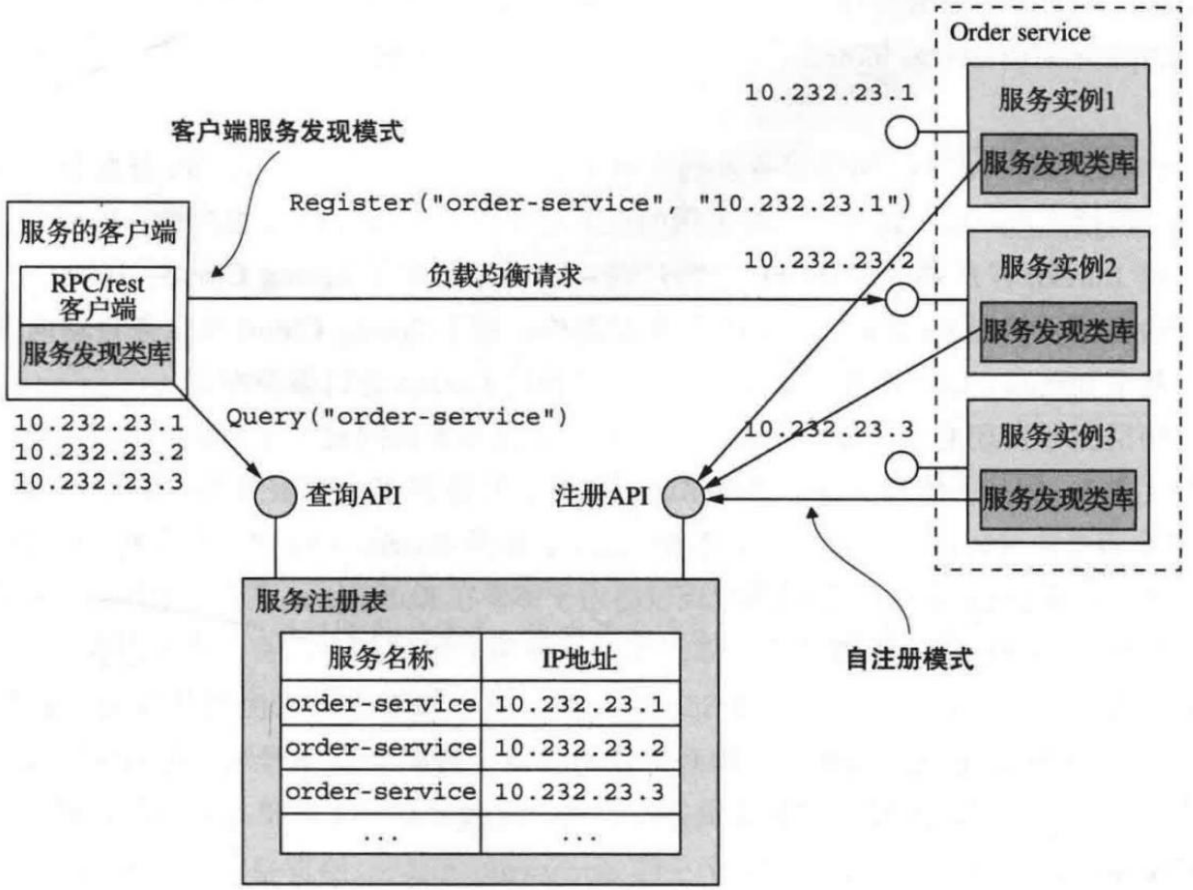
（4）服务发现

- 服务具有动态地址，如何找到需要的服务进行通信



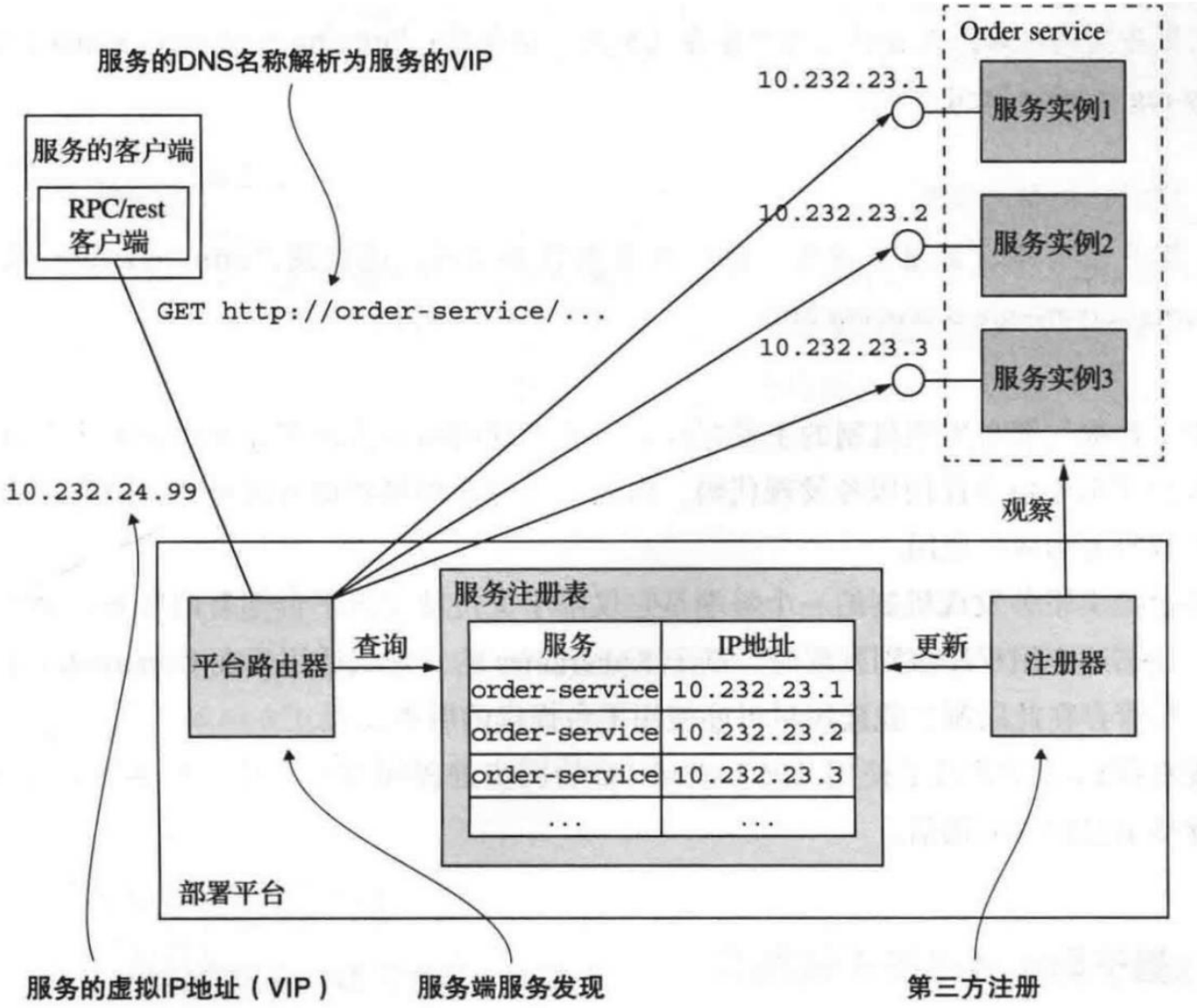
（4）服务发现

- 应用层服务注册模式（Etcd、Eureka、Consul、 Zookeeper）
- 客户端发现模式：
  - 服务注册表跟踪服务实例
  - 服务实例采用自注册模式
  - 客户端查询服务注册表
- 可以实现点对点的网状通讯，即去中心化的通讯。可以有效避开单点造成的性能瓶颈和可靠性下降等问题。
- 服务客户端与服务注册表耦合。



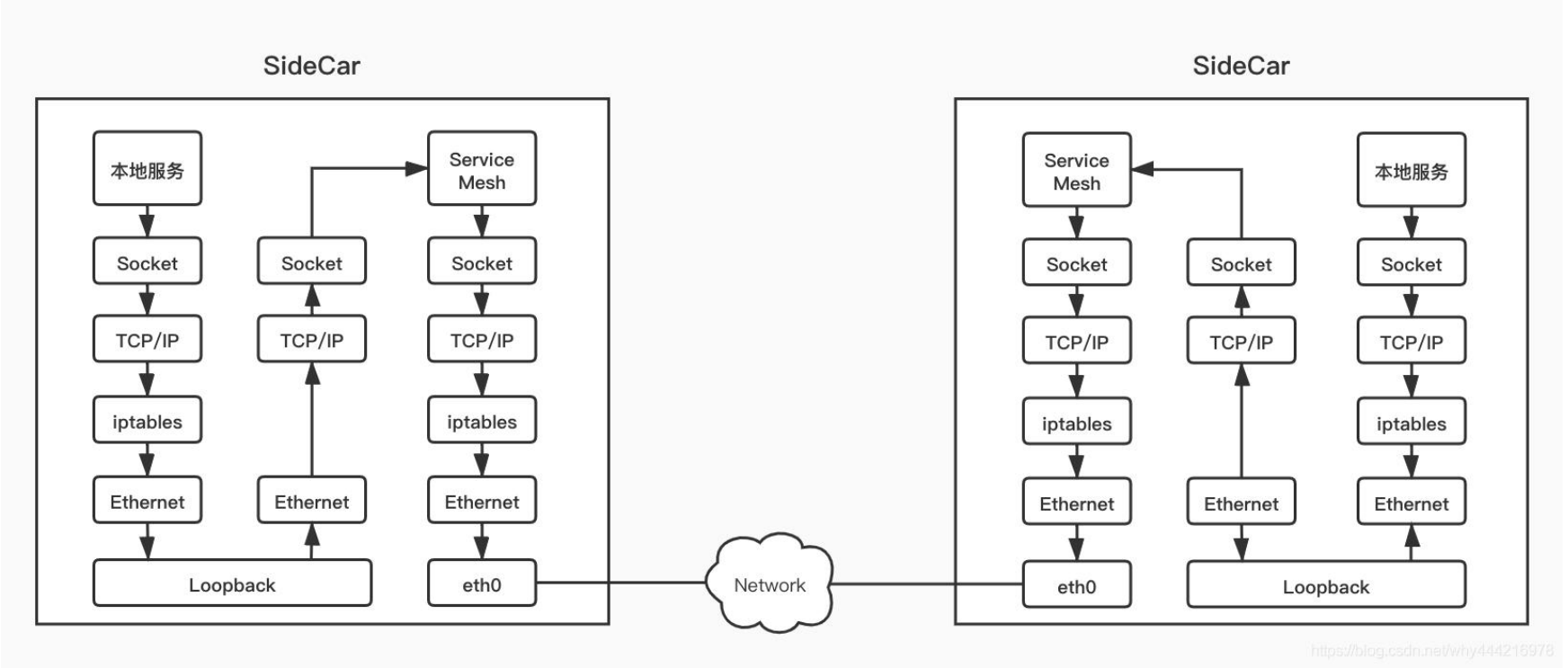
（4）服务发现

- 服务端发现模式（路由器）
- 语言无关，客户端无感知
- 部署平台壁垒
- 负责转发请求的路由器或者负载均衡器可能变成性能的瓶颈



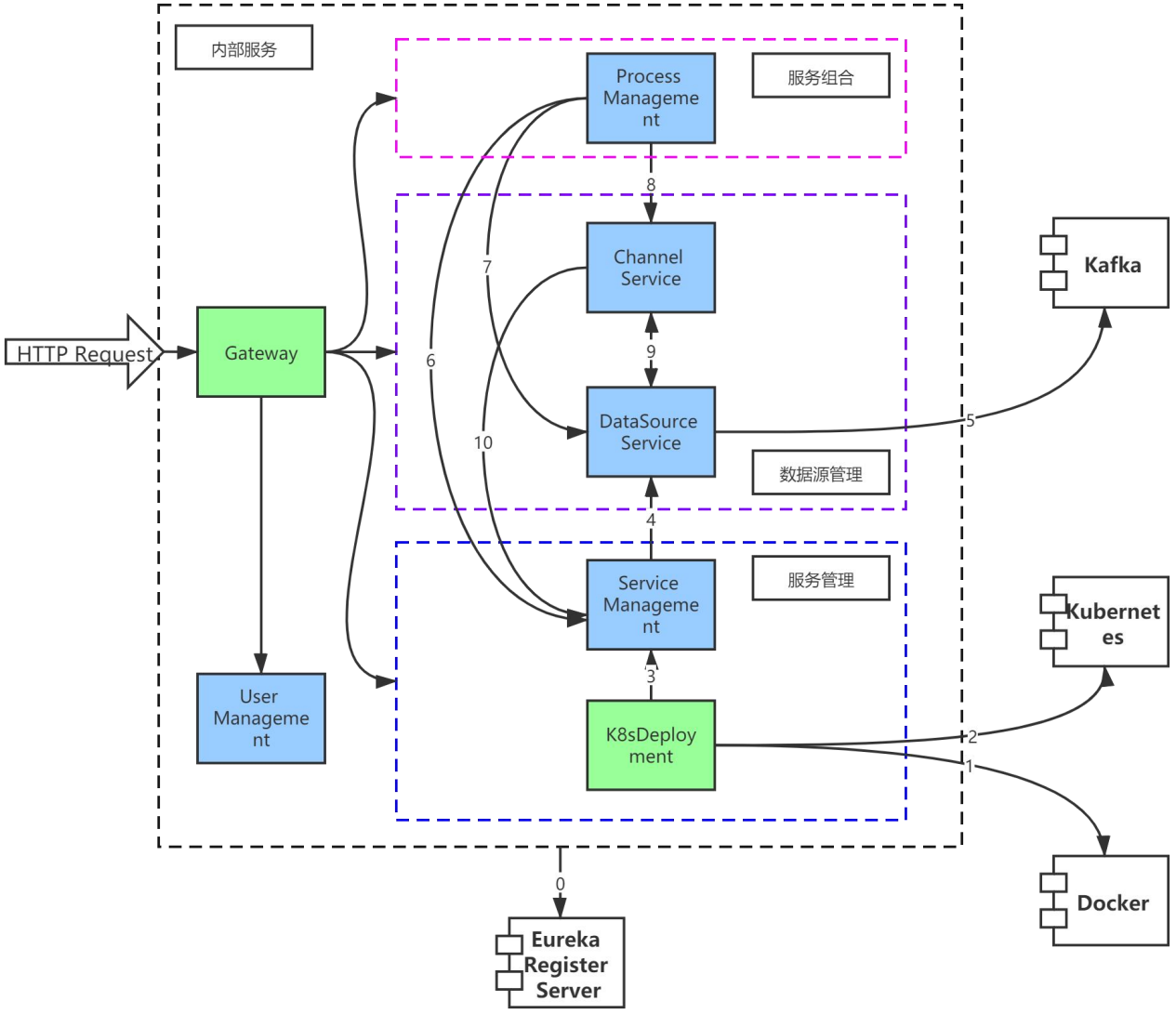
（4）服务发现

- 边车模式（Sidecar）
- Service Mesh
- 将客户端发现模式独立出来，通过本地环回接口或地址（Loopback），直接发送到本机的本地代理程序ServiceMesh



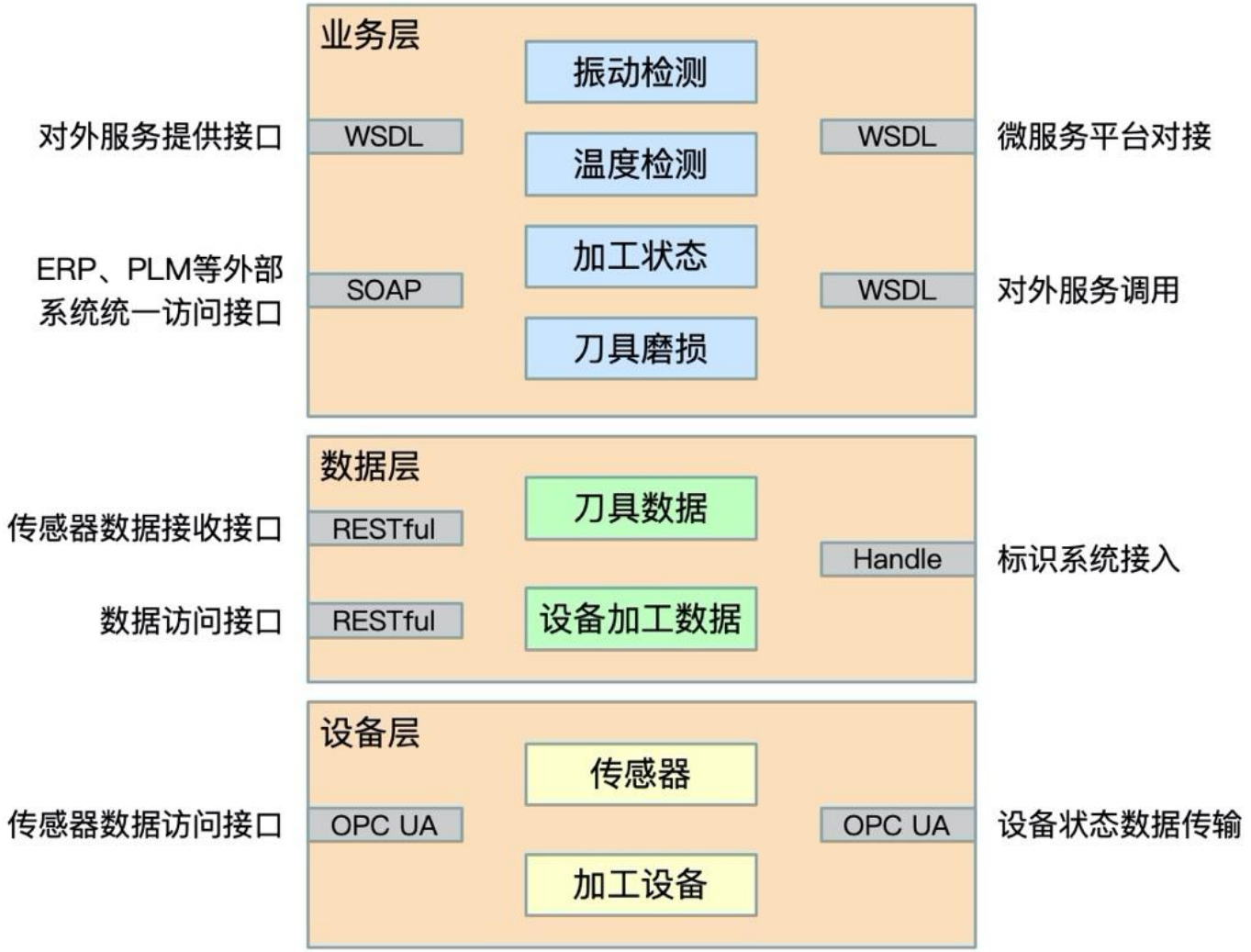
（4）服务发现

- 以IST工业互联网边缘计算管理平台为例



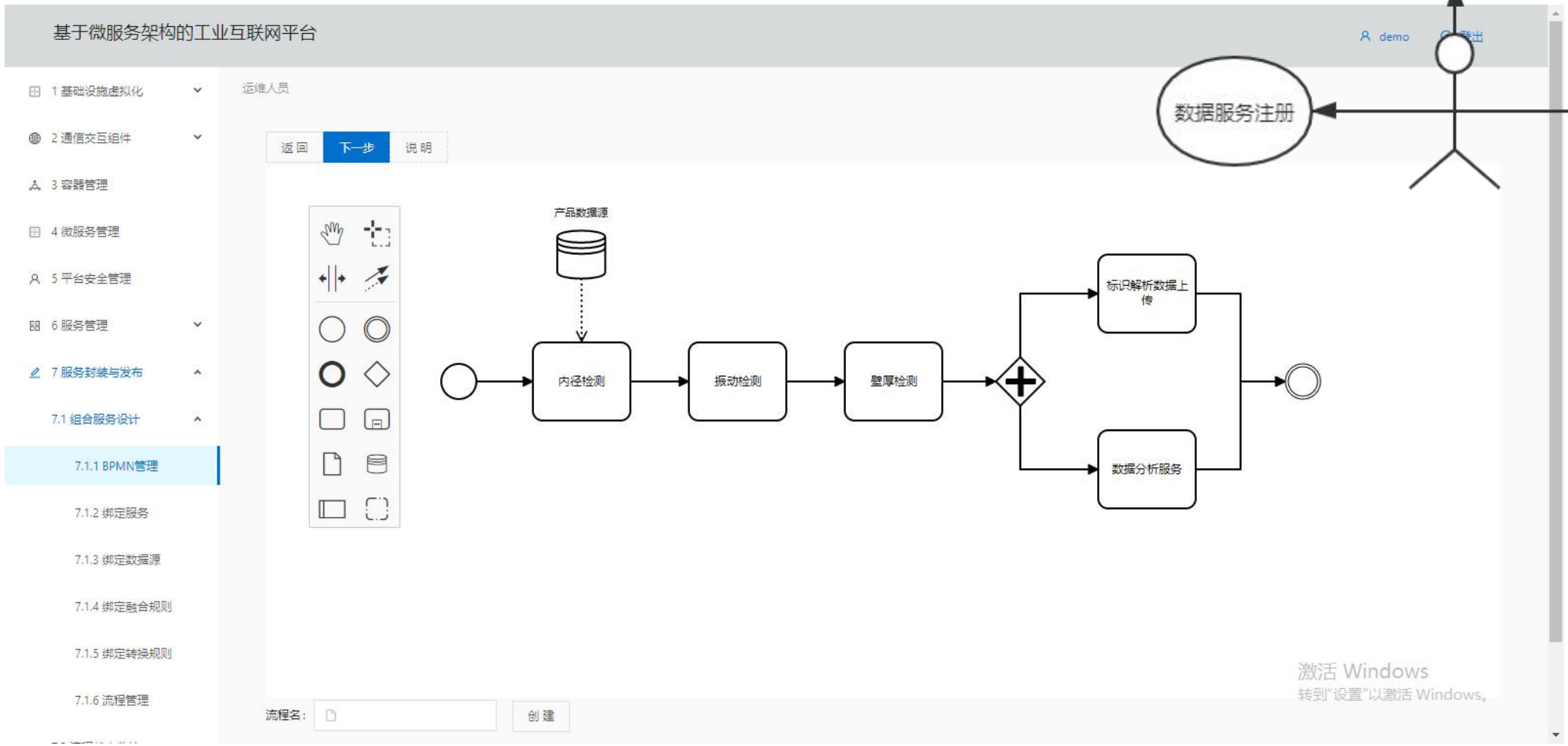
服务间通信

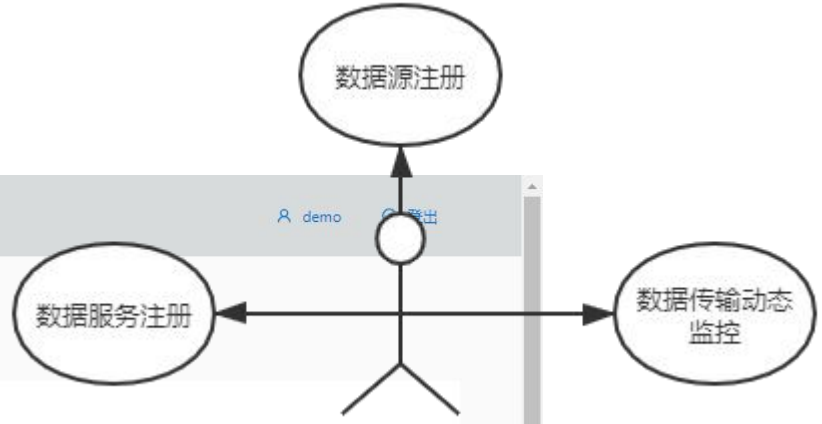
- 以IST工业互联网边缘计算管理平台为例
- 以某边缘节点为例



服务间通信

- 以IST工业互联网边缘计算管理平台为例

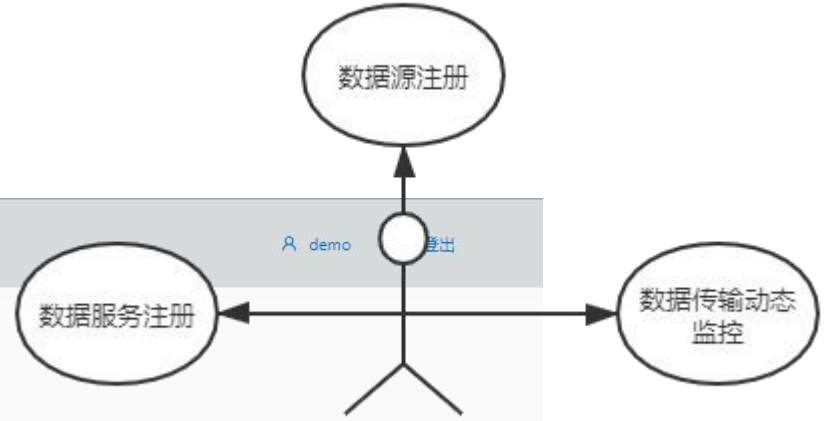




服务间通信

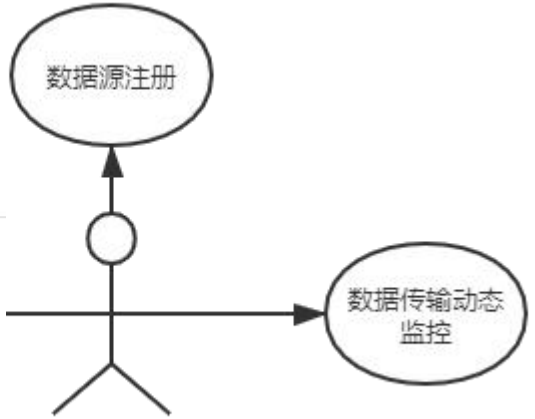
- 以IST工业互联网边缘计算管理平台为例

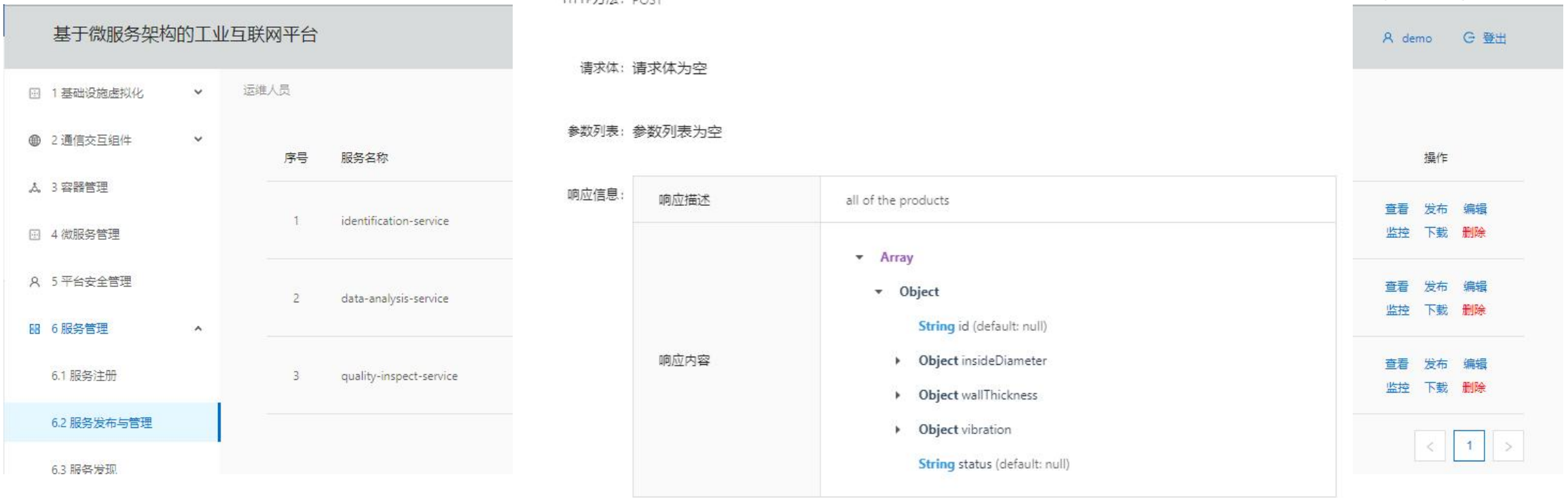




服务间通信

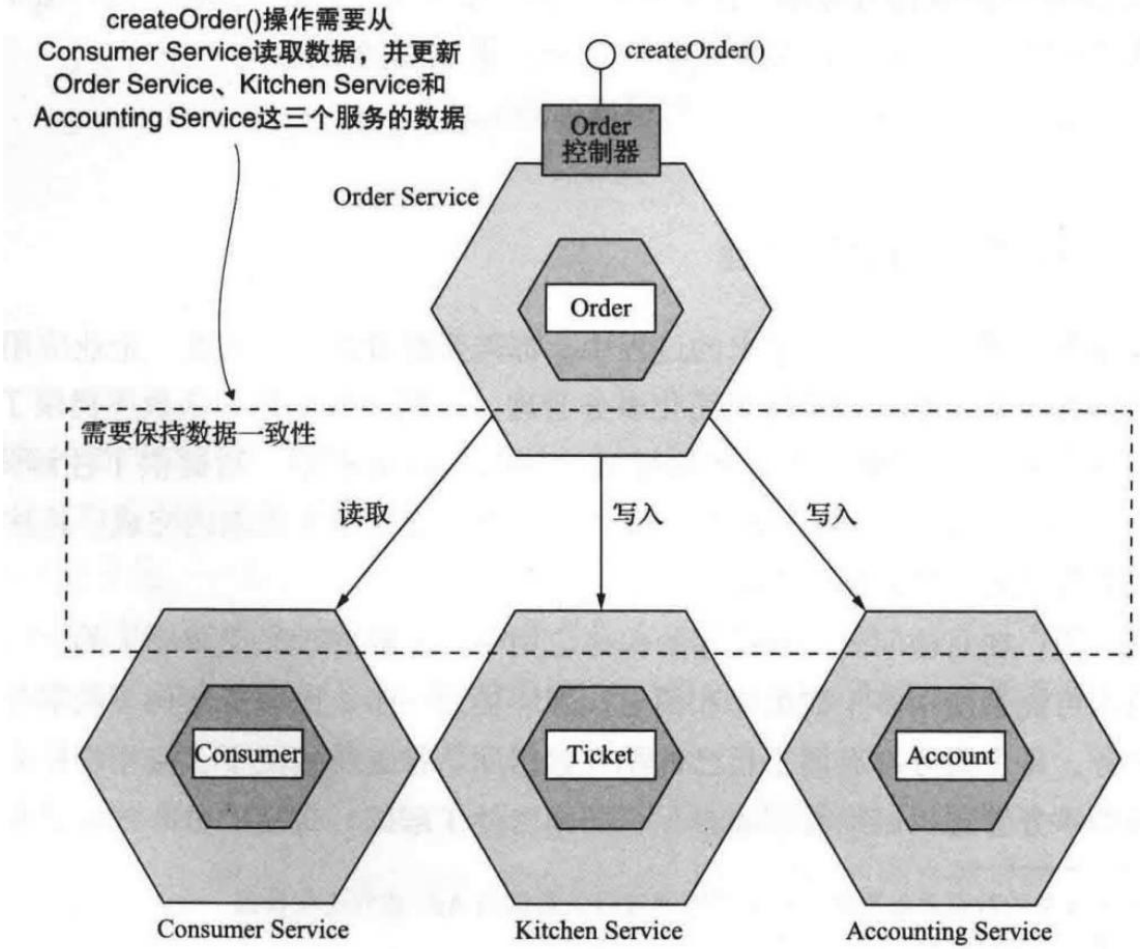
- 以IST工业互联网边缘计算管理平台为例





3.2 分布式事务

- 将一个大的应用系统拆分为多个可以独立部署的微服务后，当需要各个微服务远程协作才能完成某些事务操作时，这就涉及到了分布式事务的问题。

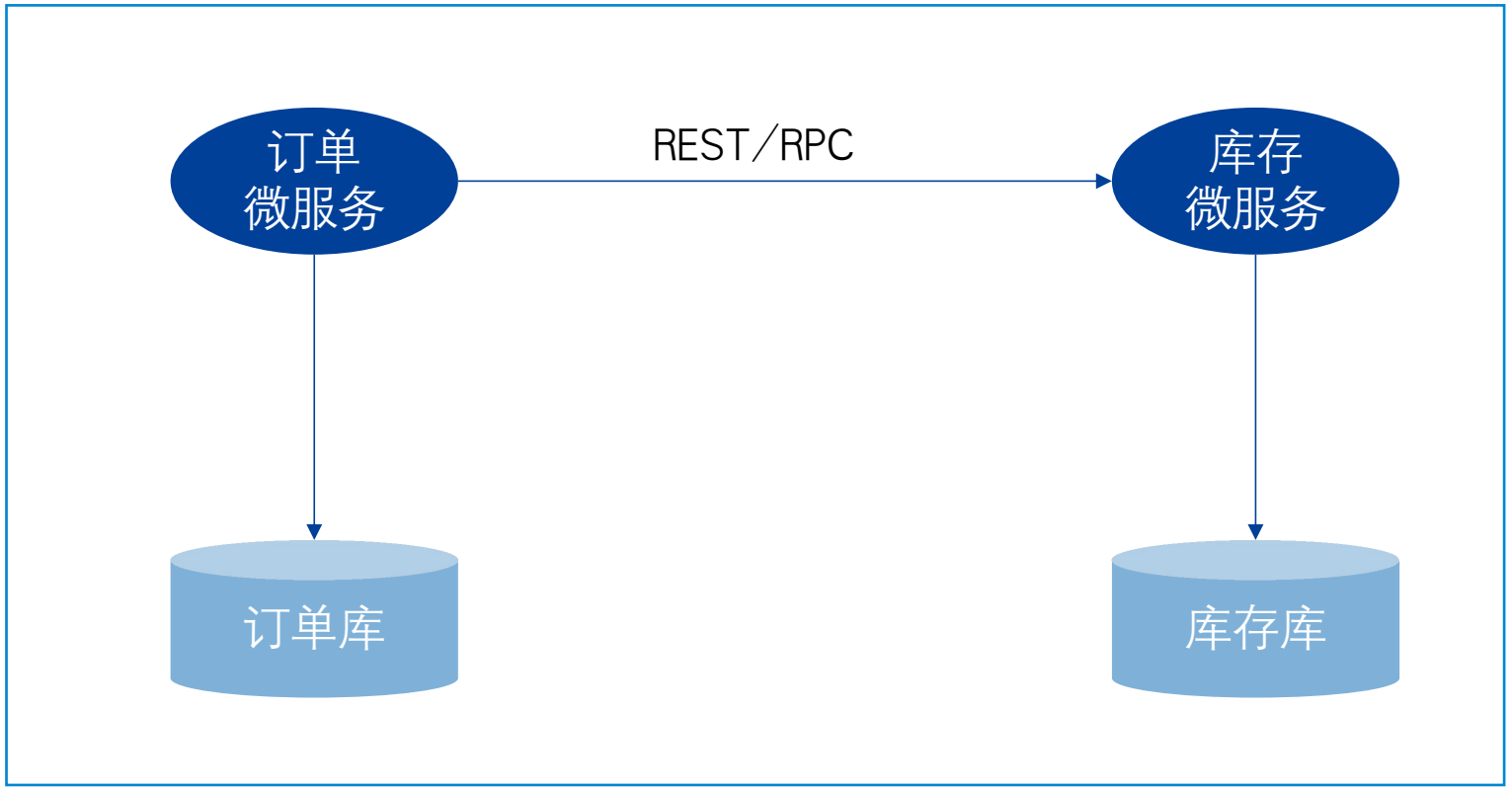


3.2 分布式事务

- 分布式事务主要会在以下三种场景下产生，微服务架构中主要是第一种场景居多：
  - 跨微服务进程
  - 跨数据库实例
  - 多服务访问单数据库
- 分布式事务产生的本质就是：产生了不同的数据库会话来操作数据库中的数据。

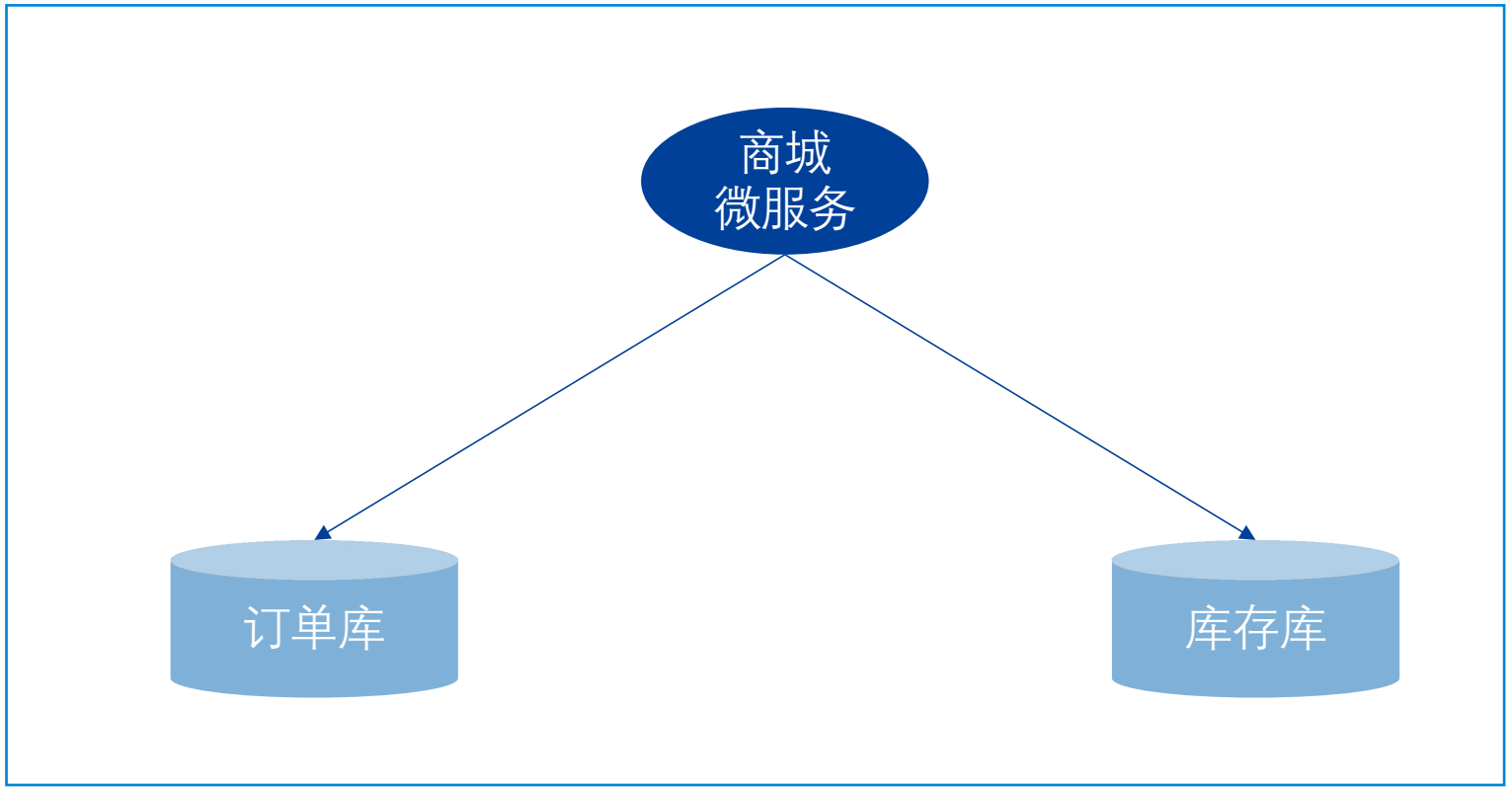
跨微服务进程

- 各个微服务之间通过远程REST或者RPC调用来系统完成业务操作，从而导致分布式事务问题



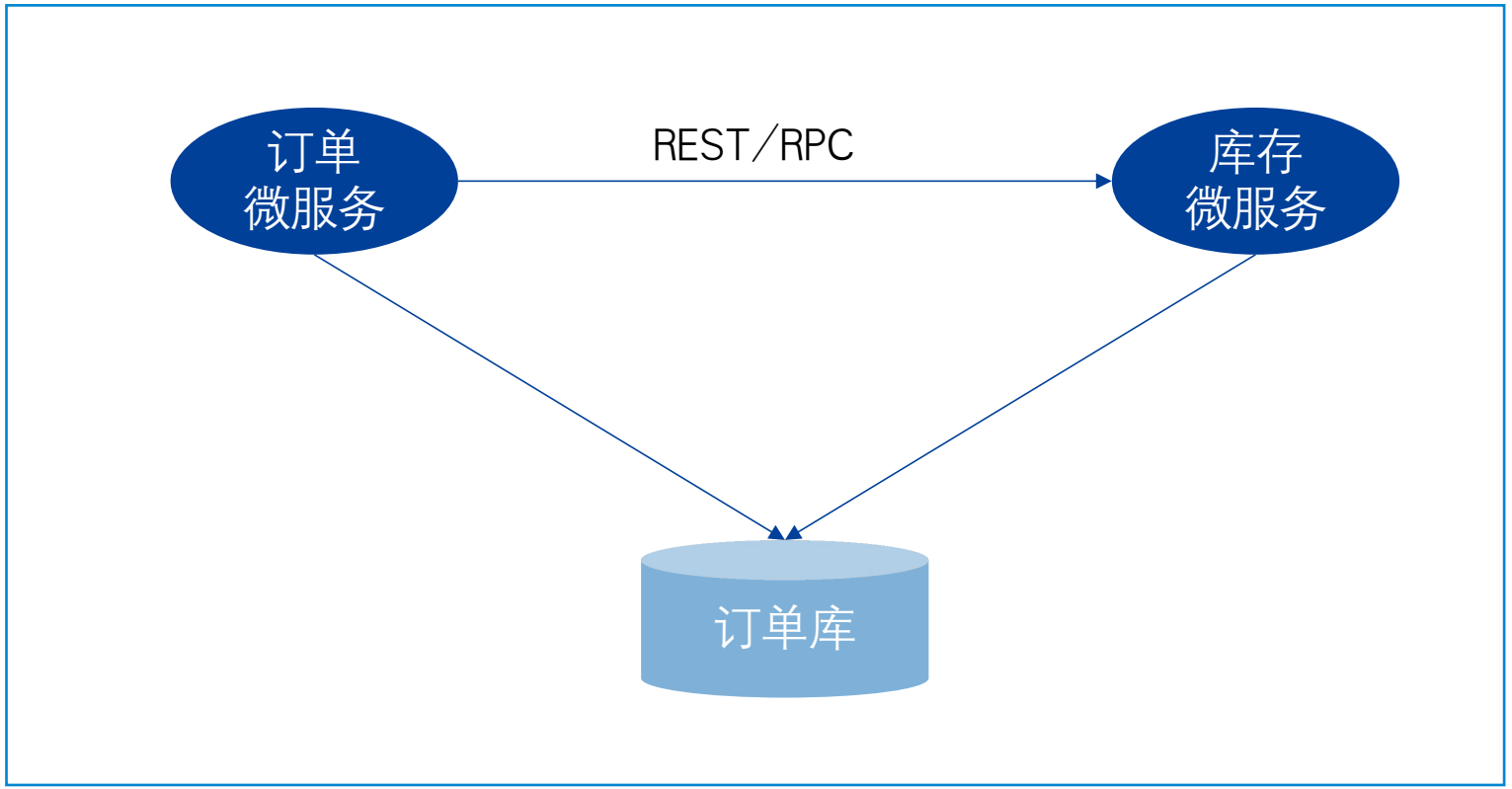
跨数据库实例

- 单个微服务访问多个数据库实例，也就是跨数据源访问时会产生分布式事务



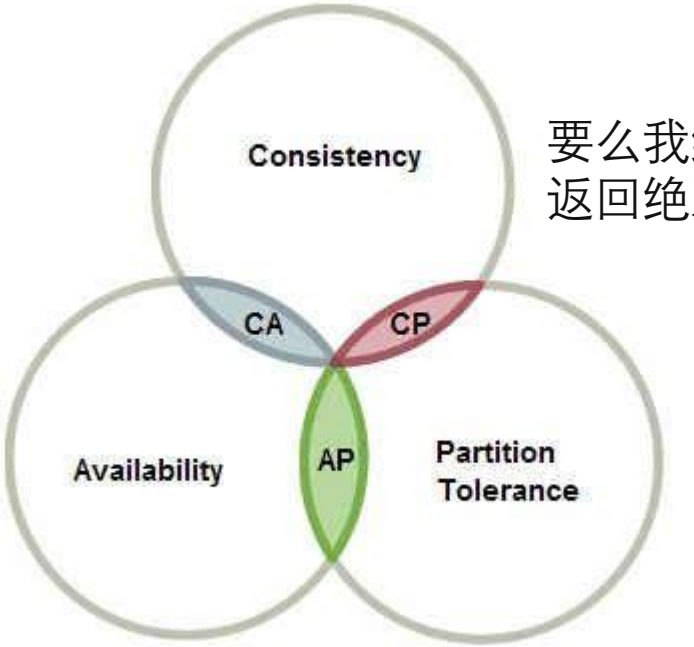
多服务访问单数据库

- 多个微服务访问同一个数据库，由于本质上也是通过不同的数据库会话来操作数据库，此时就会产生分布式事务



CAP理论

- CAP理论是分布式领域著名的理论，它是一致性（Consistency）、可用性（Availability）和分区容忍性（Partition Tolerance）首字母的缩写
  - 一致性（C）：在分布式系统中的所有数据备份，在同一时刻是否同样的值。
  - 可用性（A）：保证每个请求不管成功或者失败都有响应。
  - 分区容忍性（P）：系统中任意信息的丢失或失败不会影响系统的继续运作。
- 根据CAP理论，在分布式系统中，不会同时具备CAP三个特性，只能同时具备其中两个。



要么我给您返回一个错误，要么我给你返回绝对一致的最新数据

我一定会给您返回数据，不会给你返回错误，但不保证数据最新

我会一直运行，不管我的内部出现何种数据同步问题

CAP理论

- 因此分布式系统也就有了如下三种组合：1．AP 2．CP 3．CA

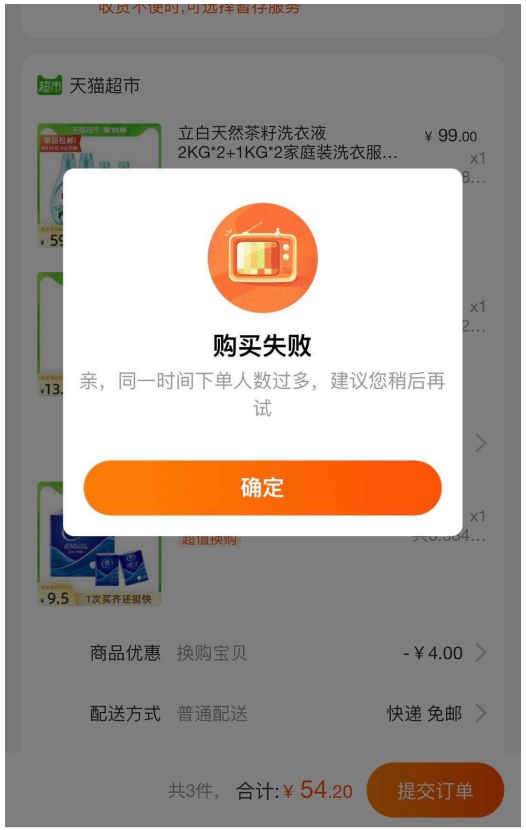
组合方式	描述
AP	放弃一致性，追求系统的可用性和分区容忍性，在实际场景中大部分分布式系统，包括微服务架构往往采用AP的方式，舍弃了一致性采用最终一致性，允许多个节点的数据在一定时间内存在差异，一段时间后达到数据一致的状态
CP	放弃可用性，追求系统的一致性和分区容忍性，往往在跨行转账业务等追求强一致性的系统中使用对应CP组合
CA	放弃分区容忍性，追求系统的一致性和可用性，但是由于在分布式系统下分区是必定会发生的，因此往往不会使用该组合

BASE理论

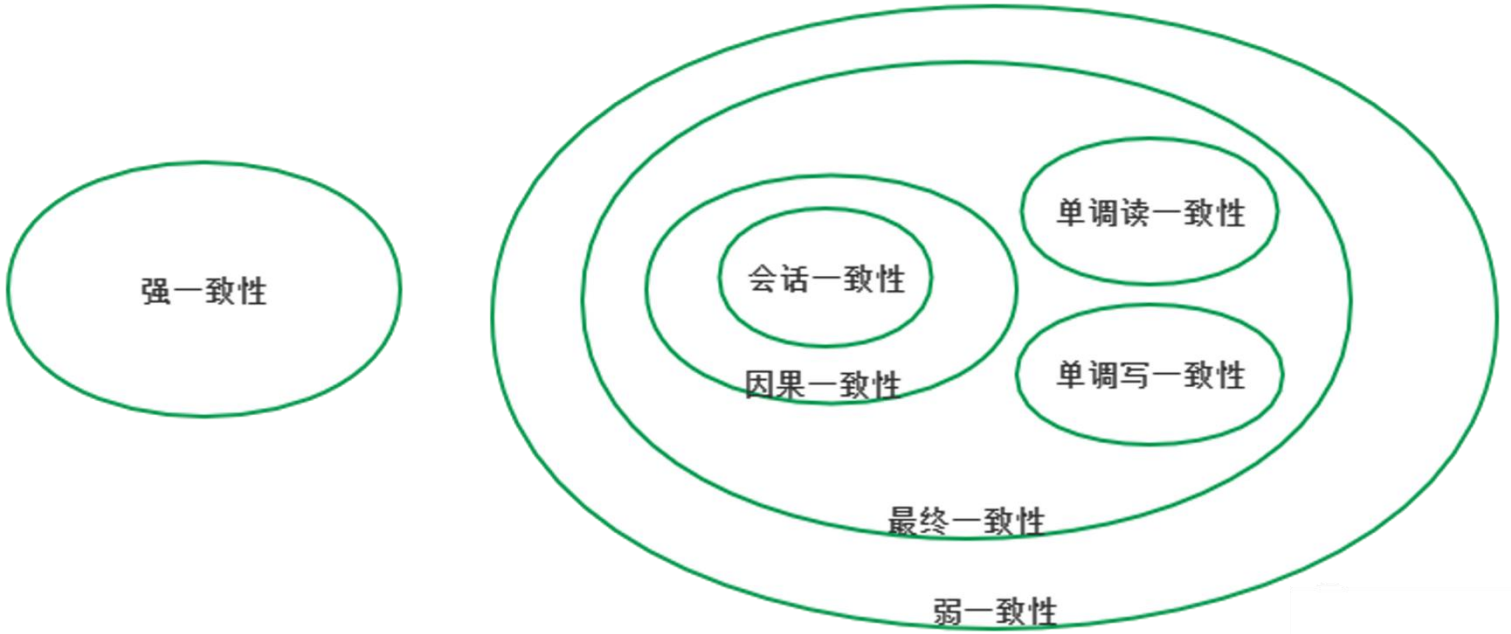
- BASE理论是对CAP理论中AP的一个扩展，它通过牺牲强一致性来获得可用性，往往用在以微服务架构为代表的分布式系统中
- BASE理论中的BASE是基本可用（Basic Available）、软状态（Soft State）和最终一致性（Eventually Consistent）的缩写：
  - 基本可用：系统出现故障时，允许损失系统的部分可用性，比如：响应时间或者功能上的损失，但是要保证系统基本可用
  - 软状态：允许系统中存在中间状态从而允许系统各个微服务节点之间的数据同步存在延迟，如：订单中的“支付中”，“退款中”等状态就是中间状态，最终会变成“支付成功”或者“支付失败”等状态
  - 最终一致性：系统中各个微服务节点的数据副本经过一段时间的同步以后，最终能达到一致性的状态
- 符合BASE理论的分布式事务可以称为柔性事务，如：TCC、Saga等

BASE理论

- 基本可用
- 软状态
- 最终一致性







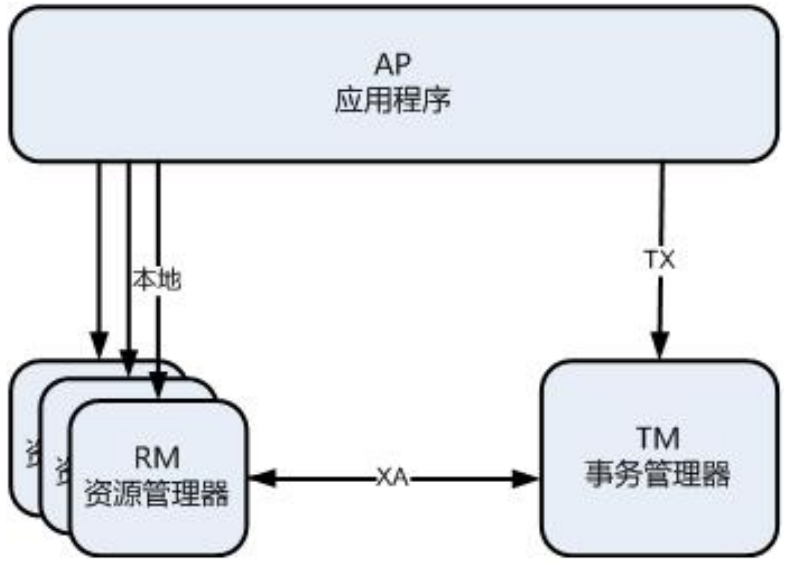
强一致性分布式事务——DTP模型

- DTP模型(Distributed Transaction Processing，分布式事务模型)是X/Open 组织定义的一套分布式事务标准，主要定义了实现分布式事务的规范和 API，具体的实现由相应的厂商来实现。
- DTP核心概念：
  - 事务：一个事务就是一个数据库本身提供的完整的工作单元，具备ACID特性
  - 全局事务：由事务管理器管理的全局事务，能够一次性操作多个资源管理器
  - 分支事务：每个资源管理器中独立执行的事务
  - 控制线程：执行全局事务的线程，这个线程用来关联应用程序、事务管理器和资源管理器

强一致性分布式事务——DTP模型

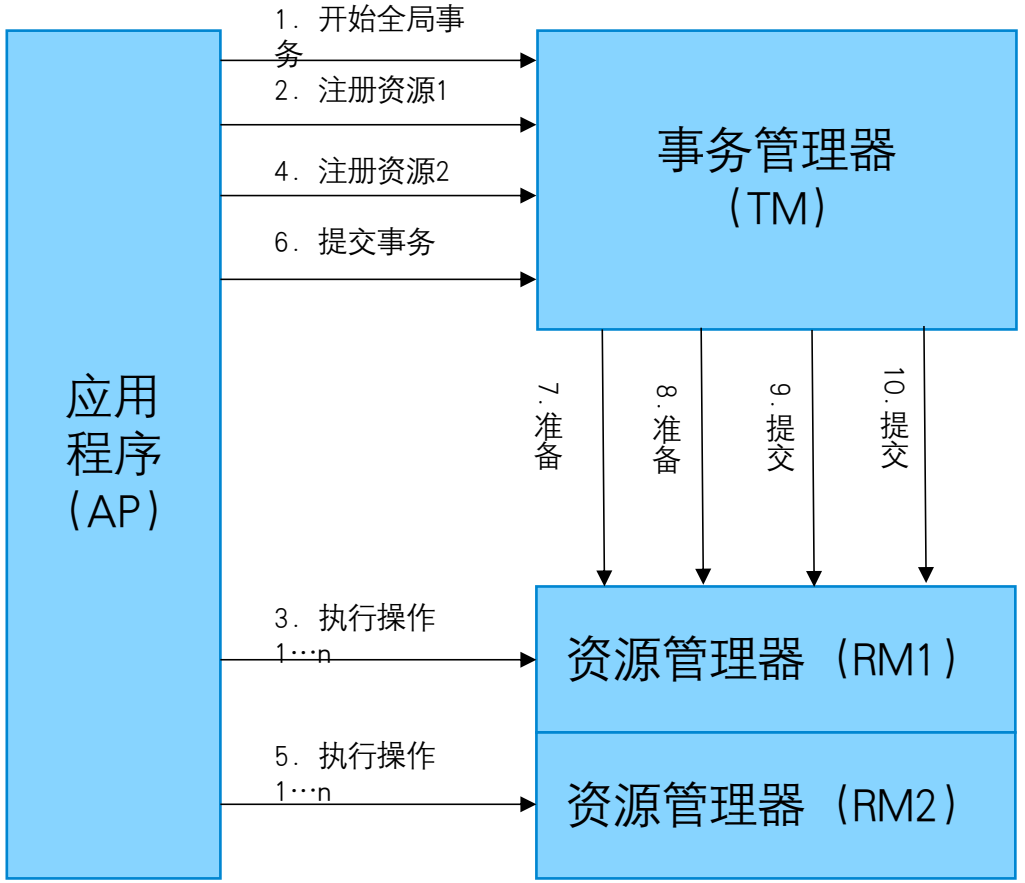
DTP核心组件：

- DTP 模型中包含一个应用程序（AP，Application Program），一个全局事务管理器（TM，Transaction Manager）和多个资源管理器（RM，Resource Manager）。
  - AP：应用程序，DTP分布式事务模型的应用程序
  - RM：资源管理器，管理每个数据库的连接数据源
  - TM：事务管理器，负责管理全局事务状态与参与的资源，协同资源一起提交或回滚



强一致性分布式事务——DTP模型

- DTP模型的主要执行流程如下，其核心在于基于2PC模型只有在所有的事务都准备好了之后才提交每个事务，从而通过每个数据库本身的事务实现全局的分布式事务

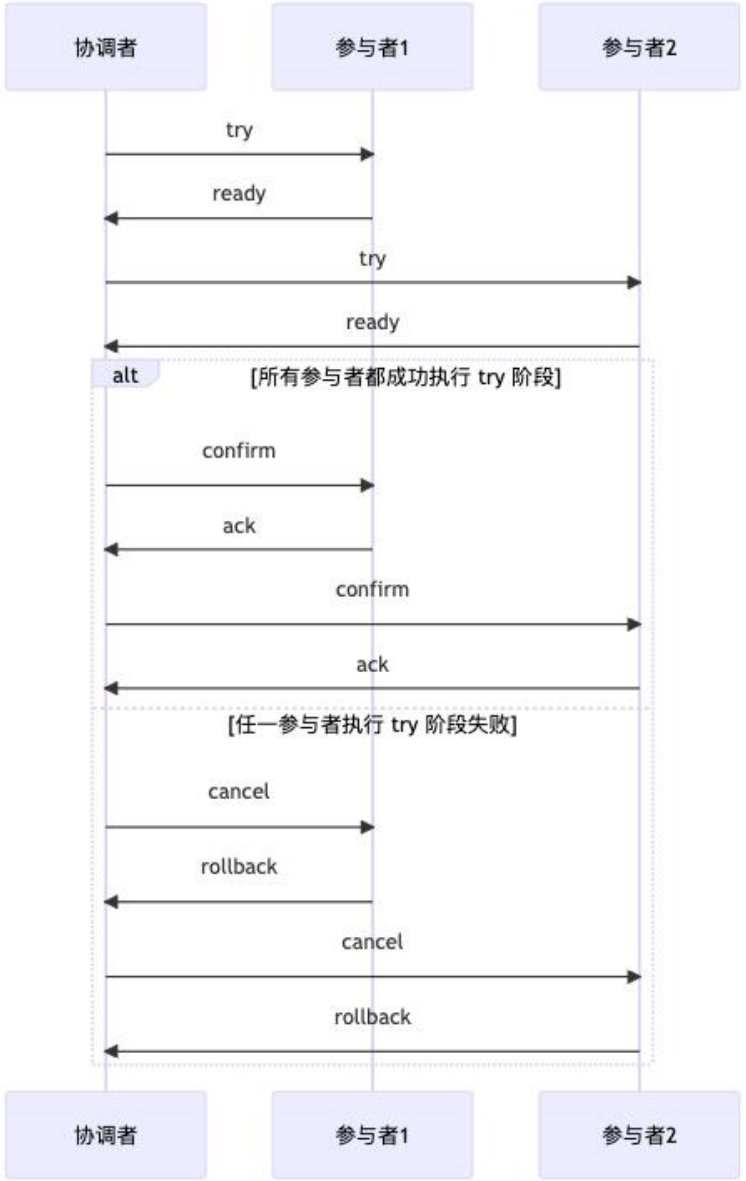


强一致性分布式事务——DTP模型

- DTP模型存在如下缺点：
  - 同步阻塞问题：事务的执行过程中，所有参与事务的节点都会开启一个比较漫长的事务并对其占用的公共资源加锁直到全局事务提交，导致其他访问公共资源的微服务阻塞
  - 单点故障问题：如果事务管理器发生故障，则资源管理器会一直阻塞
  - 数据不一致问题：如果在Commit阶段，由于网络或者部分资源管理器发生故障，导致部分资源管理器没有接收到事务管理器发送过来的Commit消息，会引起数据不一致的问题
  - Commit丢失问题：如果在Commit阶段，事务管理器发出Commit消息后宕机，并且唯一接收到这条Commit消息的资源管理器也宕机了，则无法确认事务是否已经提交

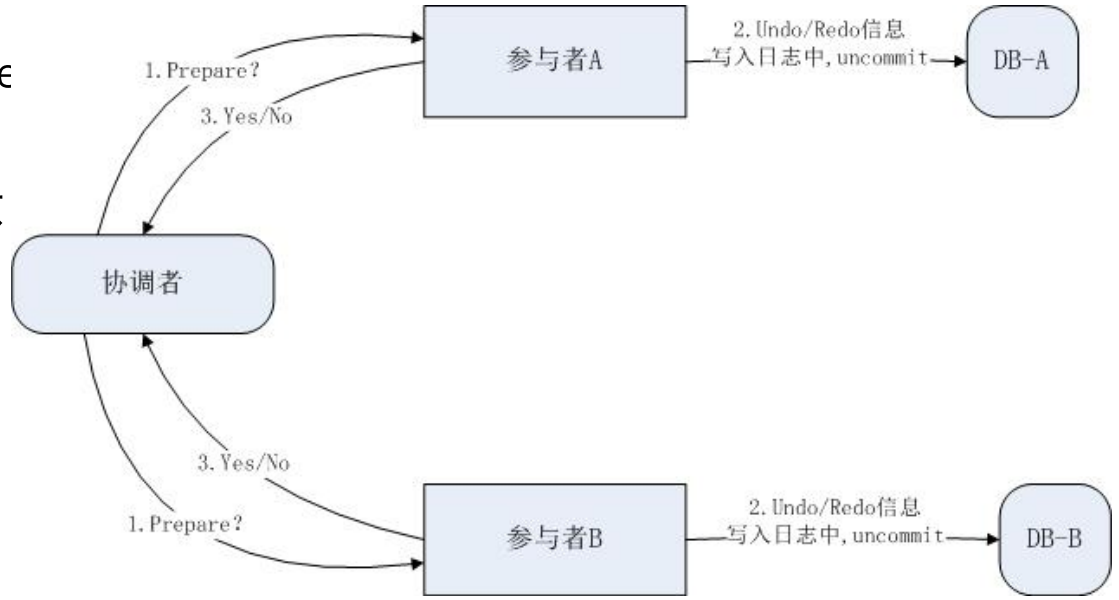
柔性分布式事务——TCC

- TCC（Try—Confirm—Cancel）分布式事务是一种柔性分布式事务，其注重于最终一致性而非强一致性
- TCC分布式事务最核心的思想就是在应用层将一个完整的事务操作分为Try、Confirm和Cancel三个阶段
  - Try：尝试执行业务，完成所有业务检查(一致性)，预留必要的业务资源(准隔离性)。
  - Confirm：确认执行业务，不再做业务检查。只使用Try阶段预留的业务资源，Confirm操作满足幂等性。
  - Cancel：取消执行业务释放Try阶段预留业务资源。



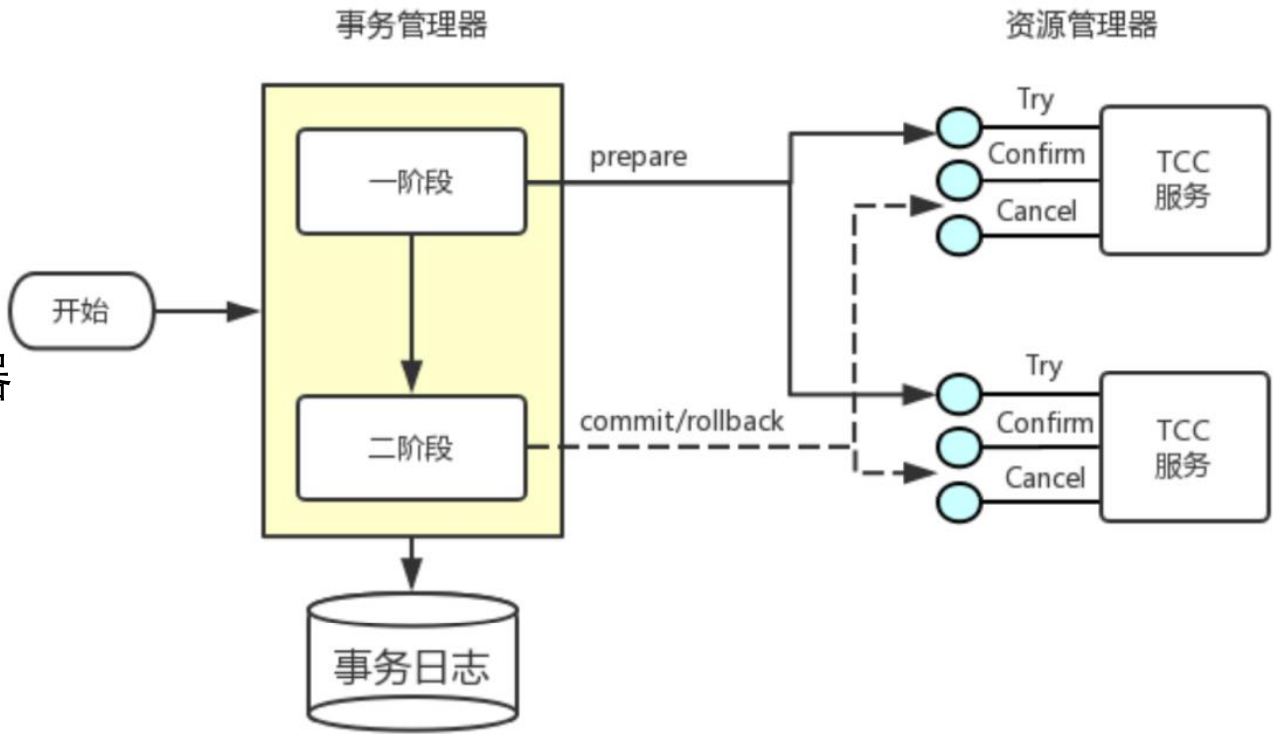
柔性分布式事务——TCC

- TCC与2PC（两阶段提交协议）的区别：
  - 2PC：
    - 准备阶段：事务协调者给每个参与者发送Prepare消息，每个参与者要么直接返回失败，要么在本地执行事务，写本地的redo和undo日志，但不提交
    - 提交阶段：如果协调者收到了参与者的失败消息或者超时，直接给每个参与者发送回滚(Rollback)消息；否则，发送提交(Commit)消息。参与者根据协调者的指令执行提交或者回滚操作，释放所有事务处理过程中使用的锁资源



柔性分布式事务——TCC

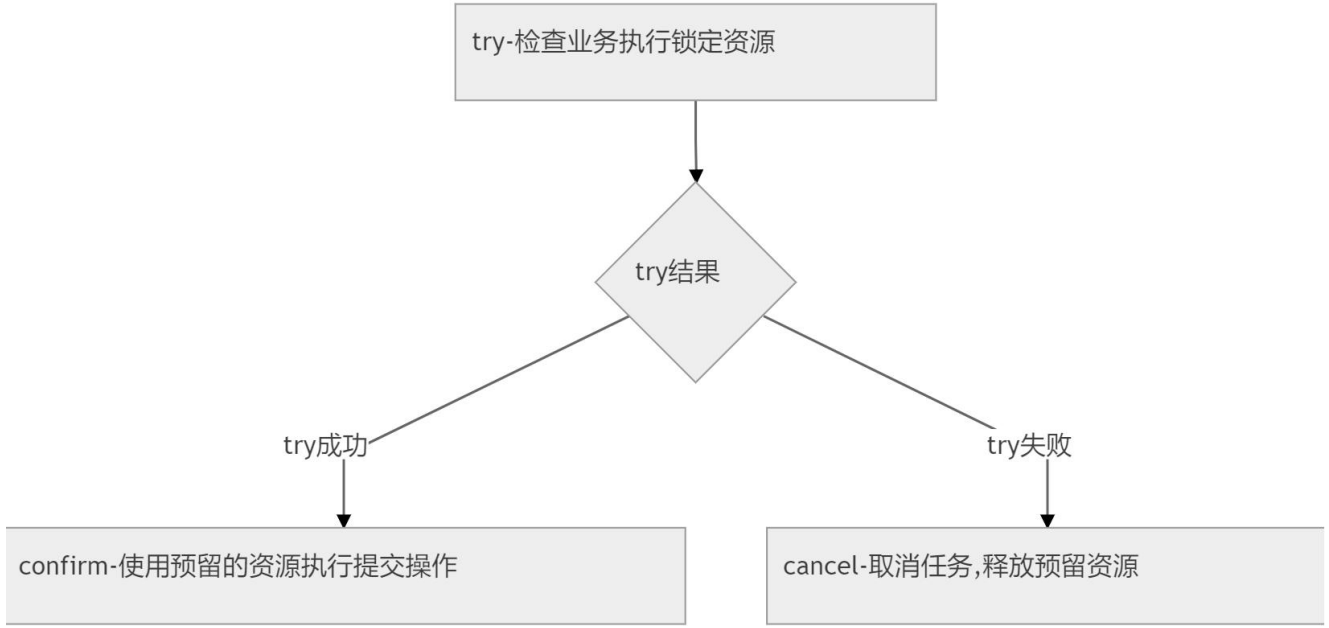
- TCC与2PC（两阶段提交协议）的区别：
  - TCC：
    - TCC是资源管理器的一种服务化的实现
    - 用户实现 TCC 服务之后，该 TCC 服务将作为分布式事务的其中一个资源，参与到整个分布式事务中；事务管理器分 2 阶段协调 TCC 服务，在第一阶段调用所有 TCC 服务的 Try 方法，在第二阶段执行所有 TCC 服务的 Confirm 或者 Cancel 方法；最终所有 TCC 服务要么全部都是提交的，要么全部都是回滚的



柔性分布式事务——TCC

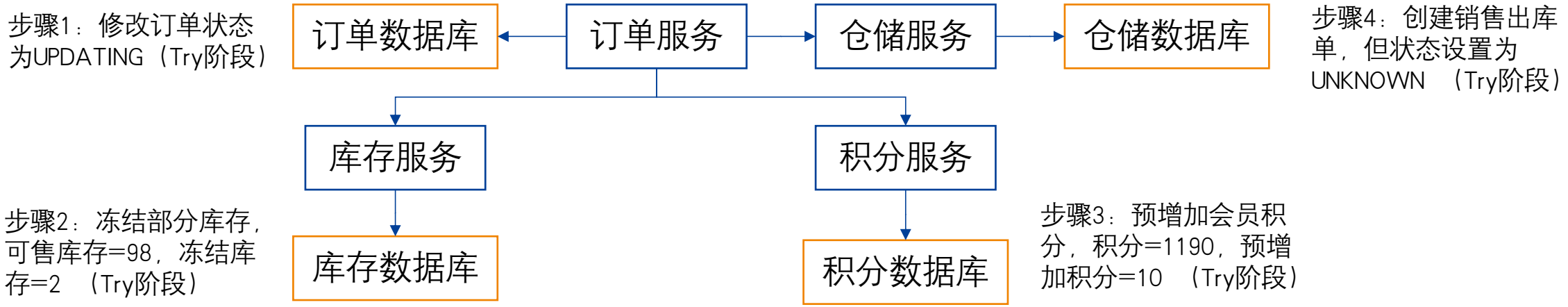
TCC与2PC（两阶段提交协议）的区别：

- TCC：
  - TCC中通过在Try阶段引入软状态（Soft State，允许中间状态出现）的概念，从而使得所有参与事务的节点各自需要在全局事务结束才能提交或者回滚的单一事务，划分成了Try阶段的事务和Confirm或者Cancel阶段的事务两个独立的事务，从而可以及时地对于公共资源进行放松，避免了其他访问公共资源的微服务的阻塞问题



TCC中的Try阶段

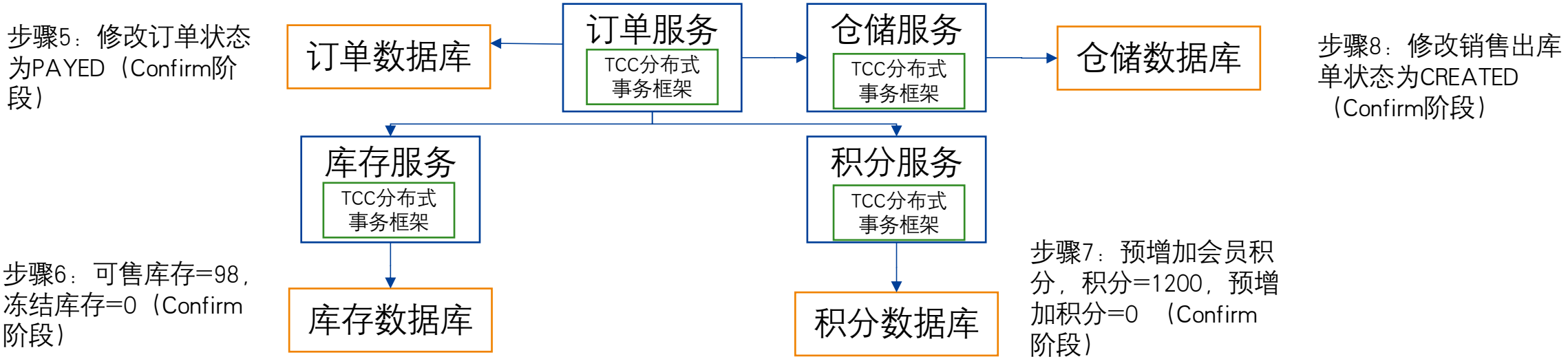
- Try阶段是TCC中最核心的阶段，在这个阶段尝试执行业务，重点关注如下事项：
  - 每个微服务完成各自业务检查，确保数据一致性
  - 每个微服务预留各自必要的业务资源，确保数据的隔离性



下单扣减库存业务场景中TCC的Try阶段

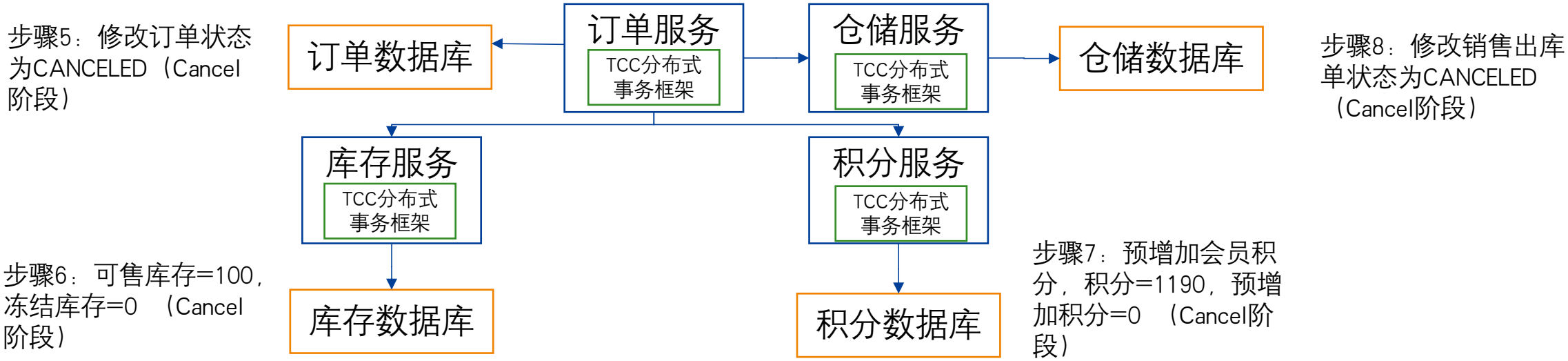
TCC中的Confirm阶段

- Confirm阶段是确认执行业务的阶段，在这个阶段确认执行的业务，此时重点关注如下事项：
  - 真正地执行业务
  - 不做任何业务逻辑检查，直接将数据持久化到数据库
  - 直接使用Try阶段预留的业务资源



TCC中的Cancel阶段

- Cancel阶段是取消执行业务的阶段，在这个阶段取消执行业务，重点关注如下事项：
  - 释放Try阶段预留的业务资源
  - 将数据库中的数据恢复到最初的状态



TCC核心组成

- 一个完整的TCC分布式事务需要包含三个部分：主业务服务、从业务服务和TCC管理器
- 主业务服务是TCC分布式事务的发起方，在下单扣减库存的业务场景中，订单服务是TCC分布式事务的发起方，就是主业务服务
- 从业务服务主要负责提供TCC业务操作，是整个业务活动的操作方。从业务服务必须实现Try、Confirm和Cancel三个阶段的接口， 在下单扣减库存的业务场景中，订单服务或者说负责创建订单的服务同时也是从业务服务，另外库存服务也是从业务服务
- TCC管理器在整个TCC分布式事务的执行过程中，管理并控制着整个事务活动，包括：
  - 记录并维护TCC全局事务的事务状态和每个从业务服务的分支事务状态
  - 在所有从业务服务的Try阶段都执行成功时，自动调用它们的Confirm阶段的操作
  - 当从业务服务的Try阶段或者Confirm阶段执行失败时，自动调用所有从业务服务的Cancel阶段的操作

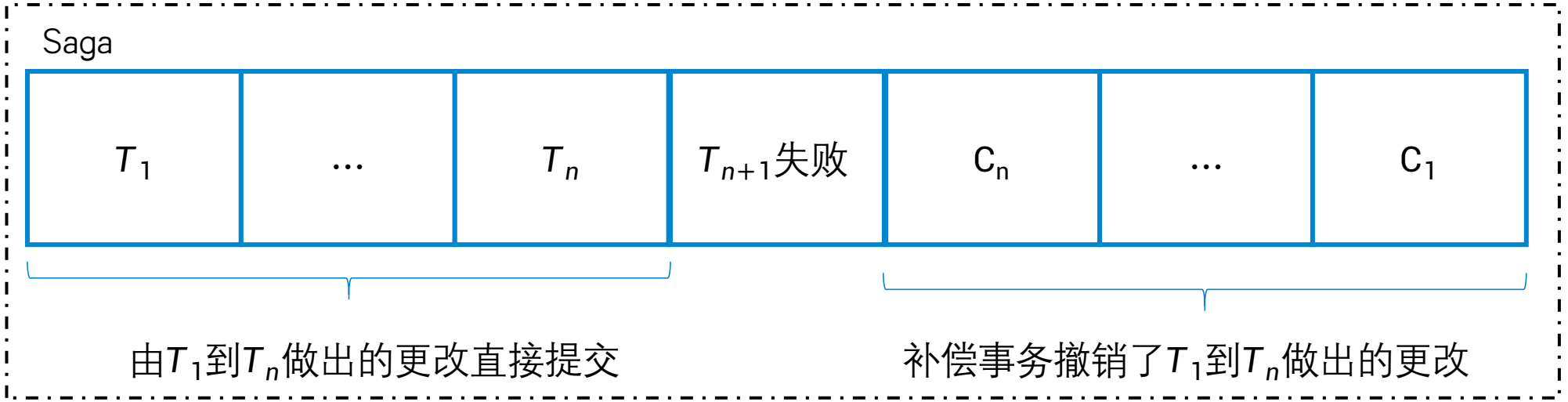
柔性分布式事务——Saga

- Saga是另外一种在微服务架构中维护数据一致性的机制
  - 1987年普林斯顿大学的Hector Garcia-Molina和Kenneth Salem发表了一篇Paper Sagas，讲述的是如何处理long lived transaction（长活事务）
  - Saga是一个长活事务可被分解成可以交错运行的子事务集合。其中每个子事务都是一个保持数据库一致性的真实事务
  - Saga 事务核心思想是将长事务拆分为多个本地短事务并依次正常提交，如果所有短事务均执行成功，那么分布式事务提交；如果出现某个参与者执行本地事务失败，则由 Saga 事务协调器协调根据相反顺序调用补偿操作，回滚已提交的参与者，使分布式事务回到最初始的状态

柔性分布式事务——Saga

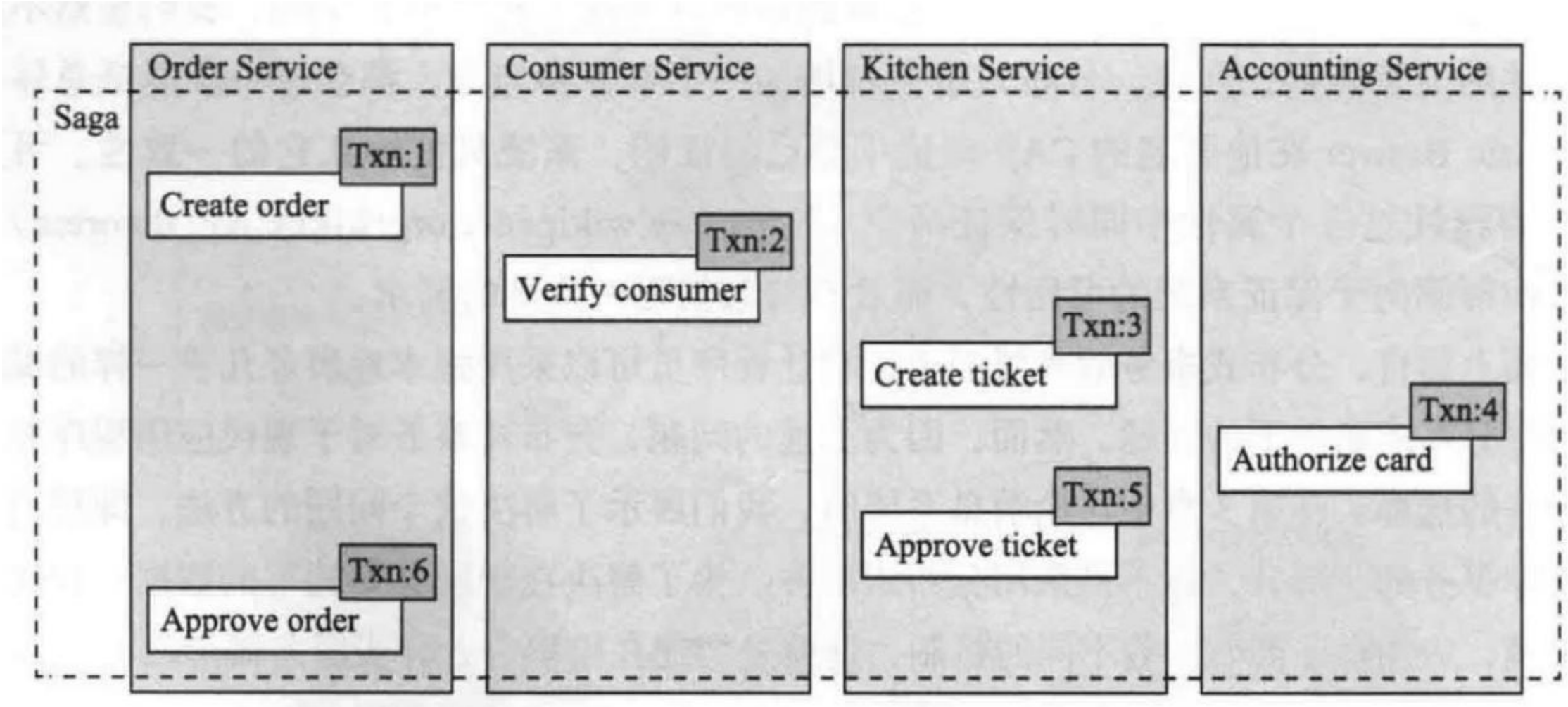
Saga事务链

- Saga 事务基本协议：
  - （1）每个 Saga 事务由一系列幂等的有序子事务Ti 组成。
  - （2）每个 Ti 都有对应的幂等补偿动作 Ci，补偿动作用于撤销 Ti 造成的结果。
- 假如一个 Saga 的分布式事务链有 n 个分支事务构成，[T1，T2，…，Tn]，那么该事务的执行情况有三种：
  - T1，T2，…，Tn：n 个事务全部执行成功了。
  - T1，T2，…，Ti，Ci，…，C2，C1：执行到第 i（i<=n）个事务的时候失败了，则按照 i->1 的顺序依次调用补偿操作。如果补偿失败了，就一直重试。补偿操作可以优化为并行执行。
  - T1，T2，…，Ti（失败），Ti（重试），Ti（重试），…，Tn：适用于事务必须成功的场景，如果发生失败了就一直重试，不会执行补偿操作。



柔性分布式事务——Saga

- Create Oder Saga

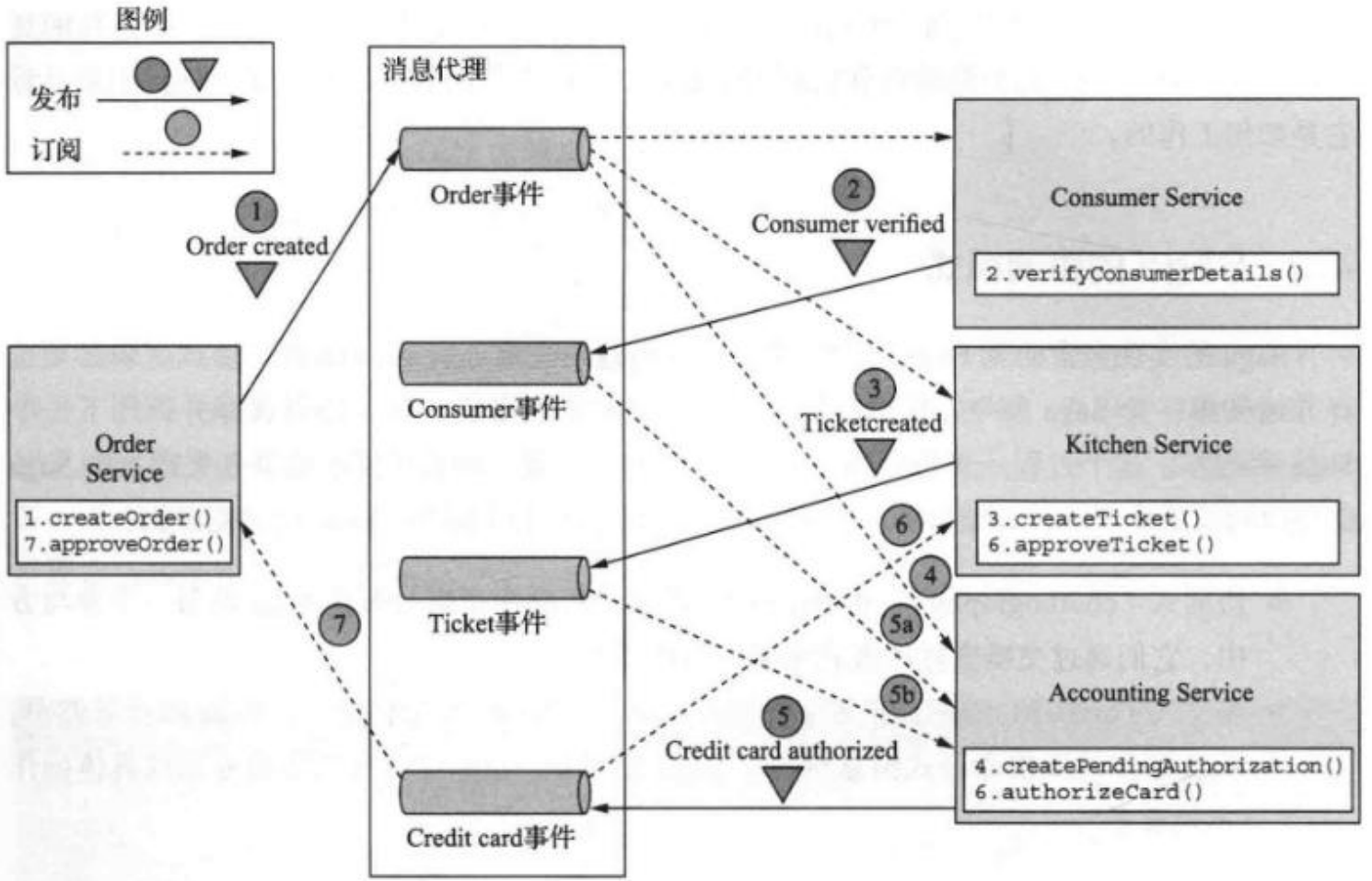


柔性分布式事务——Saga

- 相较于TCC，Saga有以下几点不同：
  - 少了一步Try的操作，TCC无论最终事务成功失败都需要与事务参与方交互两次。而Saga在事务成功的情况下只需要与事务参与方交互一次，如果事务失败，需要交互两次
  - 更关注于长事务，适用于业务流程长且需要保证事务最终一致性的业务系统，Saga 模式在一阶段就会提交本地事务，在长流程情况下可以保证较好的性能
  - Saga事务参与者可能是其它公司的服务或者是遗留系统的服务，无法进行改造和提供 TCC 要求的接口

柔性分布式事务——Saga

- Saga的实现包含协调Saga步骤的逻辑。当通过系统命令启动Saga时，协调逻辑必须选择并通知第一个Saga参与方执行本地事务。一旦该事务完成，Saga协调选择并调用下一个Saga参与方直到执行完所有步骤，如果任何本地事务失败，则Saga必须以相反的顺序执行补偿事务



使用协同式实现 Create order saga。Saga参与方通过交换事件进行沟通

柔性分布式事务——Saga

- Saga构建协调逻辑主要有以下两种方式：
  - 协同式：把Saga的决策和执行顺序逻辑分布在Saga的每一个参与方中，它们通过交换事件的方式来进行沟通
  - 编排式：把Saga的决策和执行顺序逻辑集中在一个Saga编排器类中，Saga编排器发出命令式消息给各个Saga参与方，指示它们完成具体操作（本地事务）

柔性分布式事务——Saga

- 使用Saga的挑战在于Saga事务模型是ACD的，它缺乏ACID(原子性、一致性、隔离性和持久性)事务的隔离属性，在TCC中该属性通过软状态来实现，但是在Saga中一旦事务提交每个Saga的本地事务所做的更新都会立即被其他Sagas看到，从而导致两个问题：
  - 其他Saga可以在执行时更改该Saga所访问的数据
  - 其他Saga可以在Saga完成更新之前读取其数据从而导致不一致的数据

柔性分布式事务——Saga

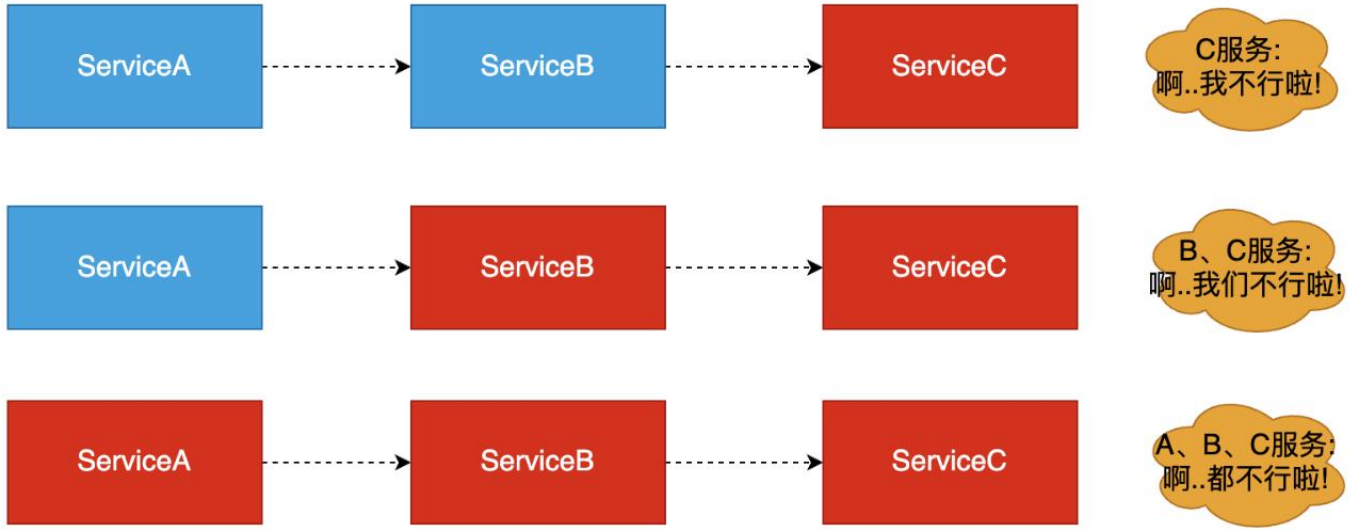
- 目前主流的解决方案是通过语义锁的方式解决：
- 类似于TCC中的软状态，开发人员在每个数据中使用*_PENDING的状态记录数据的当前状态，从而告诉其他Saga事务该数据正在被一个Saga更新，并在Saga事务的最后为之前的每一个业务事务增加对应的保证成功的可重复性事务来将状态更新为FINISHED从而结束Saga事务

3.3 容错设计及流量控制

- “容错性设计”（Design for Failure）是微服务的另一个核心原则，随着将应用程序拆分出的服务越来越多，微服务间的通信随之而来会面临以下两个问题：
  - 雪崩效应：由于某一个服务的崩溃，导致所有用到这个服务的其他服务都无法正常工作。
  - 服务超时：服务虽然没有崩溃，但由于处理能力有限，面临超过预期的突发请求时，大部分**请求直至超时都无法完成处理**。
- 流量治理包含服务容错模式及流量控制模式。

3.3 容错设计及流量控制

- 由于服务与服务之间的依赖性，故障会传播，会对整个微服务系统造成灾难性的严重后果，这就是服务故障的“雪崩效应”
- 造成雪崩效应的原因：
  - 流量激增：比如异常流量、用户重试导致系统负载升高
  - 缓存刷新：client向Server的请求超出了Server的承载能力，就会造成Server崩溃
  - 程序有Bug：代码循环调用的逻辑问题，资源未释放引起的内存泄漏等问题
  - 硬件故障：比如宕机，机房断电，光纤被挖断等
  - 线程同步等待：如果一个核心业务线程调用非核心线程，这个非核心线程交由第三方系统完成，当第三方系统本身出现问题，导致核心线程阻塞，一直处于等待状态，而进程间的调用是有超时限制的，最终这条线程将断掉，也可能引发雪崩



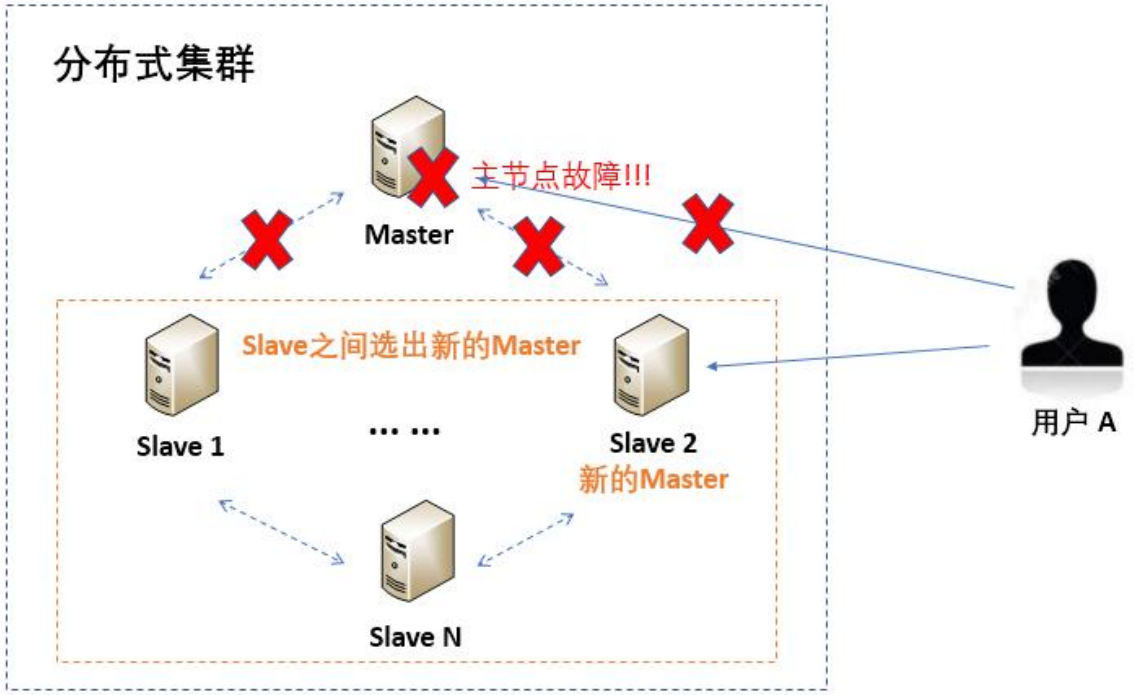
（1）服务容错策略

容错策略	优点	缺点	应用场景
故障转移	系统自动处理，调用者对失败的信息不可见	增加调用时间，额外的资源开销	调用幂等服务 对调用时间不敏感的场景
快速失败	调用者有对失败的处理完全控制权 不依赖服务的幂等性	调用者必须正确处理失败逻辑，如果一味只是对外抛异常，容易引起雪崩	调用非幂等的服务 超时阈值较低的场景
沉默失败	控制错误不影响全局	出错的地方将在一段时间内不可用	频繁超时的服务
故障恢复	调用失败后自动重试，也不影响主路逻辑	重试任务可能产生堆积，重试仍然可能失败	调用链中的旁路服务 对实时性要求不高的主路逻辑也可以使用
并行调用	尽可能在最短时间内获得最高的成功率	额外消耗机器资源，大部分调用可能都是无用功	资源充足且对失败容忍度低的场景

（1）服务容错策略

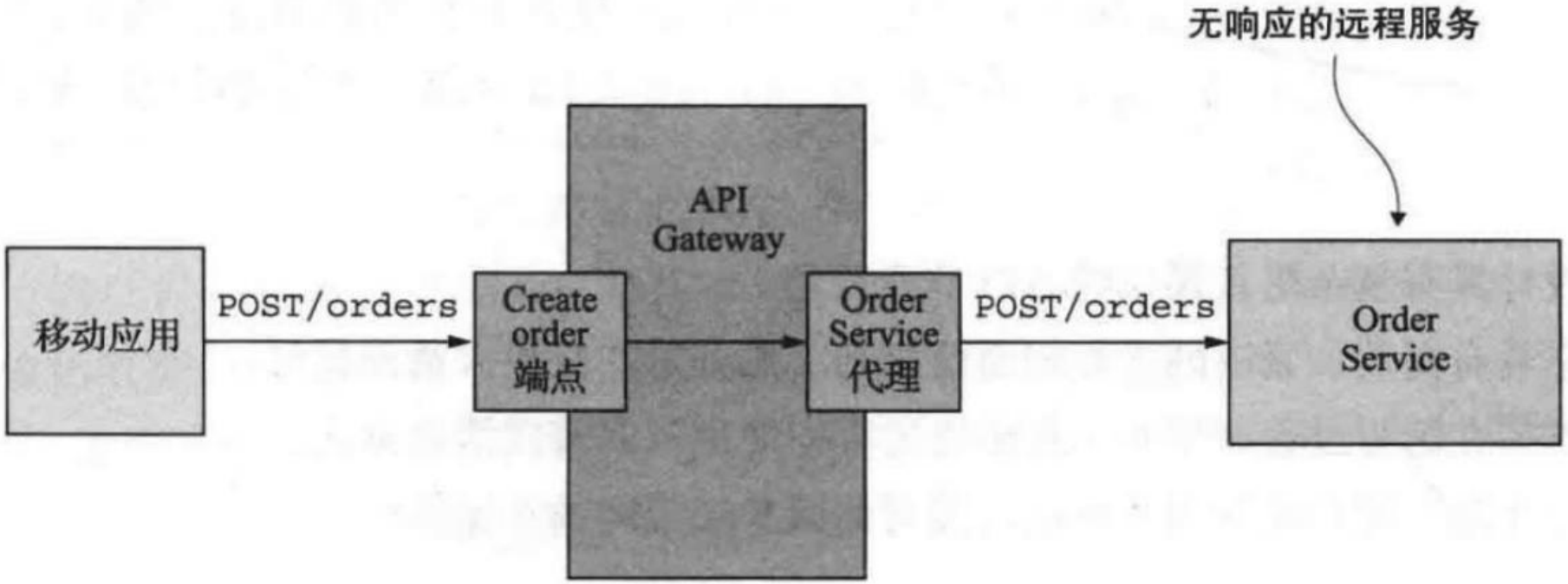
➢ 故障转移

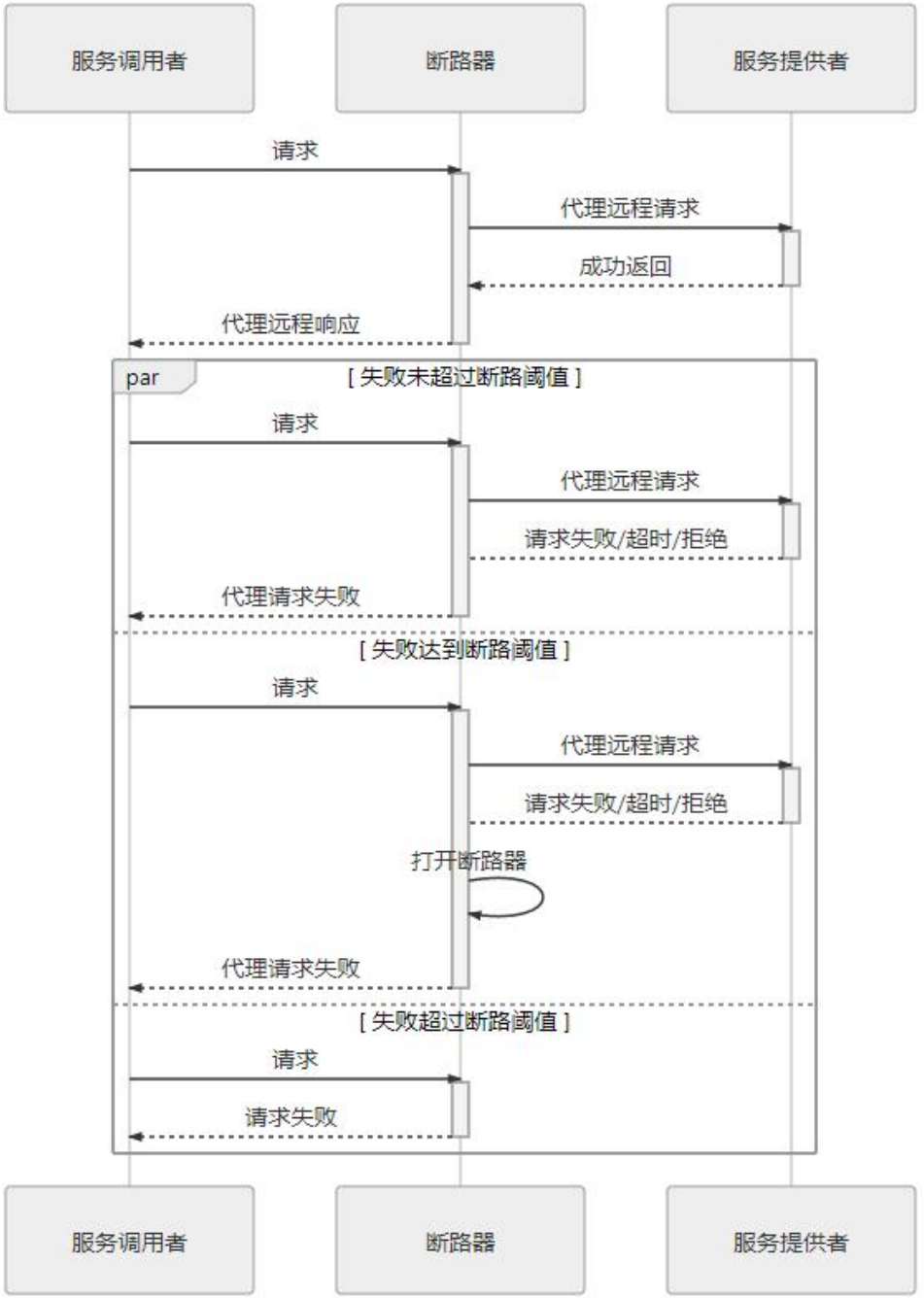
- 对于单节点故障问题，往往采取主备策略，即当主节点故障后，从备节点中选出一个作为新的主节点，以继续提供服务。
  - 用户 A 访问分布式集群时一直是与 Master 交互的，但当 Master 故障后，其他 Slave 会通过分布式选举算法选出一个新的主节点。
  - 假设，从 Slave 1、Slave 2 和 Slave 3 中选举出 Slave 2 作为新的 Master，则 Slave 2 需要承担原来 Master 的职责，继续为用户提供服务，因此当用户 A 再次访问集群时，提供服务的是新选出的 Master，也就是 Slave 2。这就是备升主的过程。



容错设计模式——断路器模式

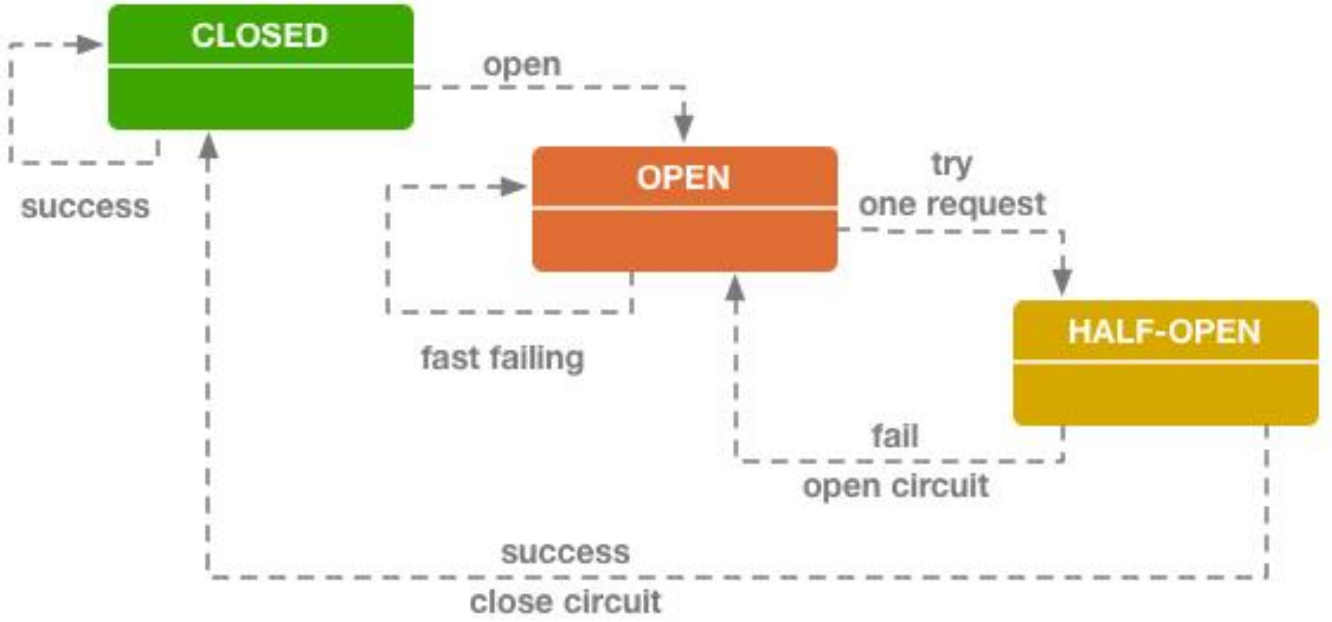
- 断路器的基本思路就是通过代理（断路器对象）来一对一地（一个远程服务对应一个断路器对象）接管服务调用者的远程请求。
- 断路器会持续监控并统计服务返回的成功、失败、超时、拒绝等各种结果，当出现故障（失败、超时、拒绝）的次数达到断路器的阈值时，它状态就自动变为“OPEN”，后续此断路器代理的远程访问都将直接返回调用失败，而不会发出真正的远程服务请求直到过了一定的时限再尝试关闭断路器。




服务调用者
断路器
服务提供者
请求
代理远程请求
成功返回
代理远程响应
par
[失败未超过断路阈值]
请求
代理远程请求
请求失败/超时/拒绝
代理请求失败
[失败达到断路阈值]
请求
代理远程请求
请求失败/超时/拒绝
打开断路器
代理请求失败
[失败超过断路阈值]
请求
请求失败
服务调用者
断路器
服务提供者

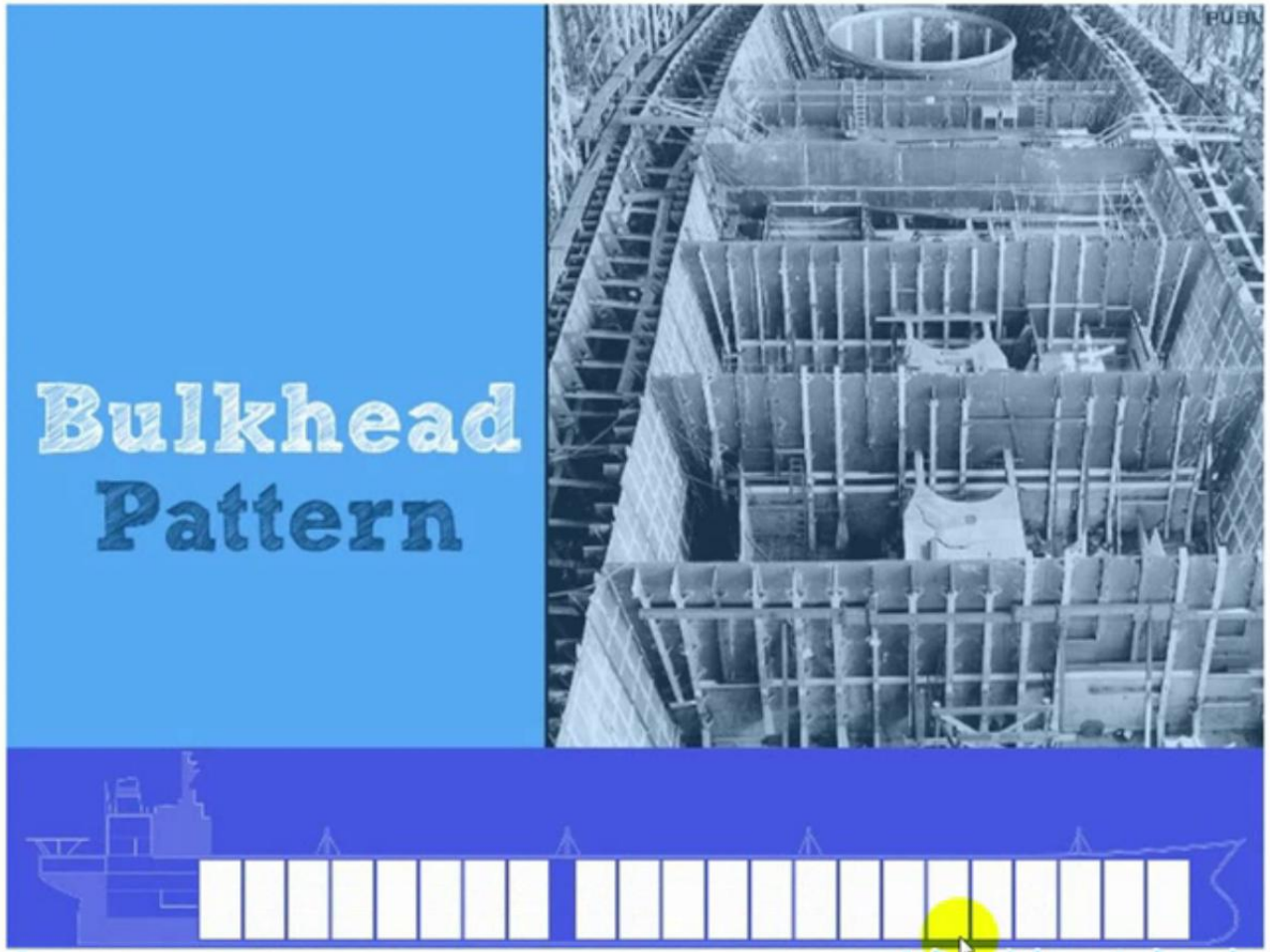
容错设计模式——如何实现断路器模式

- 从调用序列来看，断路器就是一种有限状态机，断路器模式就是根据自身状态变化自动调整代理请求策略的过程。一般要设置以下三种断路器的状态：
  - CLOSED：表示断路器关闭，此时的远程请求会真正发送给服务提供者。
  - OPEN：表示断路器开启，此时不会进行远程请求，直接给服务调用者返回调用失败的信息，以实现快速失败策略。
  - HALF OPEN：这是一种中间状态。当进入 OPEN 状态一段时间以后，将自动切换到 HALF OPEN 状态。该状态下，会放行一次远程调用，然后根据这次调用的结果成功与否，转换为 CLOSED 或者 OPEN 状态，以实现断路器的弹性恢复。



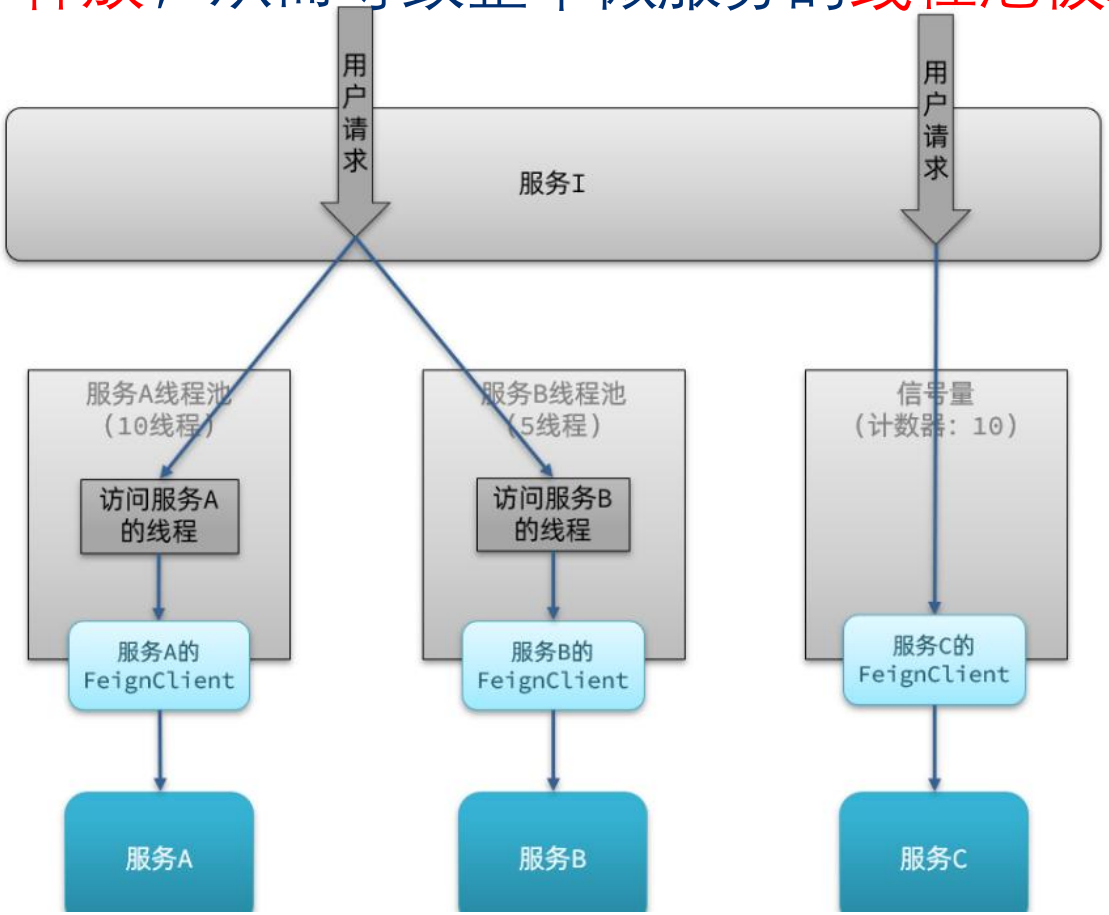
容错设计模式——什么是舱壁隔离模式

- 舱壁隔离模式是常用的实现服务隔离的设计模式，舱壁这个词是来自造船业的舶来品，它原本的意思是设计舰船时，要在每个区域设计独立的水密舱室，一旦某个舱室进水，也只是影响这个舱室中的货物，而不至于让整艘舰艇沉没。



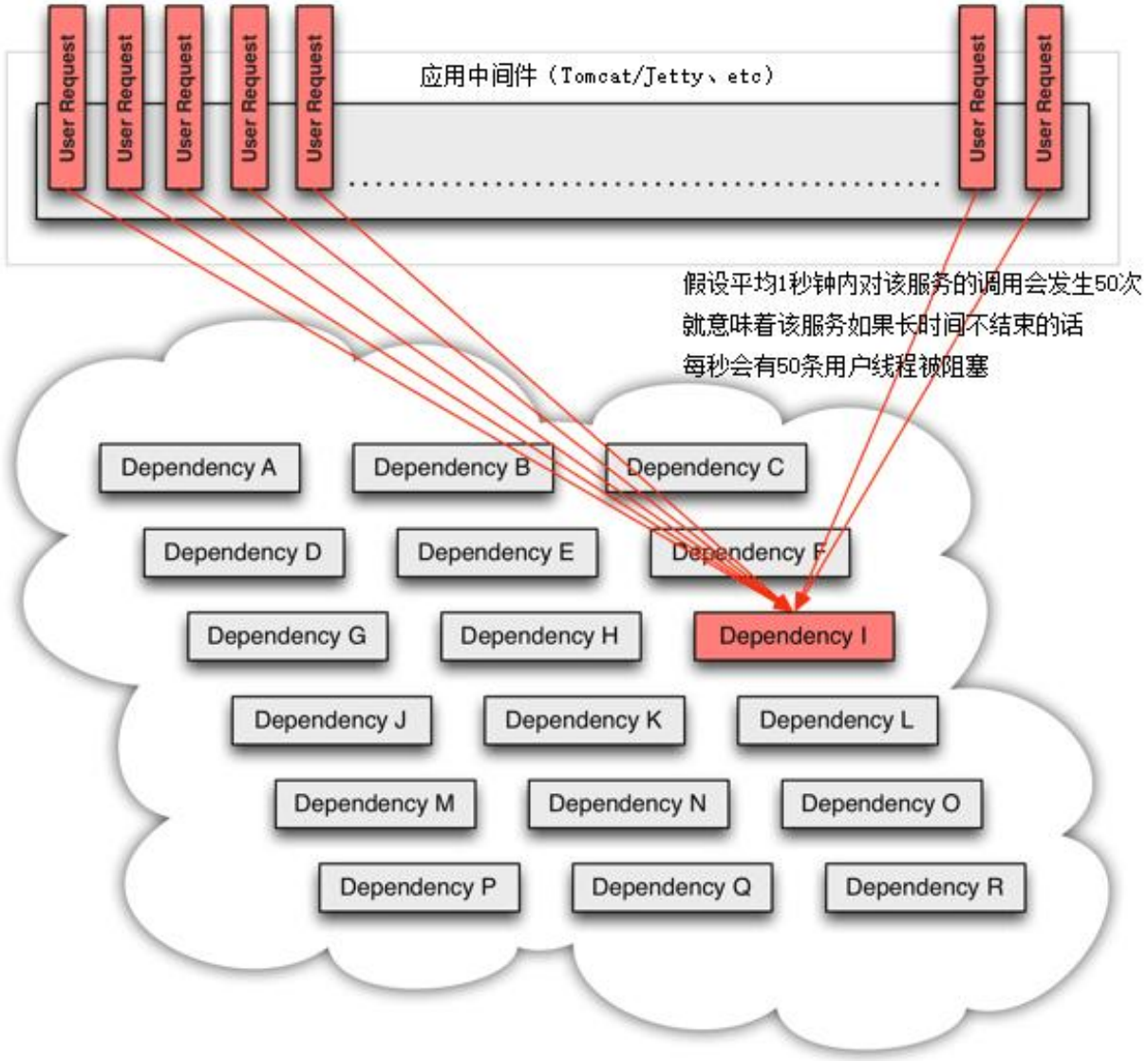
容错设计模式——什么是舱壁隔离模式

- 在微服务架构的容错设计模式中，舱壁隔离模式往往为每一类的微服务间通信调用设置一个对应的线程池，从而防止某一类的微服务间通信调用由于长时间不返回而一直占用线程不释放，从而导致整个微服务的线程池被耗尽的全局性影响



给每个服务调用业务分配一个线程池，利用线程池本身实现隔离效果

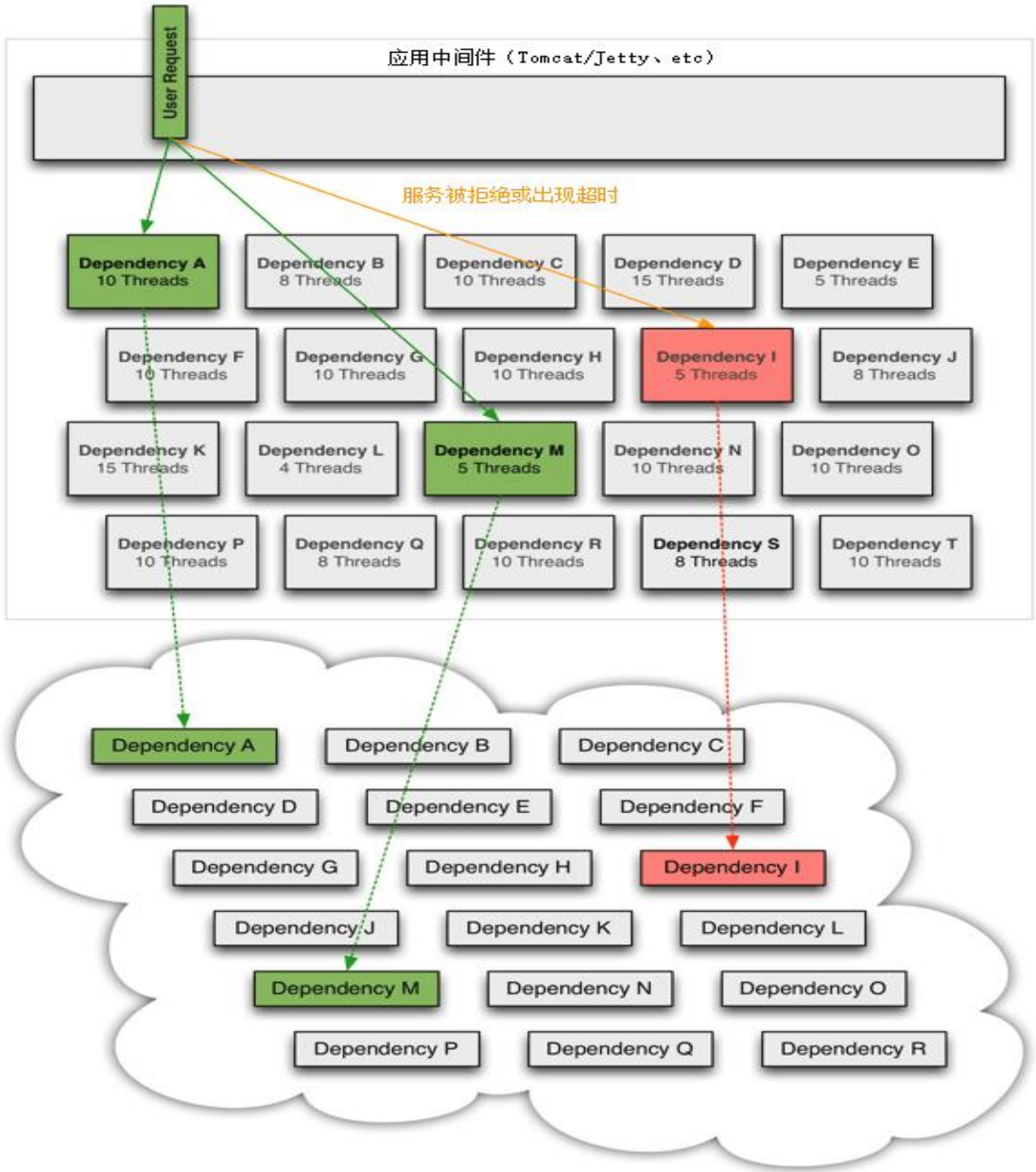
容错设计模式——为什么使用舱壁隔离模式



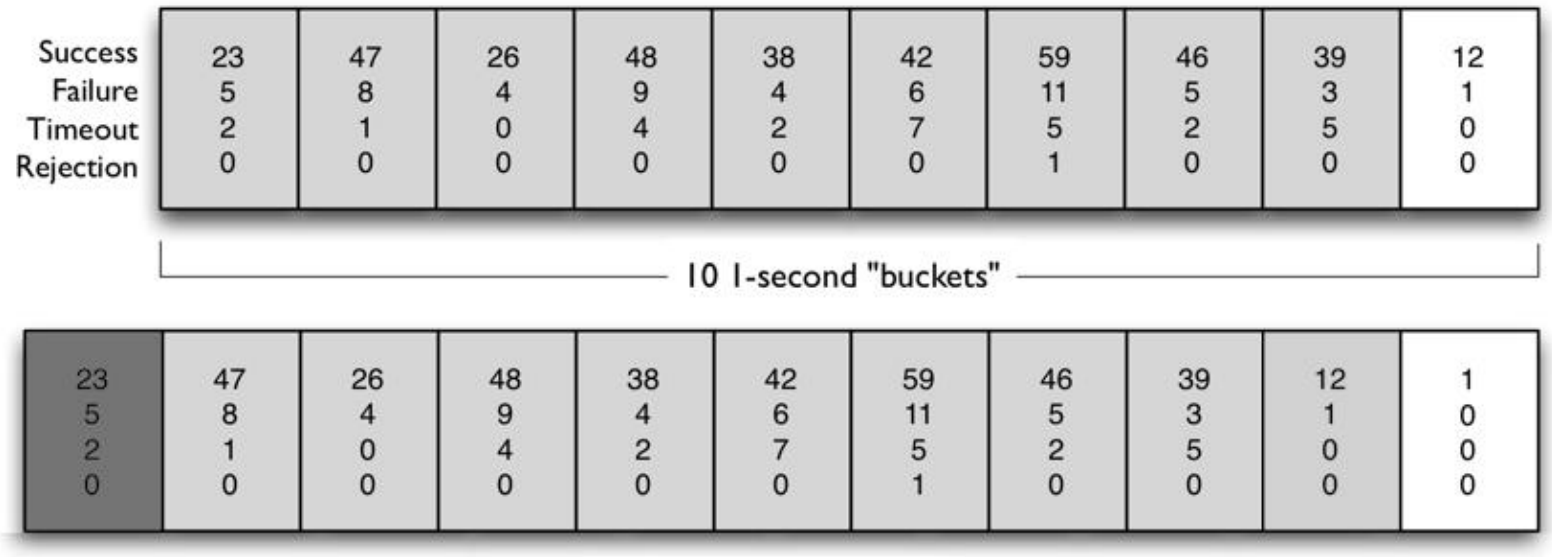
- 在没有使用舱壁隔离模式情况下，由于单类微服务间通信的长时间线程占导致整个微服务线程池资源耗尽

容错设计模式——如何使用舱壁隔离模式

- 通过使用舱壁隔离模式，给每一类微服务间通信配置线程池，从而防止整个微服务线程池资源耗尽



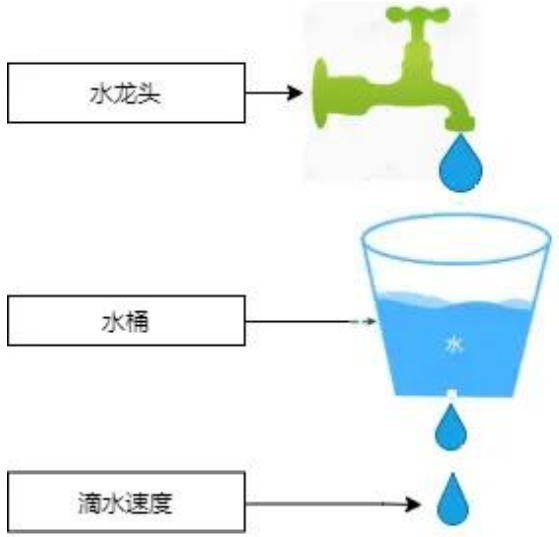
（2）流量控制模式——滑动时间窗

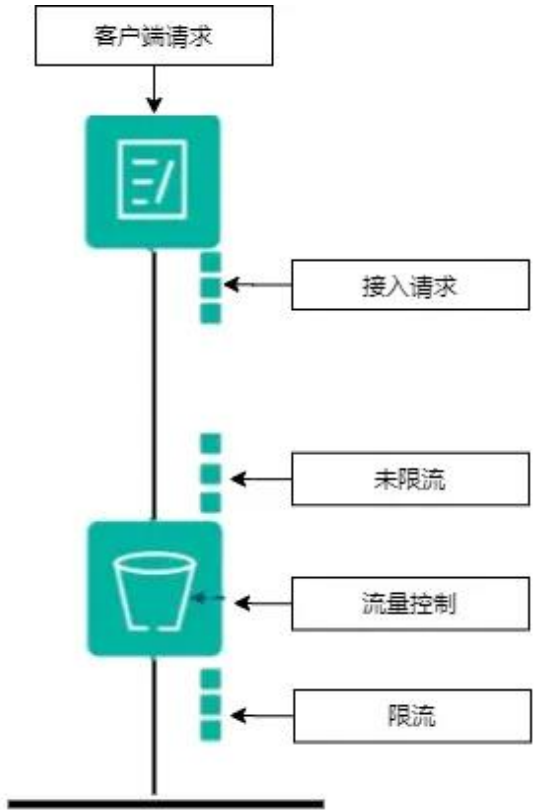


- 在滑动时间窗中会设置频率固定为N秒一次的定时器，每次定时器唤醒时滑窗将完成以下几项工作过程：
  - 将数组最后一位的元素丢弃掉，并把所有元素都后移一位，然后在数组第一个插入一个新的空元素。这个步骤即为“滑动窗口”。
  - 将计数器中所有统计信息写入到第一位的空元素中。
  - 对数组中所有元素进行统计，并复位清空计数器数据供下一个统计周期使用

流量控制模式——漏桶模式

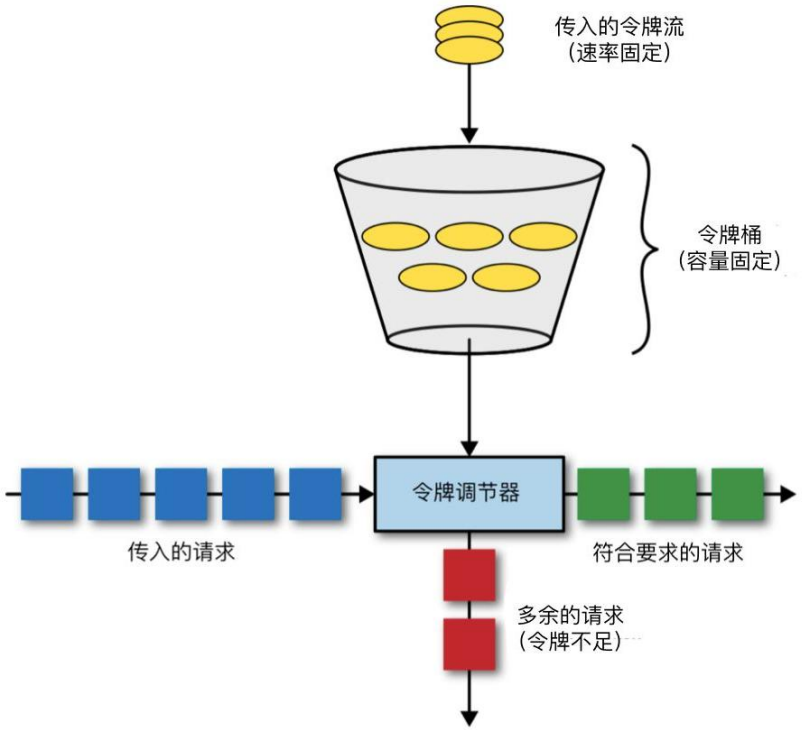
- 漏桶在代码实现上非常简单，它其实就是一个以请求对象作为元素的先入先出队列（FIFO Queue），队列长度就相当于漏桶的大小，当队列已满时便拒绝新的请求进入。
- 漏桶实现起来很容易，困难在于如何确定漏桶的两个参数：桶的大小和水的流出速率。如果桶设置得太大，那服务依然可能遭遇到流量过大的冲击，不能完全发挥限流的作用；如果设置得太小，那很可能就会误杀掉一部分正常的请求





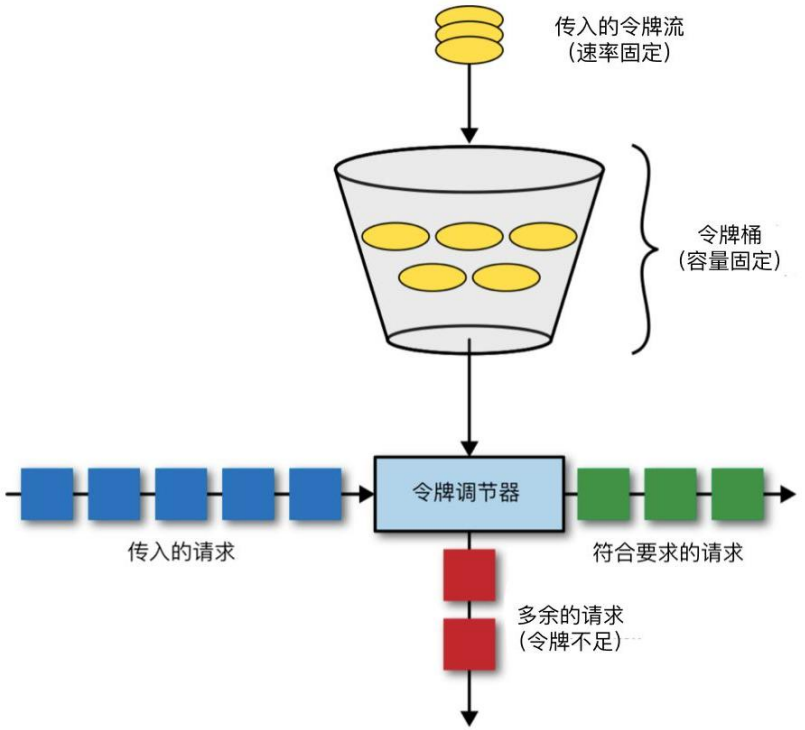
流量控制模式——令牌桶模式

- 令牌桶和漏桶一样都是基于缓冲区的限流算法，只是方向刚好相反，漏桶是从筒里往系统出水，令牌桶则是系统往筒中放入令牌
- 假设要限制系统在 X 秒内最大请求次数不超过 Y，那就每间隔 X/Y 时间就往桶中放一个令牌，当有请求进来时，首先要从桶中取得一个准入的令牌，然后才能进入系统处理。任何时候，一旦请求进入桶中却发现没有令牌可取了，就应该马上失败或进入服务降级逻辑
- 与漏桶类似，令牌桶同样有最大容量，这意味着当系统比较空闲时，桶中令牌累积到一定程度就不再无限增加，预存在桶中的令牌便是请求最大缓冲的余量



流量控制模式——令牌桶模式

- 令牌桶和漏桶的区别：
  - 漏桶算法的出水速度是恒定的，那么意味着如果瞬时大流量的话，将有大部分请求被丢弃掉（也就是所谓的溢出）。漏桶算法通常可以用于限制访问外部接口的流量，保护其他人系统，比如我们请求银行接口，通常要限制并发数。
  - 令牌桶算法生成令牌的速度是恒定的，而请求去拿令牌是没有速度限制的。这意味，面对瞬时大流量，该算法可以在短时间内请求拿到大量令牌，可以处理瞬时流量，而且拿令牌的过程并不是消耗很大的事情。令牌桶算法通常可以用于限制被访问的流量，保护自身系统。



目录

- 1 微服务设计概述
- 2 微服务架构设计
  - 定义系统操作 – 定义业务服务 – 定义服务API
- 3 微服务架构下的进程间通信
  - 服务间通信 – 分布式事务 – 容错设计及流量控制
- 4 小结

5 小结

- 阐述了微服务架构设计的方法论以及三段式方法，包括：定义系统操作、定义服务以及定义服务API。
- 基于微服务间通信、分布式事务以及流量治理三个方面阐述了微服务架构下的进程间通信模式。